Managing by Feedback

Jinci Liu *

October 25, 2026

Latest Version

Abstract

Managers give feedback, but does how they say it affect worker productivity beyond what they say? Using GitHub and LinkedIn data, I analyze over 200 million code-review messages and use large language models to distinguish feedback tone (toxicity and positivity) from informational content (constructiveness). Exploiting quasi-random reviewer assignment, I find that toxic feedback reduces workers' subsequent code output by 43% and lowers its quality, whereas nontoxic criticism has no such detrimental effects. Positive feedback improves code quality and retention, with benefits spilling over to coworkers. Constructive feedback instead reallocates effort from new code production to revision. Feedback style varies persistently across reviewers, and these differences predict 22% of the variation in reviewer value-added. These differences may partly reflect prior experience: reviewers previously exposed to toxic feedback are more likely to provide it themselves.

JEL Codes: M5, J24, J81, L86

Keywords: Feedback, worker productivity, manager quality, toxicity

*National University of Singapore. Email: jinci.liu@nus.edu.sg.

I am grateful to Arash Nekoei, Mitch Downey, and Jósef Sigurdsson for their encouragement and feedback—always positive and never toxic. I thank Jakob Beuschlein, Kirill Borusyak, Konrad Burchardi, Stefano DellaVigna, Florian Englmaier, Ingrid Haegele, Oskar Nordström Skans, Ricardo Perez-Truglia, David Schönholzer, Mateusz Stalinski, David Strömberg, and Horng Chern Wong as well as seminar participants at IIES, UPF, LSE, CUHK, HKUST, CityUHK, UNSW, UIUC, NUS, SMU, the HK Labor Symposium, CSEF-CEPR Labor & Finance, and CEPR IMO & ESF. I am also thankful to Peishan Ju and Ruipu Peng for research assistance in validating the LLM classifications. I acknowledge financial support from Google Cloud Research, the Mannerfelt Foundation, and the Institute for Evaluation of Labour Market and Education Policy (IFAU).

1 Introduction

Managers matter for worker productivity, but less is known about *how* they affect it. Recent work has begun to open this black box by studying managerial evaluations (Frederiksen et al., 2020), task allocation (Minni, 2026), talent hoarding (Haegele, 2026), and people management skills (Weidmann et al., 2026). One of managers' most common activities is giving feedback: more than 90% of managerial occupations require it (O*NET, 2025). Yet whether and how feedback improves productivity remain unclear. Existing evidence often relies on workers' reported perceptions, which do not establish whether feedback raises productivity.[1]

Feedback combines information about the work with a tone that may affect workers' motivation. These two components can have opposing effects. When a manager asks a worker to revise a task, a toxic tone may undermine confidence and reduce effort (Compte and Postlewaite, 2004), or increase effort by raising the perceived threat of punishment (Mirrlees, 1974, 1976). A positive tone may reinforce workers' beliefs about their ability and sustain effort (Bénabou and Tirole, 2003), or reduce effort by signaling that the work already meets the required standard (Murphy and Cleveland, 1991). The effect of feedback therefore depends both on what it conveys and on how it is delivered.

The distinction between toxic feedback and respectful criticism (non-toxic and negative) is especially important. Economists have documented the costs of severe workplace mistreatment, including workplace violence (Adams-Prassl et al., 2024), sexual harassment (Folke and Rickne, 2022), and hostile work environments (Collis and Van Effenterre, 2025). Toxic feedback is less severe, more common, and less visible; whether it carries similar productivity costs is unknown. A manager can criticize work without using toxic language, so identifying the cost of tone requires separating toxic feedback from non-toxic criticism. This paper asks whether positive or toxic feedback affects worker productivity beyond the information the feedback conveys.

Three empirical challenges make these effects difficult to identify. First, feedback is often verbal and unrecorded. Second, even when written feedback is available, its multidimensional nature makes it difficult to identify the relevant dimensions and measure them at scale. Third, feedback may reflect workers' unobserved ability or prior performance, so its correlation with productivity may reflect selection or reverse causality.

I address these challenges using code review on GitHub, where feedback is recorded, and developer productivity can be measured. GitHub is the world's largest online coding platform and is used by more than 90% of Fortune 100 companies (GitHub, 2025). In code review, *developers* submit proposed code changes through pull requests for inclusion into the team's codebase. *Reviewers* evaluate each submission, provide feedback, and decide whether to approve it. Developers must complete the revisions required by reviewers to obtain approval, so they cannot simply disregard this feedback as they might informal peer feedback. Code review thus combines feedback with authority over task completion, allowing me to study a managerial function at the level of

[1] In one survey, 80% of workers reported feeling more engaged after receiving "meaningful" feedback (Gallup, 2022). In another, only 14% strongly agreed that the feedback they received was helpful (Sutton and Wigert, 2019).

individual interactions.

I construct a dataset from the GitHub Archive, focusing on organizational software teams and feedback exchanged among team members. The dataset covers 525,841 teams and 204 million feedback messages from 2017 to 2023. I measure weekly developer productivity along two dimensions: code quantity (lines of code) and code quality (the share of submitted code accepted without revision). I also track developers' LinkedIn career histories from Revelio Labs, which lets me follow their careers beyond the platform: whether they stay with their employer and whether they move into a different occupation.

I classify feedback using language models. Guided by the seminal work in psychology (Ilgen et al., 1979), I distinguish feedback tone from informational content. For tone, I measure two separate dimensions: toxicity, which captures insults, profanity, aggression, or discriminatory language, and positivity, which captures encouragement or acknowledgment. A central distinction is between toxic feedback and respectful criticism (non-toxic and negative), since a manager can deliver a negative evaluation without toxic language. I measure informational content through constructiveness, defined as specific, actionable guidance given the code under review. Each message receives a separate binary label on each dimension.[2] These measures allow me to examine whether tone matters beyond measured informational content and whether toxic feedback has different effects from respectful criticism.

To identify the causal effects of feedback, I use teams that assign reviewers automatically, limiting developers' ability to choose reviewers based on anticipated feedback or approval decisions. I identify these teams through assignment configurations in their codebases. They are larger and more active than other teams, so the estimates are most relevant to high-volume professional software teams. Within these teams, automatic assignment provides a source of plausibly exogenous variation in reviewer feedback style. However, the feedback a reviewer provides on a particular submission may still depend on its underlying quality. To address this, I instrument each feedback dimension with the assigned reviewer's baseline tendency to provide that type of feedback to other developers. This serves as an instrument, as it is highly predictive of the reviewer's feedback style in the current case but, as I document, uncorrelated with observable developer and case characteristics. The strategy is analogous to judge-leniency designs, which exploit differences in judges' tendencies to make particular decisions under quasi-random case assignment (Dobbie and Song, 2015; Bhuller et al., 2020; Agan et al., 2023).

The results show that feedback tone affects subsequent productivity and retention, while informational content redirects effort toward revision. Developers who receive toxic feedback produce 42.9% less new code for their teams over the following month.[3] They do not offset this decline

[2]The labels are toxic or non-toxic, positive or non-positive, and constructive or non-constructive. I distinguish non-toxic criticism from the broader category of non-positive feedback. For example, "*What's wrong with you to write such bad code?*" is classified as toxic, whereas "*This is bad code*" is classified as non-toxic criticism. Section 3 describes the measures and their validation.

[3]I measure subsequent code rather than the reviewed submission to avoid effects mechanically linked to the review process. For example, toxic feedback may accompany rejection, directly lowering acceptance-based measures of the reviewed submission's quality.

by shifting to non-coding tasks, contributing more to other teams. Nor do they rewrite more of the submission that received the feedback. The number of new submissions does not change, but developers contribute less code per submission, consistent with reduced engagement. The acceptance rate of their new submissions also falls, indicating that lower quantity is not offset by higher quality. These findings are robust to controlling for reviewer strictness, measured by reviewers' propensity to approve submitted code, and other observed reviewer characteristics.

The adverse effects are substantially larger for toxic feedback than for non-toxic criticism. Non-toxic criticism has no statistically significant effect on code quantity, and its negative effect on code quality is much smaller. These differences suggest that the productivity costs of toxic feedback extend beyond those of an unfavorable evaluation.

Positive feedback increases code quality by 6.4%, with no statistically significant change in code quantity. The gains are larger for developers whose prior code quality is above the median. This pattern is consistent with models in which positive feedback sustains effort by reinforcing workers' beliefs about their ability (Bénabou and Tirole, 2002; Compte and Postlewaite, 2004; Kőszegi et al., 2022), although I do not observe beliefs directly. Positive feedback also has effects beyond coding: developers send more positive messages to coworkers and are less likely to leave their employer over the following year, but are no more likely to switch occupations.

Constructive feedback has a different effect on how developers allocate effort. It has no statistically significant effect on the quantity of new code but lowers the acceptance rate of new submissions. At the same time, developers rewrite 49.2% more lines of the reviewed submission. Because reviewers can require revisions before approving the code, specific guidance creates demands on developers' time. These results are consistent with developers devoting more effort to revision and suggest that requests for changes may carry an opportunity cost for subsequent productivity.

The interaction between positivity and constructiveness further distinguishes these effects. The largest gains in code quality arise from feedback that is positive but not constructive. Conditional on tone, adding specific, actionable guidance does not increase these gains and continues to redirect effort toward revision.[4]

I next examine whether feedback predicts reviewer quality, measured by reviewer value-added (VA) to developer productivity. I estimate reviewer VA using the forecast-based approach of Chetty, Friedman, and Rockoff (2014). VA captures differences in subsequent developer productivity conditional on developer and case characteristics.[5] A prediction model using feedback quantity and text embeddings explains approximately 22% of the out-of-sample variation in reviewer VA.[6] Replacing the embeddings with toxicity, positivity, and constructiveness recovers

[4]Toxic feedback is Thus, positive tone and constructive content need not have additive effects on subsequent productivity.

[5]I estimate VA separately using code quantity and code quality; the two measures are positively correlated. I first estimate VA in the automatic-assignment sample and then extend the analysis to the full sample. Among reviewers observed in both settings, VA rankings are strongly correlated. This comparison provides supporting evidence for the broader application of the estimator.

[6]I train an XGBoost model on 80% of reviewers and evaluate it on the remaining 20%, keeping all observations from a given reviewer in the same sample.

84% of their incremental predictive power beyond feedback quantity alone. These three dimensions therefore capture much of the information in feedback text that predicts reviewer VA.

Why does toxic feedback persist despite its productivity costs? One possible explanation is that these costs are difficult to infer from observed performance. OLS estimates associate toxic feedback with higher code quantity, whereas the IV estimates indicate substantial losses. An organization relying on the observed correlation could mistake toxicity for effective management.

This paper makes three contributions. First, it opens the black box of what managers do by measuring one common managerial practice directly and linking it to both worker outcomes and manager value-added. Related work studies people-management skills (Hoffman and Tadelis, 2021), talent hoarding (Haegele, 2026), task allocation (Minni, 2026), and the identification of good managers (Weidmann et al., 2026), mostly with proprietary data from a single firm. I observe the same practice across hundreds of thousands of teams, which complements the use of CEO diaries to measure managerial behavior across firms (Bandiera et al., 2020). Economists have also studied feedback as a channel for human capital accumulation (Emanuel et al., 2026) and documented gender differences in how it is given and received (Shastry et al., 2020; Saygin et al., 2025; Freimane, 2024), treating it as a bundle. I draw on psychology and education research that separates the information feedback conveys from how it is delivered (Ashford and Cummings, 1983; Ilgen et al., 1979; Kluger and DeNisi, 1996; Hattie and Timperley, 2007; Banihashem et al., 2022), and show that the two components move different margins of productivity.

Second, the paper provides field evidence for theories in which feedback affects effort through workers' beliefs about themselves. Bénabou and Tirole (2002) model self-confidence as a motivational asset, Compte and Postlewaite (2004) show that confidence can become self-sustaining when emotions feed back into performance, and in Kőszegi et al. (2022) self-image is fragile when success is central to identity. These models share a prediction that how feedback is delivered affects effort even holding fixed the information it conveys. The contrast between toxic feedback and equally unfavorable but respectful criticism is a direct test of that prediction.

Third, the paper connects workplace amenities to productivity. Research on workplace hostility (Collis and Van Effenterre, 2025), sexual harassment (Folke and Rickne, 2022), and workplace violence (Adams-Prassl et al., 2024) has focused on sorting, compensating differentials, and worker welfare. Toxic feedback is a disamenity whose productivity consequences can be measured interaction by interaction. The results imply that a hostile interpersonal environment is costly to the firm, not only to the worker.

The remainder of the paper proceeds as follows. Section 2 describes the data and setting. Section 3 defines and validates the feedback measures and documents variation across reviewers. Section 4 presents the identification strategy, and Section 5 reports the effects on productivity and retention. Section 6 relates feedback to reviewer value-added and examines the persistence and origins of toxic reviewing. Section 7 concludes.

2 Data and Setting

I create a panel dataset by merging three sources: GitHub Archive (timelines, activities, feedback, and team structure), GitHub profiles (names, images, locations), and Revelio Labs (employment histories). I use the data to (i) characterize reviewer and developer demographics; (ii) identify reviewer-to-developer feedback messages; and (iii) measure the three feedback dimensions, developer productivity, and career outcomes. The panel spans 2017 to 2023.

2.1 GitHub and the Code Review Process

GitHub is the world's largest online coding platform, where developers store code, share projects, and collaborate. Technology firms such as Google and Microsoft maintain thousands of public projects and increasingly rely on GitHub for innovation and productivity (Nagle, 2019; Hoffmann et al., 2024).[7] The GitHub Archive captures all public GitHub activity in near real-time and preserves event-level traces, even after code and feedback are modified or deleted.

Below I define *team*, *case*, *team member*, *reviewer*, and *developer*, and Section 2.5 states the final sample restrictions used in the analysis.

2.1.1 Team and Case

A repository is a workspace where developers store code and track its version history. I define a *team* as a repository owned by an organization, much like a group of researchers co-authoring a paper.[8] In these teams, developers develop software by submitting code, reviewing one another's work, and discussing revisions. I restrict attention to organization-affiliated teams because they use structured workflows, align with firm objectives such as product development (Andersen-Gott et al., 2012), and better represent workplace settings than individual projects.

A pull request on GitHub is a formal submission of code changes for integration into a team's codebase. I refer to each pull request as a *case*. Submitting a case initiates a structured review process that helps identify bugs, enforce conventions, and maintain quality.[9]

A *case* has two components: feedback and an acceptance decision. The reviewer writes comments on specific lines of code or on the submission as a whole. I refer to these comments as feedback. The reviewer also decides whether to approve the submission or request changes. When team rules require reviewer approval, this decision is binding: the submission cannot enter the team's codebase until the review requirements are satisfied. The reviewer therefore both evaluates the developer's work and controls whether it enters the codebase. Figure 1 shows an example

[7] As of July 2026, GitHub hosts over 331 million public repositories and more than 217 million users (see repository and user, accessed 2026-07-02). Appendix A describes the platform and profile data in more detail.

[8] Economists have defined teams in various ways, from common interests (Marschak and Radner, 1958) to related productive inputs (Holmström, 1982) and observed co-authorship (Jones, 2021). Appendix A discusses how these definitions map to a GitHub repository.

[9] Emanuel et al. (2026) discuss how reviews raise quality and facilitate learning; job postings at Microsoft and Google explicitly require code review.

of a code review in a Microsoft team.

2.1.2 Team Members

Teams include internal members and outside volunteers. Anyone can submit code changes, but only internal members hold project-level permissions. I identify internal members using GitHub's *author association* field. This field records the relationship between a user and the repository in which the interaction occurs. I define *team members* as users labeled "owner," "member," or "collaborator". These roles imply write-level access and active involvement in development, including pushing code, reviewing pull requests, and coordinating work.[10]

Demographics. I infer developers' gender, race, and location from public information in their GitHub profiles. GitHub profiles are often used as professional profiles in software labor markets, giving developers incentives to provide information about their skills, identity, and location (El-Komboz and Goldbeck, 2024). Gender is predicted from profile images, race is classified from names, and location is standardized from self-reported profile entries, as described in Appendix A. These variables are inferred rather than directly observed and should therefore be interpreted as approximate demographic measures. I use them mainly to describe the sample and examine heterogeneity. Among developers with classified gender, about 86% are classified as male. Figure A1 maps the geographic distribution of team members. The United States has the largest concentration, followed by India, China, and Brazil.

2.1.3 Reviewers and Developers

Reviewers are team members formally listed in the "reviewer" field of a case and are responsible for evaluating the submitted code and providing feedback. *Developers* are the team members who submitted the case. Reviewer feedback messages come from GitHub Archive, which records the original text of public GitHub events at the time they occur, including messages that may later be deleted or edited on GitHub.

An advantage of this setting is that the data reveal the exercise of task-level authority directly, without relying on formal job titles. Within the code review process, reviewers evaluate developers' work and decide whether to approve it: a reviewer can request changes, which blocks the submission until the developer responds, or approve it, which allows the code to be merged into the team's codebase. Reviewer feedback therefore carries consequences for whether the work is accepted, which distinguishes it from ordinary peer feedback that developers can ignore without directly affecting the outcome of the submitted case. I interpret reviewer feedback as managerial feedback in this functional sense.

Observable differences between reviewers and developers are consistent with this interpretation. In the matched GitHub–LinkedIn sample, reviewers often hold senior technical titles, such

[10] Because *author association* is observed only when a user acts, I impute continuous membership between the first and last months in which a user is observed as a team member. Appendix A provides details.

as " Principal Engineer" or "Senior Software Engineer." Appendix A.7 compares reviewers with the developers they review and shows that reviewers differ systematically along several observable characteristics. Many teams also display a hierarchical review structure in which reviewers provide feedback to others but rarely receive feedback from them. The main results hold and are larger in teams with highly hierarchical review structures.

2.2 Reviewer Assignment

When a developer submits a case for code review, the reviewer decides whether to approve the proposed changes for inclusion in the team's codebase and can provide feedback through an overall review or comments on specific lines of code. Reviewers can be selected manually by the developer or assigned automatically under rules established by the team.

Under manual selection, GitHub suggests potential reviewers based on their prior contributions to the code being changed. The developer can choose a suggested reviewer or request another team member directly. This discretion allows developers to select reviewers based on technical expertise, prior working relationships, or the feedback they expect to receive. Reviewer feedback may therefore be correlated with characteristics of the developer or submission that also affect subsequent productivity.

Some teams instead use automatic reviewer assignment, limiting developers' ability to choose who reviews their submissions. I use this assignment mechanism for the identification strategy described below.

2.2.1 Automatic Reviewer Assignment

In teams that handle many cases, uneven review workloads can reduce efficiency and slow collaboration. Some teams therefore assign reviewers automatically to distribute workloads more evenly. Automatic assignment also limits developers' ability to choose reviewers based on anticipated feedback or approval decisions. I exploit this mechanism as a source of plausibly exogenous variation in reviewer assignment.

Automatic assignment broadly falls into two categories. In *team assignment*, a configuration file routes cases to the responsible team, and GitHub's built-in assignment feature, when enabled, selects individual reviewers using rotation or workload-based rules. In *bot assignment*, a GitHub bot or workflow, such as *Auto Assign* or *Reviewer Lottery*, selects reviewers according to the team's configuration.

Automatic assignment limits selection based on expected feedback. Although the eligible pool may depend on the expertise required for a case, the individual reviewer is selected by the assignment rule rather than the developer. Under rotation, for example, reviewer selection depends on the sequence of preceding review requests.

I identify adopting teams through assignment configurations in their codebases. The public GitHub data do not directly record when automatic reviewer assignment is enabled, so I use the

first observed case that adds or modifies the assignment configuration as a proxy for adoption and construct the sample from that month onward. Appendix B describes the assignment rules and adoption-dating procedure and compares adopting and non-adopting teams.

The observed assignment patterns provide supporting evidence. The assigned reviewer's feedback style is uncorrelated with observable developer and case characteristics (Section 4.1). The same reviewer handles a developer's next case with probability 7.32%, indicating that reviewer–developer matches are generally not repeated across consecutive cases.

2.3 Developer Productivity Measures

The main productivity-related outcomes are code quantity and code quality, measured at the developer-team-week level. I also measure non-code activity to capture potential reallocation of effort across tasks in response to different types of feedback.

Code Quantity. I measure code quantity using the number of lines of code a developer submits to a team in a week. I also count rewritten lines to capture revisions to previously submitted code, as described in Appendix A.8. Line counts are a reasonable measure of code quantity in this setting, where the sample consists of professional developers. They are also widely used as a proxy for code output in both the computer science and economic literature (Vasilescu et al., 2015; Wagner and Ruhe, 2018; Emanuel et al., 2026).

Code Quality. My main measure of output quality is *code quality*: the share of submitted code lines that are accepted in the team codebase without revision. A higher share indicates that more submitted code was accepted as written (McIntosh et al., 2016; Emanuel et al., 2026). I assign this measure to the week in which the case was opened. As an additional measure, I use *case quality*: the share of cases whose initial submission is merged into the team's main codebase without major revisions. Appendix A.9 provides definitions and examples. Figure A4 shows that both measures are positively correlated with developers' number of GitHub followers, a signal of professional reputation.

Non-Code Activity. I measure a developer's non-code activity each week as the number of non-code GitHub activities, such as participating in issue threads, answering questions, or providing troubleshooting support.

2.4 Developer Career Histories

To measure retention, I link developers to LinkedIn-based career histories from Revelio Labs. Revelio Labs compiles information from LinkedIn and other public sources, producing a panel of 1.25 billion professional profiles worldwide. The data record job histories with start and end dates for each employer–position spell as of August 2024. I use these data to construct two measures

of career mobility. I define a *firm switch* as a move to a new LinkedIn employer within one year, and an *occupational switch* as a move to a position in a different occupation within one year, where occupation is the SOC major group (the first two digits of the O*NET-SOC code). Occupation describes the developer's role, whereas industry describes the employer. I also construct an *industry switch* and report it in Appendix F.

2.5 Sample Restrictions and Summary Statistics

I apply three restrictions to ensure the sample captures structured workplace collaboration rather than ad hoc activity. First, I restrict attention to teams under organizations, excluding projects operated by individual users. Second, I retain only code submissions made by formal team members, who contribute more regularly. Third, I focus on cases with at least one reviewer.

I construct two samples. The *full sample* includes all teams that satisfy these restrictions between 2017 and 2023. It covers 525,841 teams and 46,159,639 cases, of which 20.99% have at least one assigned reviewer. These reviewed cases generated over 203 million feedback messages.

The *automatic assignment sample* includes 3,457 teams and 4,228,687 cases. These teams represent only a small fraction of the full sample, but they are highly active. Table 1 shows that teams adopting automatic assignment are systematically larger and handle far greater review volumes than other teams. On average, they have nearly 20 members, process 1,223 cases, and generate around 7,070 feedback messages per team. They also account for 9 % of all cases and 12% of all feedback messages in the full sample. This high case volume is a plausible reason for adoption, since assignment lowers the coordination cost of allocating reviewers and distributes reviews more evenly. These teams also tend to belong to leading technology firms (Table B2).

I then turn to developers' outcomes, summarized in Table 2. Productivity is measured along three dimensions: code quantity, code quality, and non-code activity. Because outcome distributions are heavy-tailed, I winsorize all continuous variables at the 95th percentile while keeping bounded outcomes such as case quality and code quality unchanged. Table 2 further distinguishes the "Focal Team," where the developer received feedback on a specific case in a given week, from "Other Teams," which aggregates the developer's activity across all other teams in the same week.

3 Measuring Feedback: Tone and Information

This section classifies reviewer feedback and documents its variation across reviewers. Manual classification is infeasible at this scale and may also be inconsistent across raters. I therefore use fixed language models to apply common classification criteria across feedback messages.

I measure three dimensions of feedback, and each message receives a separate label on each dimension: toxic or non-toxic, positive or non-positive, and constructive or non-constructive. The labels are not mutually exclusive.[11]

[11] The three dimensions are not chosen arbitrarily. Before imposing any labels, I summarize the corpus using principal component analysis (PCA), which recovers the axes of variation directly from the text rather than from predefined

Before classification, I clean and standardize the feedback text. I remove quoted blocks and code snippets, expand software acronyms, and adjust for programming terms, such as "kill" or "dump," that are benign in code but may be flagged by general-purpose language models (Sarker et al., 2020). Appendix C describes these preprocessing steps.

For toxicity and positivity, I use published BERT-family classifiers that have been evaluated against human-labeled benchmarks. These tasks focus on the language of the feedback and can be assessed from the message itself. Publicly available model weights allow other researchers to reproduce the classifications using the same fixed model. The classifiers require no prompts and can be retained independently of updates to API-based models. Their lower computational cost also makes them practical for classifying hundreds of millions of messages. The choice therefore reflects the requirements of these tasks rather than a general preference for smaller models.

Constructiveness differs because assessing whether feedback provides specific, actionable guidance requires interpreting the message in the context of the code under review. In the absence of a suitable task-specific classifier, I pair each message with the relevant code and use a fixed GPT model to assess constructiveness. Appendix C.3 reports the definition, prompt, and model version. For all three dimensions, I fix the classification procedure in advance and apply it uniformly across feedback messages.

3.1 The Tone of Feedback

3.1.1 Toxicity

I use a published, reproducible rule to classify toxic feedback. Following the ToxiGen annotation guideline (Hartvigsen et al., 2022), feedback is toxic if it contains insults, profanity, aggression, or discriminatory language.[12] I implement this definition using ToxiGen-RoBERTa, a fine-tuned RoBERTa classifier (Liu et al., 2019) designed to detect both explicit and implicit toxicity. The same classifier is used by Ederer et al. (2024). Its publicly available model weights allow other researchers to reproduce the classifications.

The classifier labels "This is bad code" as non-toxic (probability 0.918), but labels "What's wrong with you to write such bad code?" as toxic (0.712), as well as the discriminatory statement "Women are just not good at coding" (0.988). Figure C6 presents examples from the data alongside developers' replies. In these examples, feedback with higher toxicity scores is accompanied by more toxic replies, consistent with recipients responding to toxic language. Appendix C provides additional examples illustrating the distinction between criticism and toxicity.

dimensions. Two dimensions dominate: positivity (for example, *good*, *thank*) and constructiveness (for example, *add*, *test*), as shown in Figure C1. Thus, a decomposition that imposes no labels recovers the same dimensions I study. Toxic feedback is too rare on public platforms to appear among the leading axes (Beknazar-Yuzbashev et al., 2022), but prevalence need not correspond to importance. Toxic language is explicitly governed by workplace conduct policies, and exposure to it is associated with lower well-being (Nielsen and Einarsen, 2012). Whether these rare messages have economically meaningful effects is an empirical question, which I examine in Section 5.

[12] Figure C2 reproduces the guideline. The ToxiGen-RoBERTa checkpoint achieves 94.5% accuracy on the human-annotated ToxiGen benchmark. See https://www.microsoft.com/en-us/research/publication/toxigen, accessed 2025-10-14.

Human validation supports the classifier's use in the code-review setting. Because ToxiGen was developed for hate-speech detection, I assess its classifications using human annotations of 1,000 code-review messages. A human rater classified 92% of model-flagged messages as toxic, indicating high precision relative to the human labels.[13]

This validation does not establish a universally accepted boundary between toxic and non-toxic feedback. Readers may disagree with individual labels, and recipients may interpret the same message differently. The estimates should therefore be read as effects of feedback classified as toxic under this fixed rule.

3.1.2 Positivity

The second dimension is positivity, whether feedback is supportive or discouraging. I measure it with SiEBERT, a RoBERTa-large sentiment classifier fine-tuned on 15 datasets from diverse English-language sources (Hartmann et al., 2023). SiEBERT has also been used to measure workplace communication in economics (Brynjolfsson et al., 2025). The model classifies "This is bad code" as negative (0.999) and "Look good to me" as positive (0.996).

3.2 The Informational Content of Feedback

3.2.1 Constructiveness

The third dimension captures the informational content of feedback. Because feedback can convey many types of information, I focus on a feature of the text rather than a judgment about the person: whether the feedback message provides specific, actionable information about the submitted code. I call this dimension constructiveness. Feedback is constructive if it identifies a concrete issue or recommends a specific action the developer can take. For example, "You can do it in a better way" is non-constructive because it contains little actionable information, even if some developers may still find it useful.[14]

Unlike toxicity and positivity, constructiveness has no well-established classifier suited to this setting. I therefore use GPT to evaluate each feedback message together with the relevant code. This allows the classification to incorporate programming context needed to determine whether a stated problem or recommended action is specific and actionable.

[13] The validation sample contains 500 flagged and 500 unflagged messages from the automatic assignment sample. The 92% figure measures agreement among flagged messages, not overall accuracy. ToxiGen flags toxicity more conservatively than Detoxify (Hanu and Unitary team, 2020), identifying 0.21% of messages as toxic compared with 3% on the same sample.

[14] An alternative is to measure the informational usefulness of feedback. Usefulness, however, is not observable in the text, and the natural proxy is whether the developer's subsequent code improves. Defining the treatment this way would make it a function of the outcome it is meant to explain. The Cambridge Dictionary defines "constructive" as advice, criticism, or actions that are useful and intended to help or improve something. Kolhatkar et al. (2020) define constructive comments as high-quality contributions to a conversation. Their setting is news discussion, whereas GitHub feedback is shorter, more technical, and directed toward a specific piece of code. I adapt the concept to code review by focusing on explicit problems and recommended actions.

I pair each message with the code submission it refers to and ask a fixed model to score its specificity and actionability from 0 to 10. Messages scoring 4 or above are classified as constructive. To select the prompt and model, I benchmark four OpenAI models, GPT-4.1-nano, GPT-4o-mini, GPT-4o, and o3, against human labels. Appendix C.3 describes the procedure. For example, "Agree" scores 0, "This is not clear" scores 1, and a specific suggestion to handle severe errors scores 4. Table C2 provides further examples.

Human validation supports this measure of constructiveness. Human annotators agreed with the model's binary classification in 90% of messages in a validation subsample. The empirical results are also robust to lowering the classification cutoff from 4 to 3.

3.3 Classification Results

Panel C of Table 1 compares feedback classifications between the full sample and the automatic-assignment sample. [15] The two samples yield similar patterns: at the message level, about 0.21% of feedback is toxic, 39.40% is positive, and about 31.36% is constructive. Figure 2a illustrates these shares. [16]

Table 3 cross-classifies feedback along all three dimensions. The largest group is non-toxic, negative, and non-constructive (38.5%), for example "This is not clear." Toxic feedback is overwhelmingly negative and non-constructive (85%), such as "You are talking nonsense." Positive feedback is often non-constructive (76%), typically short affirmations like "Looks good to me." Most constructive feedback is non-toxic and negative (70%). For example, "Still feels redundant. An angle should just be a union of float and FreeParameter. Once the parameter is bound, you have a new circuit with no free parameters."

3.4 Descriptive Facts: Reviewer Styles and Team Hierarchy

The classified feedback reveals two facts about code review that motivate the empirical strategy. First, feedback style varies substantially across reviewers. Second, feedback flows hierarchically within teams, with some members regularly reviewing others but rarely being reviewed by them.

Fact 1: Reviewers explain most of the variation in feedback. To describe the sources of variation in feedback, I aggregate messages to developer–team–week–reviewer cells. For each feedback dimension, the dependent variable is the mean predicted score across feedback messages in the cell, weighted by the number of messages. I regress this score on team, reviewer, and developer fixed effects, together with the case feature controls. I then measure the explanatory power of each component by the increase in R^2. Figure 3 shows that reviewer fixed effects explain the largest

[15] In the automatic assignment sample, all feedback messages are classified. In the full sample, toxicity and positivity are measured on all messages, while constructiveness is evaluated on a subsample due to budget constraints. Unless otherwise noted, classification results refer to the automatic assignment sample.

[16] Figure C3 plots the distributions of predicted scores. Toxicity probabilities cluster between 0.8 and 1.0, so few messages sit near the threshold.

share of variation in all three feedback dimensions. Team and developer fixed effects explain substantially less, while case characteristics add little. Feedback therefore appears to reflect persistent differences across reviewers more than team context, developer identity, or case characteristics.

Fact 2: Most teams follow a hierarchical feedback structure. I measure team hierarchy from the direction of feedback exchanges. A fully hierarchical team has feedback flowing in one direction, as when managers review juniors but not vice versa; reciprocal feedback, where two members review each other's code, indicates a more egalitarian structure. For each team-year, I build a directed network in which an edge from A to B indicates that A reviewed B's code at least once, and define the hierarchy score as

$$\text{Hierarchy Score} = 1 - \frac{2R}{E},$$

where E is the number of directed edges and R is the number of pairs that review each other in both directions. Each reciprocated pair accounts for two directed edges, so $2R/E$ is the share of review relationships that are reciprocated. A score of 1 indicates purely one-way review, while 0 means every relationship is reciprocated. Figure 4 shows the distribution of hierarchy scores, together with three example feedback networks at low, medium, and high hierarchy. Most teams are hierarchical: code reviews tend to flow in one direction rather than between peers, and Figure C5 restricts to five-member teams and to the automatic assignment sample.

Taken together, these facts guide the empirical strategy. Feedback tone and information vary primarily across reviewers, not teams or developers. This combination means that reviewer styles generate meaningful, plausibly exogenous variation in the feedback developers receive, which I exploit to identify their effects on developer productivity.

4 Empirical Strategy

I estimate how different types of reviewer feedback affect developers' subsequent productivity in the same team during the four weeks after review. The unit of observation is at the developer–team–week level.[17] The treatment is whether the developer receives a given feedback type in week t, and outcomes are measured over weeks $t+1$ to $t+4$. To illustrate the empirical strategy, I use toxic feedback as the running example, and the same framework applies to other types of feedback.

Consider the following structural equation, where t indexes calendar weeks when feedback is given:

$$Y_{im,[t+1,t+4]} = \beta\, I^{\text{Toxic}}_{im,t} + \mathbf{X}'_{i,t}\Gamma + \gamma_{m,\text{year}(t)} + \varepsilon_{im,t}, \tag{1}$$

[17] More precisely, an observation corresponds to a week in which developer i received reviewer feedback on at least one case in team m. When a developer has multiple reviewed cases in the same week, I retain each case as a separate observation. Because outcomes are measured at the developer–team–week level, cases reviewed within the same week share the same outcome. This structure induces correlated errors and gives greater weight to developers who submit more cases. Restricting the sample to each developer's first reviewed case in a given week yields similar estimates.

where $Y_{im,[t+1,t+4]}$ is the productivity of developer i in team m over weeks $t+1$ to $t+4$ (e.g., code quantity or quality, defined in Section 2.3). The indicator $I^{\text{Toxic}}_{im,t}$ equals 1 if developer i received toxic feedback in team m during week t, and 0 if the feedback was non-toxic. The vector $X_{i,t}$ controls for feedback volume and activity level (measured in deciles) in week $t-1$. I also include team-by-year fixed effects $\gamma_{m,\text{year}(t)}$.[18] The coefficient β captures the effect of receiving toxic feedback in week t.

The main challenge is that OLS estimates of (1) may be biased if the receipt of toxic feedback correlates with unobserved determinants of productivity. The bias can run in either direction. On the one hand, less able developers may be both less productive and more likely to attract toxic feedback, so OLS overstates the harm of toxicity. On the other hand, reviewers may direct toxic feedback at developers they expect to respond with greater effort, for instance, those with weak outside options who fear dismissal, so OLS understates it.

Conditional on team-by-year fixed effects, reviewer assignment is as-good-as random: it is independent of developer and case characteristics. Reviewers differ systematically in their feedback styles: some are more toxic than others. This difference in feedback style generates variation in developers' exposure to toxic feedback. The assigned reviewer therefore shifts a developer's exposure to toxicity for reasons unrelated to the developer or the case.[19]

To instrument for receiving toxic feedback, I construct a leave-developer-out measure of reviewer toxicity. This follows the judge or officer designs in Dobbie and Song (2015); Dobbie et al. (2017, 2018); Bhuller et al. (2020). Reviewers and developers may interact repeatedly, so a simple leave-one-case-out measure risks bias if the same pair appears elsewhere. I therefore exclude all cases involving the focal developer when computing a reviewer's toxicity rate, not only the focal case. Concretely, the instrument is the average share of cases with other developers in which the reviewer gave toxic feedback. This procedure ensures that the instrument is not mechanically correlated with the focal developer's outcomes.

For each case of developer i reviewed by reviewer j, I construct the reviewer's leave-developer-out toxicity propensity. Let $N_{j,-i} = \sum_{h \neq i} n_{hj}$ denote the number of cases that reviewer j reviewed for developers other than i. The instrument is

$$Z^{\text{Toxic}}_{j,-i} = \frac{\sum_{h \neq i} D^{\text{Toxic}}_{hj}}{N_{j,-i}},$$

where D^{Toxic}_{hj} is the number of cases in which reviewer j gave toxic feedback to developer h. Both

[18] Following the judge leniency literature, I condition on the block where assignment is as-good-as random and include block fixed effects. Identification comes from within-block comparisons (Dobbie and Song, 2015; Dobbie et al., 2017; Bhuller et al., 2020; Beuschlein, 2024). In the data, reviewer assignment primarily occurs within team-by-year cells. The average reviewer tenure on a team is 1.12 years. I therefore include team-by-year fixed effects. Results are robust to team-by-month fixed effects.

[19] The same logic applies to positive and constructive feedback. Some reviewers are systematically more positive (Figure C4b) or more constructive (Figure C4c). Consequently, the probability that a developer receives a given feedback type depends on the tendencies of the assigned reviewer. I exploit this reviewer-driven variation as the basis for an instrumental variables strategy to identify the causal effects of feedback types.

the numerator and denominator exclude all cases involving the focal developer, and the instrument does not vary over time.

The main analysis is based on 2SLS estimates of the second-stage equation (1) and the first-stage for developer i and reviewer j at week t is given by:

$$I_{im,t}^{\text{Toxic}} = \alpha \, Z_{j,-i}^{\text{Toxic}} + \mathbf{X}_{i,t}'\delta + \gamma_{m,\text{year}(t)} + \nu_{im,t}, \tag{2}$$

where $Z_{j,-i}^{\text{Toxic}}$ denotes the toxicity of reviewer j assigned to developer i, and α is the effect of reviewer toxicity on the likelihood of receiving toxic feedback. Robust standard errors are clustered at the reviewer level in both stages.[20] I report the robust first-stage F-statistic, which is large in this setting (Staiger and Stock, 1994; Lee et al., 2022).[21]

I interpret the 2SLS estimates within the Local Average Treatment Effect (LATE) framework (Angrist and Imbens, 1994). Under conditional independence, exclusion, and monotonicity, the instrument recovers the local causal effect of receiving toxic feedback among the subgroup of developers whose likelihood of exposure varies with the assigned reviewer. Conditional independence follows from as-good-as-random reviewer assignment within team-by-year cells and is assessed in Figure 6. Exclusion requires that the assigned reviewer's style affects developer outcomes only through the targeted feedback dimension. I discuss this assumption below and assess it in Section 5.5. Monotonicity requires the instrument to shift exposure in the same direction for all developers. Section 4.1 presents evidence consistent with this condition.

4.1 Instrument Validity

Instrument Relevance. Figure 5 provides a graphical representation of the first stage. It shows reviewer-level feedback propensities after residualizing team-by-year fixed effects. To reduce measurement error, I only include teams with at least 2 reviewers and reviewers who have handled at least 10 cases. This selection leaves 480,697 cases and 10,873 reviewers, with an average of 44.2 cases per reviewer. All three panels reveal substantial cross-reviewer variation in the likelihood of sending toxic, positive, or constructive feedback.

Panel (a) in Figure 5 shows residualized reviewer toxicity: moving from the 10th to the 90th percentile raises the probability of receiving toxic feedback by 0.2 pp, a 26.3% increase relative to the mean rate of 0.76%.[22] For positivity in Panel (b), moving from the 10th percentile to the 90th

[20] I cluster standard errors at the reviewer level because misclassification in feedback detection is likely correlated within reviewers. Each reviewer evaluates many pull requests, which induces within-reviewer correlation in the errors. Clustering at the reviewer level addresses this dependence and yields valid inference. As a robustness check, Figure D1 reports results with team-level clustering; the estimates are similar.

[21] Assessing the risk of many-weak-instrument bias in judge-style designs remains an open question (Hull, 2017; Frandsen et al., 2023; Bhuller et al., 2020; Agan et al., 2023). In Table D1, I evaluate the robustness of the leave-developer-out reviewer leniency instrument using alternative IV estimators. These include: (i) limited information maximum likelihood (LIML) with all reviewer dummies as instruments; (ii) the modified bias-corrected two-stage least squares (MBTSLS) estimator of Kolesár et al. (2015); and (iii) the unbiased jackknife instrumental variable estimator (UJIVE) of Kolesár (2013). The estimates are quite stable.

[22] The message-level toxicity rate is 0.21 % as shown in Table 1, while the case-level treatment rate in the IV sample

raises the probability of positive feedback by 31 pp, a 43.9% increase over the mean of 70.6%. For constructiveness in Panel (c), moving from the 10th percentile to the 90th raises the likelihood of constructive feedback by 41 pp, a 79.2% increase over the mean of 51.8%.

Table 4 presents first-stage estimates from regressions of indicator variables for receiving each feedback type on the corresponding reviewer instrument, defined as the reviewer's residualized rate of providing that feedback type. Column (1) includes team fixed effects, Column (2) replaces these with team-by-year fixed effects, and Column (3) adds controls for developer gender, race, GitHub activity in the year before submission, the first year of GitHub activity, case characteristics, and project stage, defined based on the release version of the team product (see Appendix A.10). Standard errors are clustered at the reviewer level. The coefficients are large and statistically significant across all panels. In Panel (a), a 10 pp increase in a reviewer's residualized toxicity rate raises the probability of receiving toxic feedback by roughly 7 pp. Panel (b) shows a comparable effect for positive feedback (8.2 pp), while Panel (c) reports a slightly larger effect for constructive feedback (8.6 pp).

Conditional Independence. A valid instrument must be uncorrelated with developer and case characteristics that affect outcomes. Column (3) of Table 4 adds controls for these predetermined variables to the first-stage regressions. If reviewer assignment is random, the coefficients should remain stable, which is exactly what I find.

As a second test, Figure 6 assesses the randomness of case assignment after controlling for team-by-year fixed effects. In Panel (a), the blue lines with red markers plot coefficients from regressions of reviewer toxicity on standardized developer and case characteristics. None of the coefficients is statistically significant. Similar patterns hold for positive and constructive feedback in Panels (b) and (c). Taken together, these findings provide strong evidence that reviewer assignment is conditionally random.

Exclusion Restriction. Interpreting the IV estimates as causal effects of a specific feedback type requires that, conditional on team-by-year fixed effects and controls, the assigned reviewer's style in that dimension affects developer outcomes only through the included feedback treatment. This restriction is potentially threatened because reviewer assignment determines not only the focal feedback dimension but also other correlated dimensions of feedback and characteristics of the reviewer. Section 5.5 examines these threats by jointly instrumenting for different feedback types and progressively holding fixed reviewers' other feedback tendencies, observable characteristics, and approval leniency.

Monotonicity Condition. If treatment effects are homogeneous across developers, identification requires relevance, conditional independence, and the exclusion restriction. When treatment

is 0.76% because a case can contain multiple reviewer messages and is classified as toxic if any reviewer feedback in the case is toxic.

effects are heterogeneous, these assumptions remain necessary, and a local-effect interpretation additionally requires monotonicity: the instrument must shift the probability of receiving a given feedback type in one direction across developers. Intuitively, a developer who would not receive toxic feedback from a more toxic reviewer would also not receive it from a less toxic one. I cannot directly test this assumption, but I can use an indirect test common in the literature. Following Bhuller et al. (2020) and Beuschlein (2024), I confirm that first-stage estimates are positive across subsamples split by gender, race, activity level, GitHub experience, and follower count. Across all splits, the first stage is positive and statistically different from zero (Appendix Table D2), consistent with the monotonicity assumption.

5 Effects of Feedback on Developer Productivity and Retention

This section presents the main results on developer productivity and retention. I measure productivity on new code written after feedback, which avoids mechanically linking feedback to the code under review. I report effects on code quantity and quality, and on non-code task activity. Finally, I link feedback to developers' career trajectories to examine its effect on retention. The main results estimate the effect of each feedback dimension separately; I also report a joint specification that includes all three dimensions in Figure D2.

5.1 The Tone of Feedback

Panel A of Table 5 presents 2SLS estimates of the effect of feedback tone on developer productivity within a team over the four weeks following a review.

Toxic feedback reduces code quantity and quality. Toxic feedback lowers both the quantity and the quality of subsequent code. In the four weeks after the review, developers are 7 pp less likely to contribute any code, and conditional on coding, they write 43% fewer lines (0.56 log points).[23] Quality falls alongside quantity: the share of accepted lines drops by 9.1 pp (13%), and the share of merged cases also declines, although this estimate is not statistically significant. Developers are therefore not trading quantity for quality.

Effort is withdrawn, not reallocated. Less new code need not mean less effort. If developers shifted effort elsewhere, output should rise at the destination. I examine three possible destinations: the focal case, other work in the focal team, and other teams. Because the new-code measure counts only cases opened after the focal review, effort on the focal case appears instead in rewritten lines. Developers do not rewrite more code on the case that received toxic feedback. Nor do they shift toward non-code work in the team: activity such as answering users' questions about the product is unaffected. Code written for other teams is likewise unchanged (Figure G8).

[23] The OLS estimates in Table 5 have the opposite sign: developers who receive toxic feedback produce more code, not less. Section 6.3 returns to this contrast as one reason toxic feedback persists.

The decline also does not reflect reviewers withholding future work. If reviewers responded by routing fewer cases to the developer, output would fall even with effort held constant. However, the number of cases developers open is unchanged (Figure G9b), so developers face the same set of tasks and produce less on each. The decline thus reflects a reduction in the developer's labor supply rather than in reviewers' demand for it.

Toxicity, not criticism. Most toxic feedback is also negative, raising the question of whether the harm comes from toxic language or from criticism itself. Figure 7 compares the effects of toxic feedback with those of respectful criticism (non-toxic & negative). While toxic feedback significantly reduces both code quantity and quality, respectful criticism does not reduce code quantity and has only modest effects on quality. These results suggest that the harm associated with toxic feedback stems from its toxic tone rather than from criticism itself.

Positive feedback increases code quality. Positive feedback works in the opposite direction on quality (Panel A of Table 5). The share of accepted lines rises 4.4 pp (6.35%). Quantity does not rise with it: conditional on coding, the change in code lines is insignificant. Positive feedback thus raises the quality of what developers write rather than the amount.

These findings on feedback tone align with theoretical models of confidence and performance. Compte and Postlewaite (2004) formalize how positive affect and selective recall of successes sustain effort and productivity. Similarly, Bénabou and Tirole (2002) and Kőszegi et al. (2022) emphasize the central role of self-confidence: positive feedback reinforces beliefs about ability and raises effort, while toxic feedback can undermine these beliefs and reduce productivity.

5.2 The Information Content of Feedback

Constructive feedback, defined as specific and actionable information about submitted code, affects developers differently from feedback tone. Panel B of Table 5 shows that, conditional on coding, developers write about as much new code as before, but the quality of the new code they do write declines: the share of accepted code falls by 10.39%. [24]

Constructive feedback redirects effort rather than reducing it. Across the same three destinations as for toxic feedback, developers rewrite 49.18% more lines of the reviewed case (Figure 9a), while their non-code activity and the code they write for other teams do not change. Developers take the time for revision out of code quality, not code quantity: the amount of new code they write is unchanged, but the share of their code that reviewers accept falls.

This shift carries an opportunity cost. Reviewers focus on improving the current submission and may not account for the developer time their feedback demands. Developers satisfy the reviewer, but the quality of their next piece of work falls. Whether this trade pays off depends on

[24] Figure D5 defines constructive feedback with an alternative threshold (≥ 3). The results are similar, so the choice of threshold does not drive them.

how much the revisions improve the reviewed code. I therefore read the result as a shift of effort toward revision, not a net loss in productivity.

Tone versus information. The previous results study feedback tone and information content separately. I next examine feedback that combines these dimensions to ask whether productivity gains come from what reviewers say or how they say it. Figure 8 shows that the largest improvements in code quality come from positive feedback without specific and actionable information.

5.3 Retention and Spillovers

Retention. The effects of feedback extend beyond the coding week. Linking developers' GitHub activity to their LinkedIn careers (Appendix F), I find that positive feedback lowers the probability of switching firms within one year by 8 pp, while toxic feedback has no significant effect on retention (Figure 9b). The effect is specific to firm attachment: no feedback type significantly changes the probability of switching occupations or leaving the industry (Figure F1). These results indicate that feedback is a micro-level channel through which managerial practice shapes retention in knowledge-intensive firms. Because employee knowledge is a core asset and turnover is costly, high-tech firms invest heavily in predicting and reducing attrition.[25] The findings suggest that improving feedback practices, and in particular fostering a positive feedback environment, may help strengthen retention and preserve firm-specific human capital.

Spillover. Feedback also affects developers' interactions with coworkers and their work in other teams (Figure 9c). Developers who receive positive feedback subsequently send a higher share of positive messages to coworkers, suggesting that positive tone improves the communication environment within the team. They also increase the quality of code they submit to *other* teams by 5.8%. These patterns are consistent with positive feedback raising workers' confidence in their ability and their motivation to contribute beyond the original code tasks. Constructive feedback, by contrast, lowers code quality in other teams, consistent with the rewrite tradeoff crowding out attention to work elsewhere. Since code quantity in other teams does not change (Figure G8), these effects reflect spillovers in code quality rather than a relabeling of activity across teams.

5.4 Heterogeneous Effects

The average effects mask meaningful heterogeneity. I examine which developers and teams are most affected by splitting the sample by demographics, ability, prior exposure to feedback, tenure, team structure, and the reviewer–developer match. Two patterns are most informative. First, feedback effects are stronger in more hierarchical teams, where reviewer feedback may carry greater evaluative weight. Second, positive feedback has larger effects on code quality after repeated

[25] Hoffman and Tadelis (2021) measure management skill with six survey items, two of which concern feedback directly: communicates clear expectations" and provides continuous coaching."

exposure, consistent with appreciation accumulating over time. Because these analyses involve many subgroup comparisons, I interpret the patterns as suggestive rather than definitive.

By Race. Figure G1 compares Asian and White developers, the two largest racial groups in the sample (Appendix A.5). The estimated effects of most feedback types are similar across the two groups, but responses to toxic feedback differ. Asian developers increase subsequent code output after receiving toxic feedback, whereas White developers reduce output. One possible interpretation is that developers differ in whether they view person-directed toxicity, which may vary with norms surrounding directness, criticism, and authority in workplace and educational settings (Kim et al., 2013; Gelfand et al., 2011; Pinquart and Kauser, 2018).

By Gender. Figure G2 reports feedback effects by developer gender (Appendix A.4). Male and female developers respond similarly to positive and constructive feedback, but differ on toxic feedback: males reduce code quantity, while females slightly increase it, though this is not statistically significant. With women only 12% of the sample, this subgroup is likely selected.

By First vs. Repeat Exposure. Figure G3 compares developers' responses to their first exposure to each feedback type with later exposures. Most patterns are similar, suggesting developers do not quickly adapt. The exception is positive feedback: quality gains appear mainly after repeated exposures, consistent with developers internalizing appreciation over time.

By Developer Ability. Figure G4 splits developers by a proxy for ability: whether their pre-feedback code quality is above or below the median. Toxic feedback reduces code quality for both groups, suggesting that its negative effect is not concentrated among lower-ability developers. Positive feedback has larger benefits for developers with above-median prior quality. This pattern is consistent with positive feedback reinforcing developers' confidence when they have already performed well, thereby increasing their motivation to maintain high-quality work. Constructive feedback, by contrast, tends to reduce output for developers with below-median prior quality, suggesting that detailed feedback may impose larger adjustment costs on developers who are less prepared to incorporate it.

By New Developers vs. Incumbents. Figure G5 compares developers in their first two months on a team with longer-tenured incumbents. While incumbents are a selected group who have remained on the team, the key distinction is between developers who are still learning the team's norms and those who are already integrated. Toxic feedback is especially damaging for incumbents, whereas positive feedback most strongly increases their code quality. This contrast suggests that feedback becomes more consequential once developers are integrated into the team.

By Team Hierarchy. Figure G6 examines effects by team hierarchy score. In more hierarchical teams, toxic feedback reduces future code quality more and positive feedback improves it more, perhaps because appreciation from high-status reviewers carries stronger motivational weight.

5.5 Threats to the Exclusion Restriction

Interpreting the IV estimates as the effect of a specific type of feedback requires that each type affect outcomes only through its own channel. Three concerns threaten this exclusion restriction.

First, feedback is multidimensional, and its dimensions are correlated at the message level. For example, toxic feedback is rarely positive, so an instrument for toxicity may capture the effect of receiving less positive feedback. Second, correlation also arises at the reviewer level: a reviewer who gives more toxic feedback may also give less positive feedback, so the toxicity instrument may reflect the reviewer's style in the other dimensions rather than toxicity itself. Third, reviewer characteristics may be correlated with feedback style and independently affect developer outcomes. For example, seniority, reputation, demographics, or approval leniency may influence how developers respond to a review even holding the language of the feedback fixed.

I address these concerns in turn. I jointly instrument for the different feedback types, hold fixed the reviewer's tendencies in the other feedback dimensions, and then additionally hold fixed observable reviewer characteristics and strictness.

5.5.1 Jointly Instrumenting Other Feedback Dimensions

A single feedback message can be toxic, non-positive, and non-constructive at once, so a reviewer's toxicity instrument may also shift the chance of receiving positive or constructive feedback. If it does, the estimated effect of toxicity absorbs the influence of encouragement or information. To separate the three channels, I instrument them jointly, extending the baseline model in equations (1) and (2):

$$Y_{im,[t+1,t+4]} = \beta^{\text{Toxic}} I^{\text{Toxic}}_{im,t} + \beta^{\text{P}} I^{\text{P}}_{im,t} + \beta^{\text{C}} I^{\text{C}}_{im,t} + \mathbf{X}'_{i,t}\Gamma + \gamma_{m,\text{year}(t)} + \varepsilon_{im,t}, \tag{3}$$

$$I^{\text{Toxic}}_{im,t} = \phi^{\text{Toxic}}_{\text{T}} Z^{\text{Toxic}}_{j,-i} + \phi^{\text{Toxic}}_{\text{P}} Z^{\text{P}}_{j,-i} + \phi^{\text{Toxic}}_{\text{C}} Z^{\text{C}}_{j,-i} + \mathbf{X}'_{i,t}\delta^{\text{Toxic}} + \gamma^{\text{Toxic}}_{m,\text{year}(t)} + \nu^{\text{Toxic}}_{im,t}, \tag{4}$$

$$I^{\text{P}}_{im,t} = \phi^{\text{P}}_{\text{T}} Z^{\text{Toxic}}_{j,-i} + \phi^{\text{P}}_{\text{P}} Z^{\text{P}}_{j,-i} + \phi^{\text{P}}_{\text{C}} Z^{\text{C}}_{j,-i} + \mathbf{X}'_{i,t}\delta^{\text{P}} + \gamma^{\text{P}}_{m,\text{year}(t)} + \nu^{\text{P}}_{im,t}, \tag{5}$$

$$I^{\text{C}}_{im,t} = \phi^{\text{C}}_{\text{T}} Z^{\text{Toxic}}_{j,-i} + \phi^{\text{C}}_{\text{P}} Z^{\text{P}}_{j,-i} + \phi^{\text{C}}_{\text{C}} Z^{\text{C}}_{j,-i} + \mathbf{X}'_{i,t}\delta^{\text{C}} + \gamma^{\text{C}}_{m,\text{year}(t)} + \nu^{\text{C}}_{im,t}. \tag{6}$$

Here $I^{\text{Toxic}}_{im,t}$, $I^{\text{P}}_{im,t}$, and $I^{\text{C}}_{im,t}$ equal 1 if developer i received toxic, positive, or constructive feedback in team m during week t, and 0 otherwise. The three treatments are instrumented jointly by the leave-developer-out reviewer propensities $Z^{\text{Toxic}}_{j,-i}$, $Z^{\text{P}}_{j,-i}$, and $Z^{\text{C}}_{j,-i}$, where $j = j(i,t)$ is the assigned reviewer. With all three channels instrumented simultaneously, β^{Toxic} isolates the effect of toxicity holding positive and constructive feedback fixed. Figure D2 reports the results, which are nearly identical to the baseline in Table 5.

5.5.2 Controlling for Reviewer Feedback Styles in Other Feedback Dimensions

The joint system above addresses correlation among the three feedback *treatments*. The second concern is different: it concerns correlation in the *reviewer's style*. A reviewer who gives more toxic feedback may also give less positive or less constructive feedback, so the toxicity instrument may proxy for the reviewer's tendencies in the other dimensions rather than for toxicity itself.

Rather than instrumenting those other dimensions as additional endogenous treatments, I hold the reviewer's style in them fixed. I keep only the focal feedback type instrumented and add the non-focal reviewer instruments, $Z^{\mathrm{P}} * i, t$ and $Z^{\mathrm{C}} * i, t$, as controls in both stages. For example, the toxicity specification controls for the assigned reviewer's positivity and constructiveness tendencies, so identification comes from variation in reviewer toxicity among reviewers with similar styles in the other dimensions. Figure D3 shows that the estimates are again nearly identical to the baseline.

5.5.3 Controlling for Reviewer Characteristics and Strictness

The exercises above address whether the estimated effects reflect differences in feedback. A separate concern is that the feedback instruments also capture other reviewer attributes that affect developer outcomes, violating the exclusion restriction. Reviewers with different feedback styles may differ in experience, reputation, demographics, or strictness. In particular, reviewers who tend to use toxic language may also impose more demanding standards. Assignment to such reviewers could affect subsequent developer output through their approval and revision decisions, independently of toxic feedback tone.

To address this concern, I control for observable reviewer characteristics and strictness in both stages of the baseline specification. Reviewer characteristics include the *first year of GitHub activity*, a proxy for seniority; *the number of followers*, a proxy for reputation; and *gender* and *race*. My primary measure of *strictness* is the reviewer's approval rate, defined as the share of cases the reviewer approves. This measure captures the reviewer's final decision and corresponds to the decision margin emphasized in the judge-leniency literature. As an additional control, I include the *revision request rate*, defined as the share of cases in which the reviewer requests changes.[26] For each measure, I construct a leave-developer-out mean using cases involving other developers, following the construction of the feedback instruments in Section 4.1.

Panel (a) of Figure D4 relates reviewer characteristics and strictness to the reviewer-toxicity instrument, with team-year fixed effects. Reviewer characteristics explain little of the variation in toxicity. Gender and follower count are unrelated to toxicity, while more senior reviewers and reviewers predicted to be Asian are slightly less toxic. Toxic reviewers are also less likely to approve submissions and more likely to request revisions.

[26] Revision request rate is a more demanding control and, in one respect, a less clean one: a revision request is issued in the same review as the feedback itself, and toxic feedback accompanies a revision request about twice as often as other feedback types (42% versus 23% of cases). Holding the reviewer's revision request propensity fixed therefore removes not only strictness but also part of what a toxic feedback message is, so estimates that control for it are a lower bound on the effect of tone.

Panels (b) and (c) add these controls to both stages of the baseline specification. Controlling for reviewer characteristics and the approval rate leaves the estimates largely unchanged. The estimated effect of toxic feedback on code quality moves from 0.091 to 0.085, and its effect on new code lines moves from 0.56 to 0.55. The estimates for positive and constructive feedback are similarly stable. Observable reviewer characteristics and approval leniency therefore do not account for the baseline results.

Adding the revision request rate leaves the code-quality results largely unchanged: the effects of all three feedback dimensions remain statistically significant. The estimated effect of toxic feedback on code quantity declines in magnitude to 0.47, compared with 0.56 in the baseline, while the effects of positive and constructive feedback remain small and statistically insignificant. Overall, the estimates are robust to controlling for reviewer characteristics and approval leniency.

5.6 Unobserved In-Person Feedback

The analysis observes only feedback delivered on GitHub, yet reviewers and developers on the same team may also interact in person, and the direction of the resulting bias is ambiguous. Consider toxic feedback. If reviewers who are toxic online are also toxic in person, the on-platform estimate $\hat{\beta}_t^{\text{online}}$ overstates the online-specific effect, since it absorbs both channels. If instead reviewers temper their online feedback and redirect toxicity to in-person exchanges, the on-platform estimate understates it. The same logic applies to positive and constructive feedback: whether in-person interaction reinforces or substitutes for online feedback determines whether the measured effect is too large or too small.[27]

Co-located vs. Multi-located Teams. In-person feedback requires physical proximity, so offline feedback is more likely when team members share a location. I compare co-located teams, whose members are more likely to interact in person, with multi-located teams, for which platform feedback is more likely to capture most of the feedback. This comparison helps assess whether unobserved offline feedback attenuates the estimated effects. Because co-location is a team-level characteristic, its main effect is absorbed by the team-by-year fixed effects. The interaction coefficient η_t^{θ} is identified by comparing the relationship between feedback and outcomes across co-located and multi-located teams, using within-team-year variation in feedback. It is not identified by teams changing co-location status. A causal interpretation therefore requires that the two types of teams do not differ in other characteristics that also affect how developers respond to feedback.

[27] Two features of the setting limit the scope for unobserved offline feedback. Code review is conducted on the platform by design: reviewers record their approval or change requests through GitHub to merge code, so the substantive feedback on a submission is captured on-platform. More broadly, developers and reviewers tend to consolidate work-related communication within a single platform rather than switching channels.

$$
\begin{aligned}
Y_{im,[t+1,t+4]} &= \beta^{\theta} I^{\theta}_{im,t} + \eta^{\theta}\left(I^{\theta}_{im,t} \times C_m\right) + \mathbf{X}'_{i,t}\Gamma + \gamma_{m,\text{year}(t)} + \varepsilon_{im,t}, \\
I^{\theta}_{im,t} &= \pi^{\theta}_1 Z^{\theta}_{j,-i} + \pi^{\theta}_2\left(Z^{\theta}_{j,-i} \times C_m\right) + \mathbf{X}'_{i,t}\delta^{I} + \gamma_{m,\text{year}(t)} + u^{I}_{im,t}, \\
I^{\theta}_{im,t} \times C_m &= \lambda^{\theta}_1 Z^{\theta}_{j,-i} + \lambda^{\theta}_2\left(Z^{\theta}_{j,-i} \times C_m\right) + \mathbf{X}'_{i,t}\delta^{IC} + \gamma_{m,\text{year}(t)} + u^{IC}_{im,t}.
\end{aligned}
$$

I classify a team as co-located if all members are based in the same country (28.17% of teams). This measure overstates true physical proximity because members of the same country may still be geographically distant. The measured co-located group is therefore an upper bound on the set of teams with meaningful in-person interaction. Any resulting measurement error attenuates the estimated interaction toward zero. Consequently, a null interaction should be interpreted as weak rather than conclusive evidence against an offline channel. I report this exercise as a bound, since a sharper same-city classification is not feasible given that GitHub profile locations are typically available only at the country level.[28] Table G1 reports the interaction estimates η^{θ} and nearly all are statistically insignificant.

6 Discussion

The analysis so far has examined how feedback affects developers. This section turns to the reviewers who provide it. I first assess how much variation in reviewer value-added can be predicted by feedback. I then examine reviewers' prior exposure to toxicity and consider why toxic feedback persists despite its productivity costs.

6.1 Feedback as a Component of Manager Quality

Reviewers differ in their effects on developer productivity. To understand what distinguishes more effective reviewers, I examine how much of the variation in reviewer quality can be predicted by the feedback they give.

I measure manager quality using reviewer value-added (VA) to developer productivity, following the forecast-based approach of Chetty, Friedman, and Rockoff (2014). Reviewer VA captures whether developers produce more or better code after assignment to a reviewer, relative to their own baseline and conditional on developer and case characteristics. I first estimate VA in the automatic assignment sample and then extend the estimator to the full sample. Among reviewers observed in both settings, VA rankings are strongly correlated, providing supporting evidence for this extension (Appendix E.1). The differences are economically meaningful: a one standard de-

[28] Location is inferred from developers' self-reported profiles; see Appendix A.6. I construct C_{im} from team members with an observed location: a team is co-located if all such members report the same country, and multi-located otherwise. Teams keep all their developer-team-week observations in the regressions. As a more flexible alternative, I identify the country with the largest share of located members and classify teams below the sample median share as multi-located, yielding a more balanced split. Figure G7 reports results from separate regressions in each subsample.

viation increase in reviewer VA corresponds to 0.20 standard deviations higher code quantity and 0.10 standard deviations higher code quality.

Reviewer VA reflects feedback as well as other aspects of reviewing. To assess how informative feedback is about VA, I train an XGBoost model on 80% of the sample and evaluate its predictions on the remaining 20%. I compare specifications using reviewer demographics, feedback quantity, text embeddings, and the three feedback dimensions studied in this paper: toxicity, positivity, and constructiveness.

Figure 10 reports results for VA measured from code quantity. Reviewer gender and race explain little of the out-of-sample variation, whereas feedback is substantially more predictive. In the automatic assignment sample, feedback quantity alone explains about 7% of the variation. Adding text embeddings raises this share to about 22%, an increase of roughly 15 pp. Replacing the embeddings with toxicity, positivity, and constructiveness recovers 84% of this incremental predictive power. These three dimensions therefore capture much of the information in feedback text that predicts reviewer quality beyond feedback quantity. The full sample yields a similar pattern, as does VA measured from code quality (Appendix E.3).

Together with the earlier causal estimates, these results highlight feedback as a measurable component of manager quality. Organizations that record managerial feedback may therefore use these measures to learn about manager quality beyond observable demographic characteristics.

6.2 Why Are Some Reviewers Toxic?

Reviewer fixed effects explain most of the variation in toxic feedback (Figure 3). To explore these differences, I compare reviewers who never send toxic feedback with those who do so repeatedly. The groups are similar in seniority, managerial status, location, and postgraduate education. Repeatedly toxic reviewers nevertheless have less work experience and are less likely to work at a large technology firm in the adjusted comparisons (Appendix Table F3).[29]

The groups differ more consistently in their prior exposure to toxic reviewing. Before becoming reviewers, repeatedly toxic reviewers were more likely to have received toxic feedback (36% versus 24%). The reviewers they encountered in the automatic assignment sample had a leave-one-out toxicity rate of 1.18%, compared with 0.72% for those encountered by never-toxic reviewers, a difference of approximately 64%. The teams in which they worked also had a higher toxicity rate (0.34% versus 0.17%). These exposure differences remain statistically significant (Table F2).

These patterns are consistent with reviewers adopting the behavior of those who reviewed them (Tepper et al., 2017; Molitor, 2018). Exposure through automatic assignment limits concerns about developers directly choosing particular reviewers, but the comparisons remain descriptive and cannot rule out sorting across teams or shared environmental influences. If exposure shapes subsequent reviewing behavior, the costs of toxic feedback extend beyond the productivity losses

[29] The adjusted differences are 0.74 fewer years of experience and a 3.5 pp lower probability of employment at a large technology firm, significant at the 5% and 10% levels, respectively. The profile comparison includes reviewers with at least 20 review cases who are matched to LinkedIn.

estimated in Section 5 to how recipients later treat other developers.

6.3 Why Might Toxic Feedback Persist?

Why does toxic feedback persist despite its productivity costs? One possible explanation is that these costs are difficult to infer from observed performance. In OLS (Table 5), developers who receive toxic feedback produce 6.72% more code and 8.33% more non-code activity. The IV estimates instead show that toxic feedback reduces code quantity. An organization or reviewer relying on the observed correlations may therefore conclude that toxic feedback is effective. Developers may continue to produce substantial output after receiving it while producing less than they would have in its absence. The difficulty of recognizing this lost output may allow harmful feedback practices to go uncorrected.

The observed career outcomes also provide little evidence of penalties for toxic reviewing. Among reviewers matched to LinkedIn, those who send at least one toxic review in their first year of reviewing show no statistically significant difference in the probability of promotion over the following three years, either within their firm or across employers. They are also 7.6 pp more likely to switch employers (Appendix Table F4). These associations do not identify employers' responses to toxicity, but they suggest that toxic reviewing is compatible with continued career advancement and mobility.

7 Conclusion

Economists have long known that managers matter for worker productivity. This paper focuses on feedback, a common managerial activity, and shows that how managers give feedback matters. Using more than 200 million code review messages from software teams on GitHub and exploiting the quasi-random reviewer assignment for identification, I find that toxic feedback reduces both the quantity and quality of developers' code over the following month, whereas nontoxic criticism has no detectable effect on code quantity and a substantially smaller effect on quality. This suggests that much of the cost of toxicity comes from its toxic tone rather than from criticism itself. Positive feedback raises code quality and retention at the firm and generates spillovers to coworkers and other teams. Constructive feedback operates on a different margin: it leaves new output unchanged but lowers the quality of new code, as developers shift effort toward revising the reviewed case.

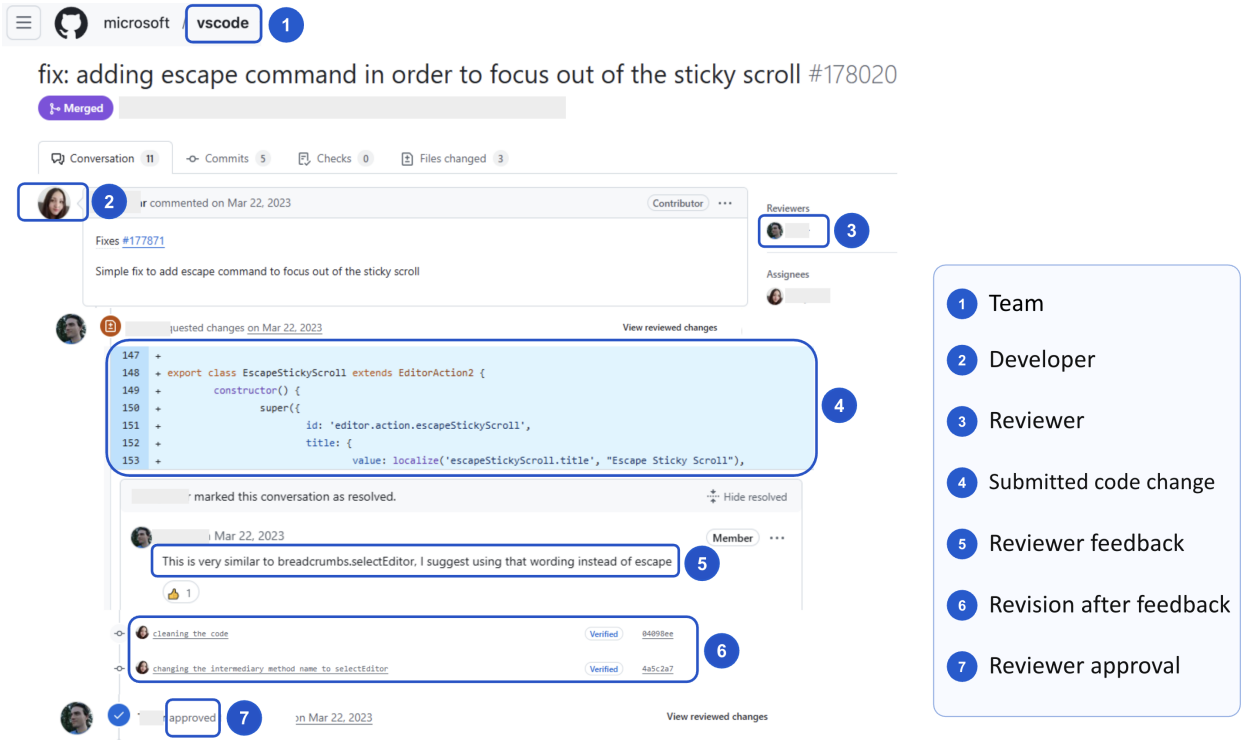
If toxic feedback is so costly, why does it persist? Toxic reviewers cannot be identified from their professional backgrounds, their cost is hidden in the correlations organizations observe, and they are no less likely to be promoted. Toxicity also appears to be passed on: reviewers who become toxic were more often reviewed by toxic reviewers themselves. This evidence is descriptive, but it suggests that the cost of toxic feedback extends beyond the developers who receive it.

Two implications follow. First, the tone of feedback is a managerial instrument that can affect worker motivation and productivity. It differs from traditional organizational tools such as pay

and promotion because changing how feedback is delivered requires few additional financial or organizational resources. Managers can criticize poor work without attacking the worker and can express the same substantive content in a more positive tone, which may improve subsequent productivity. Second, feedback reveals a persistent component of managerial quality. Feedback explains about 22% of the variation in reviewer value-added, and toxicity, positivity, and constructiveness alone capture most of this.

More broadly, the paper shows how everyday managerial behavior can now be measured directly and at scale. Feedback is one way managers direct work, motivate workers, and coordinate production. I focus on tone and informational content because they capture two broad dimensions of feedback: how it is delivered and what it conveys. The same approach can be applied to other dimensions of managerial communication, such as gendered language, formality, and stylistic variation, and to other records of managerial behavior, including emails, meetings, and performance reviews. Separating learning from sorting, and understanding whether managers target feedback strategically or adjust it in response to workers, are natural next questions. The substantial heterogeneity across developers also points to adaptive or targeted feedback. Future work could study upward feedback and which forms of managerial communication make teams more productive.

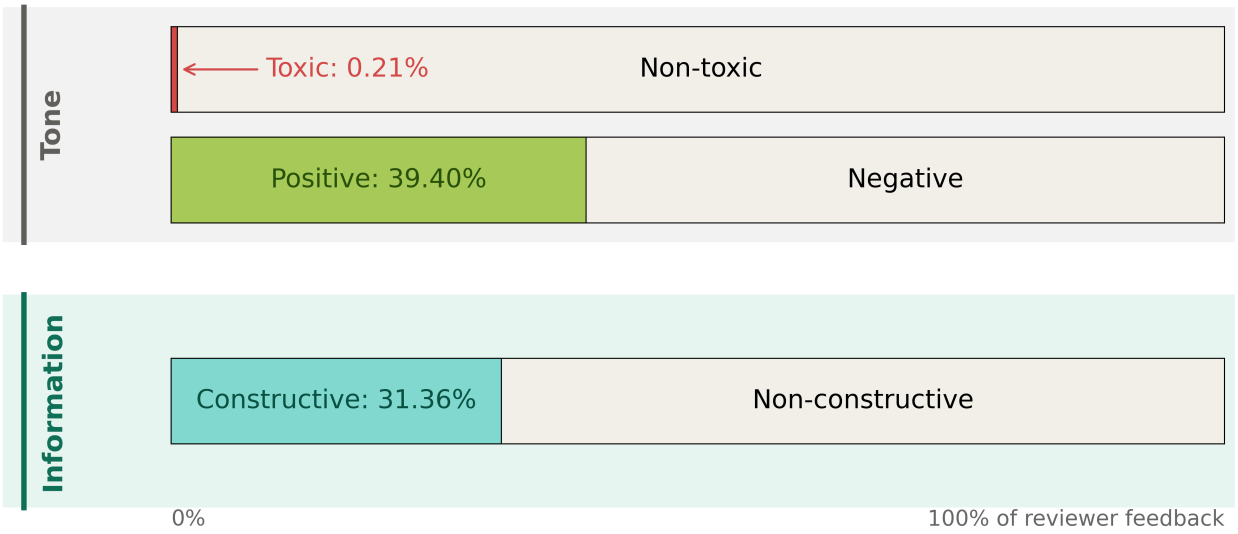
Figure 1: Example of GitHub Code Review



Notes: This figure shows an example of a code review in the Microsoft VS Code team (a "repository," in GitHub terminology). A developer submits a code change (a "pull request") to the team's codebase, and a reviewer checks the code and gives feedback on the submission. The developer then revises the code in response. The reviewer decides whether to approve the change. In this example, the revised change is approved.

Figure 2: Classification of Reviewer Feedback by Tone and Information

(a) Share of Feedback by Tone and Information

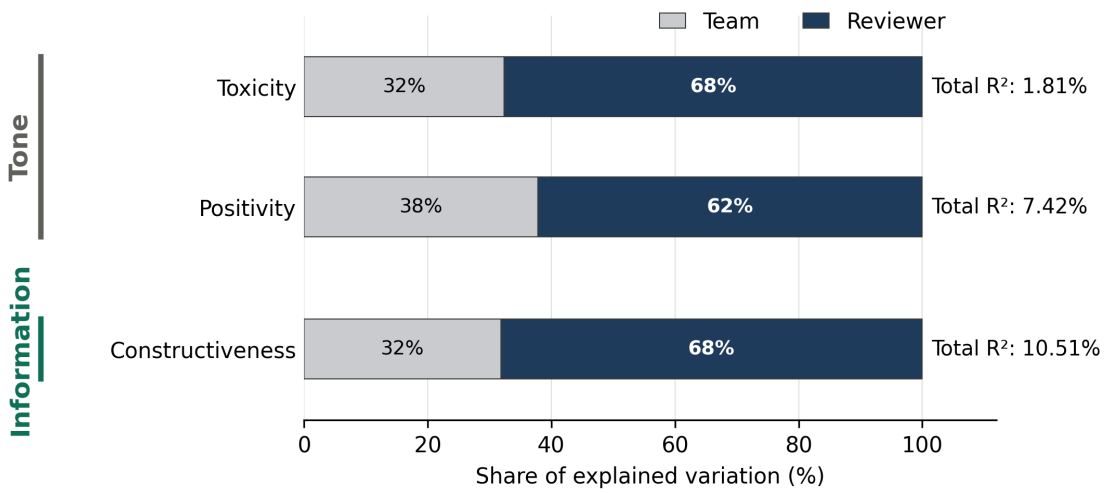


(b) Joint Distribution of Feedback Tone

		Toxicity	
		Toxic	**Non-toxic**
Positivity	**Positive**	Toxic Positive 0.02%	Non-toxic Positive 39.38%
	Negative	Toxic Negative 0.19%	Non-toxic Negative 60.41%

Notes: The figure classifies reviewer feedback along two dimensions: tone, which comprises toxicity and positivity, and information, measured by constructiveness. Toxicity captures intentional harm, positivity a supportive and encouraging tone, and constructiveness, whether feedback provides specific, actionable information about the submitted code. **Panel (a)** reports the share of feedback in each dimension in the automatic assignment sample. Each bar represents 100% of reviewer feedback; the shaded portion is the share in the named dimension, and the remainder is its complement. Table 1 reports the corresponding shares in the full sample. **Panel (b)** reports the joint distribution of feedback tone. Columns classify feedback by toxicity (toxic or non-toxic) and rows by positivity (positive or negative). The four cells sum to 100%. Table 3 reports the full joint distribution across all three dimensions, together with examples for each type.

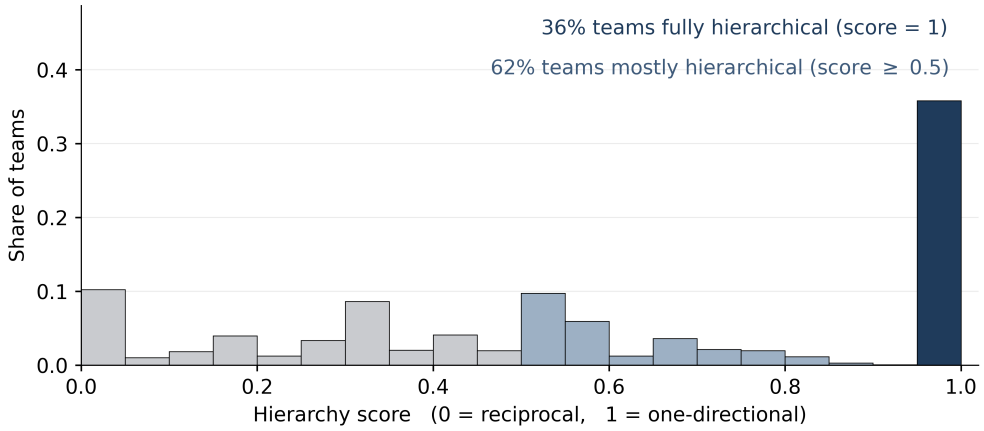
Figure 3: Sources of Feedback Variation



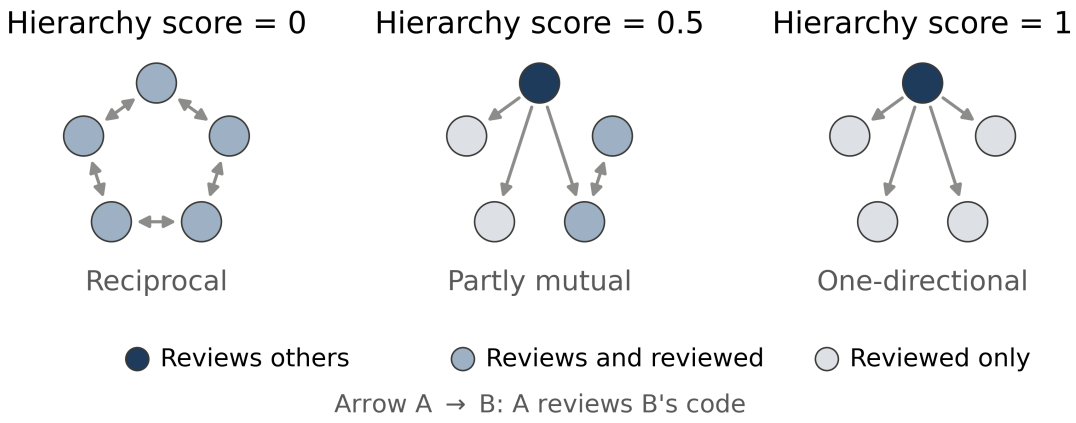
Notes: This figure decomposes the explained variation in feedback tone and information. Tone is measured by toxicity and positivity. Information is measured by constructiveness. The unit of observation is a developer–team–week, and the dependent variable is the share of feedback a developer receives in that team-week that is toxic, positive, or constructive. Bars report the share of explained variation accounted for by team and reviewer fixed effects. Total R^2 values are reported to the right of each bar. Additional factors, including developer fixed effects and case characteristics such as whether the case involves an enhancement or a bug, each add less than 0.01 pp and are not shown. The sample is the random reviewer subsample.

Figure 4: Feedback Hierarchy within Team

(a) Distribution of Team Hierarchy Scores



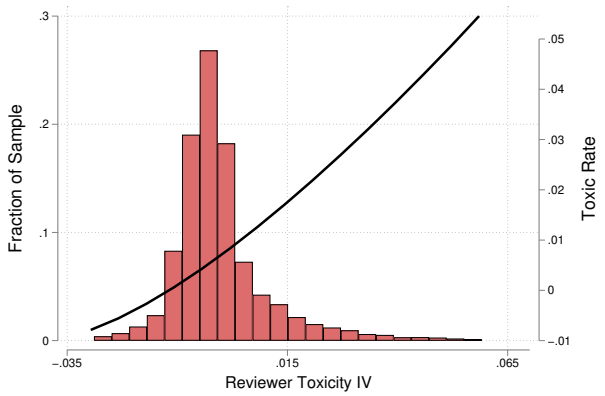
(b) Example Feedback Networks



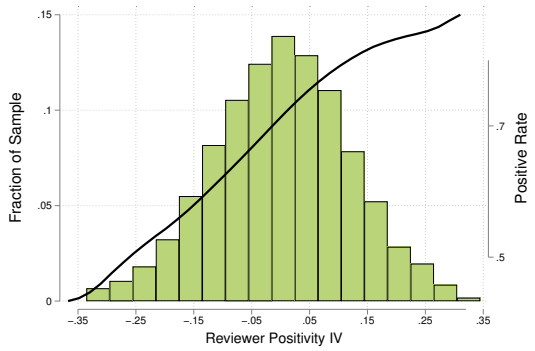
Notes: This figure summarizes how feedback flows within teams. For each team, the hierarchy score is $1 - 2R/E$, where E is the number of directed reviewer–developer pairs and R is the number of reciprocated reviewer-developer pairs. It ranges from 0, when members review each other symmetrically (reciprocal), to 1, when feedback is one-directional, with some members reviewing others but rarely receiving feedback in return. Higher scores indicate a more hierarchical review structure. Teams with a single case or a single member are excluded. **Panel (a)** plots the distribution of hierarchy scores across teams. The navy bar marks fully hierarchical teams (score $= 1$) and the shaded region mostly hierarchical teams (score ≥ 0.5). The concentration near 1 is consistent with feedback that resembles manager-like evaluation rather than symmetric peer review. **Panel (b)** shows three stylized five-member teams, holding the layout fixed and varying only the direction of feedback. Node color denotes a member's role: reviews only, reviews and is reviewed, or reviewed only. A directed edge A $\rightarrow$ B means A reviews B's code. Figure C5 shows the distribution for teams with exactly five members and for the automatic assignment sample.

Figure 5: Distribution of Reviewer Instruments and First Stage

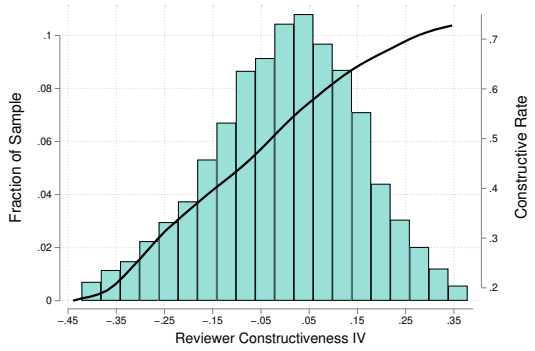
(a) Toxicity



(b) Positivity



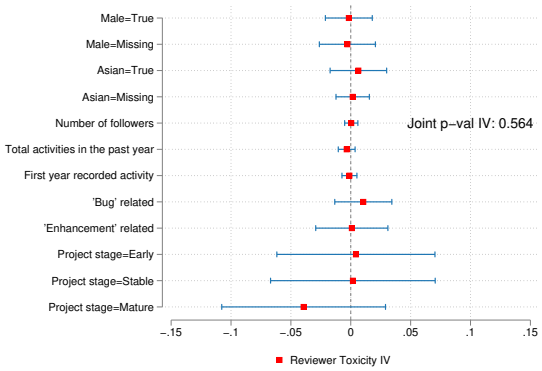
(c) Constructiveness



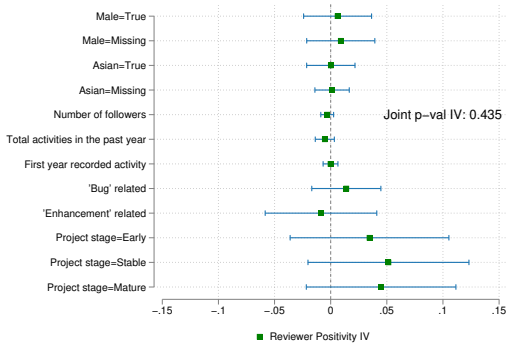
Notes: Each panel plots the distribution of reviewer instruments and the nonparametric first-stage relationship between the instrument and the probability that the focal case receives the corresponding feedback type. The instrument is the reviewer's leave-developer-out average feedback share, residualized by team-by-year fixed effects. I include teams with at least 2 reviewers and reviewers who have handled at least 10 cases, leaving 480,697 cases and 10,873 reviewers, with an average of 44.2 cases per reviewer. The bars show the distribution of reviewer instrument values, and the solid line shows the kernel-weighted local polynomial fit of the first stage. Panels are ordered by the feedback classification: toxicity and positivity measure tone, and constructiveness measures information content. The sample is the automatic assignment sample, excluding the top and bottom 1% of instrument values.

Figure 6: Balance of Reviewer Instruments

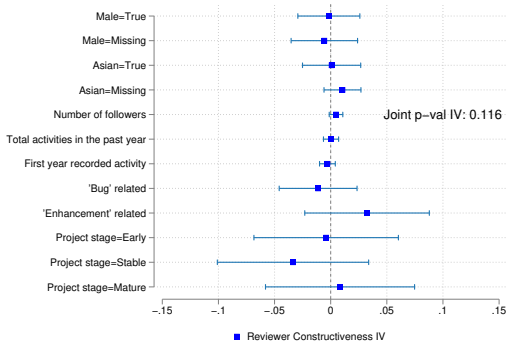
(a) Toxicity



(b) Positivity

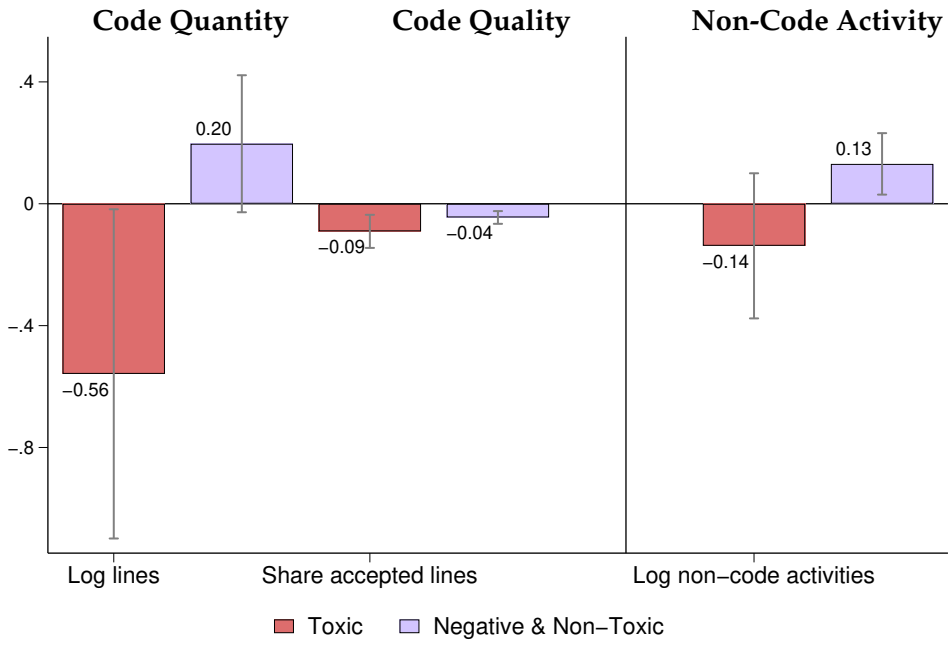


(c) Constructiveness



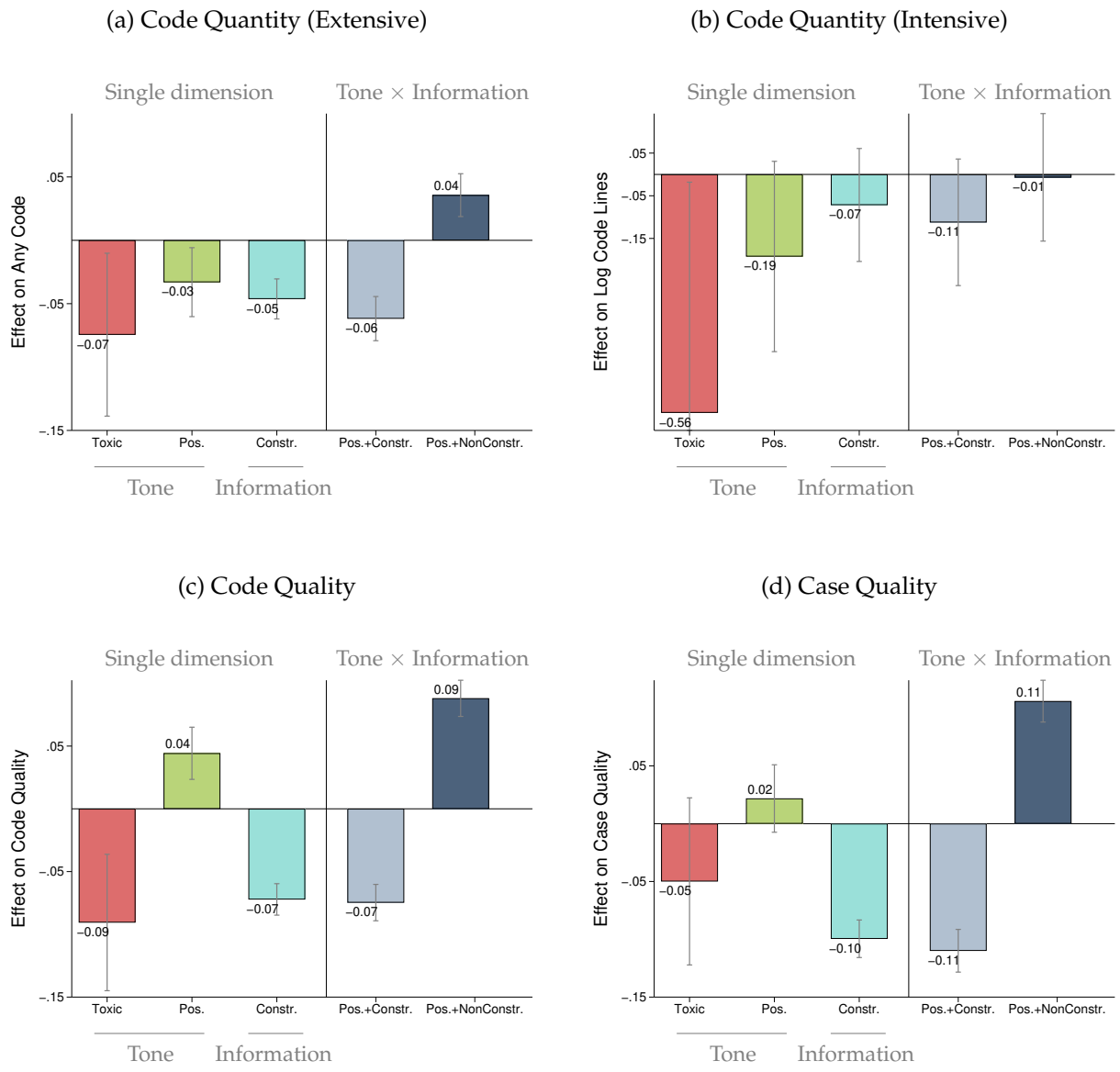
Notes: Each panel reports OLS coefficients from regressions of the reviewer instrument on standardized developer and case characteristics. The instrument is the reviewer's leave-developer-out average feedback share, residualized by team-by-year fixed effects. Coefficients are shown with 95 % confidence intervals. The p-values shown in the panels are from joint significance tests of all characteristics. Standard errors are clustered at the reviewer level. The sample is the automatic assignment sample. Appendix A.4 and Appendix A.10 define the variables.

Figure 7: Effects of Toxic versus Respectful Critical Feedback on Developer



Notes: This figure plots 2SLS estimates of the effect of toxic feedback (red) and respectful criticism (purple) on developer productivity, where respectful criticism is feedback coded as negative but non-toxic. Each bar is from a separate regression of the outcome on an indicator for receiving the relevant type of feedback. Outcomes are measured on activity within the team during weeks 1–4 after a review: log code lines (conditional on coding), the share of new lines accepted, and log non-code activity such as answering user questions. All regressions include team-by-year fixed effects and controls for the developer's lagged activity and message deciles. Lines indicate 90% confidence intervals. Standard errors are clustered at the reviewer level.

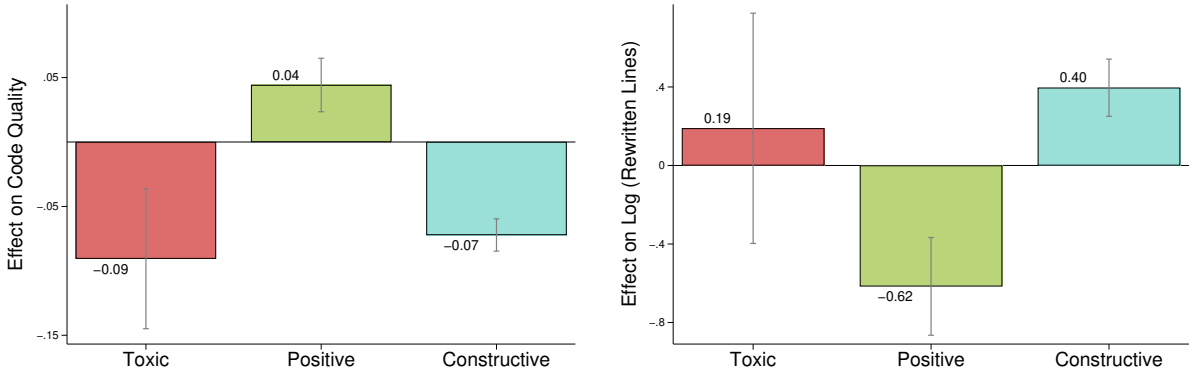
Figure 8: Joint Effects of Feedback Tone and Information on Developer Productivity



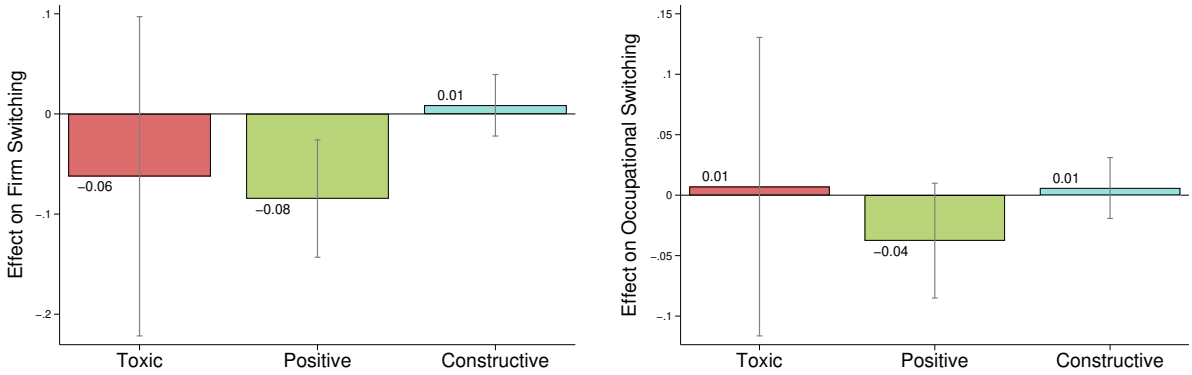
Notes: This figure reports 2SLS estimates of the effects of single and joint feedback types on developer productivity within a team in the next 1–4 weeks following feedback. The first three bars correspond to single feedback types: toxic, positive, and constructive; the fourth is non-toxic & negative; the fifth, positive & constructive; and the sixth, positive & non-constructive. Joint feedback types with very small shares are omitted, based on the share reported in Table 3. **Panels (a)** and **(b)** report *Code Quantity*, the extensive margin (Ever) and the intensive margin (Log). **Panel (c)** reports *Code Quality*, the share of lines of new code accepted, and **Panel (d)** reports *Case Quality*, the share of new cases merged into the team's codebase. Feedback exposure is instrumented using reviewer-level feedback tendencies within the random assignment sample. Lines indicate 90% confidence intervals, which are trimmed at –0.6 and 0.3 in the graph to focus on the main range of estimated effects. Standard errors are clustered at the reviewer level.

Figure 9: Effects of Feedback on Developer Outcomes

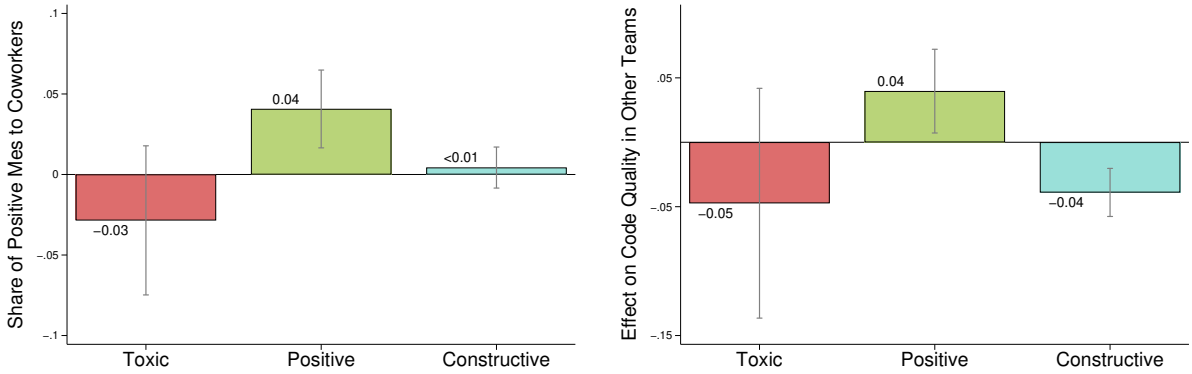
(a) Tradeoff (GitHub): New Code Quality and Old Code Rewritten



(b) Retention (LinkedIn): Firm and Occupational Switch

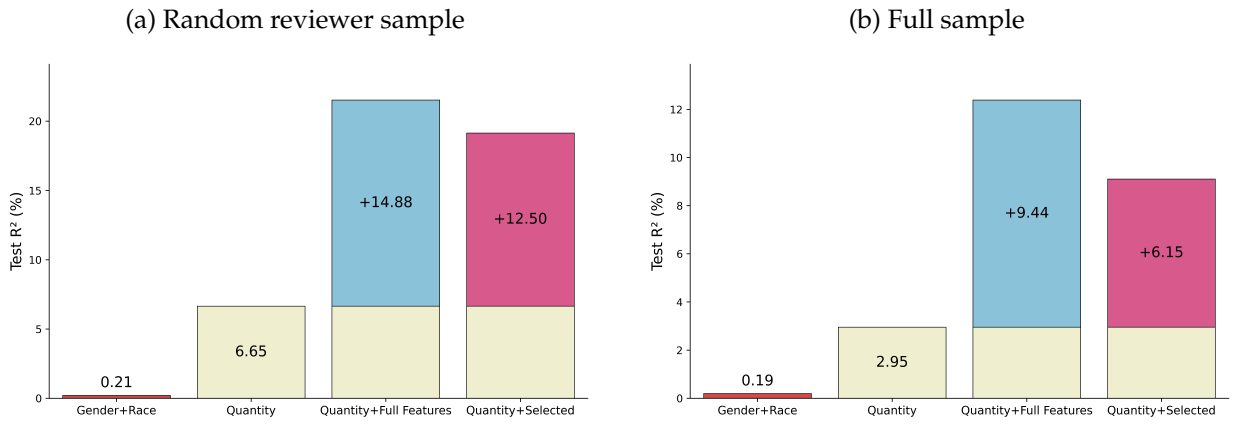


(c) Spillover Effects (GitHub): Coworkers and Other Teams



Notes: This figure reports 2SLS estimates of the effects of different feedback types on developer outcomes after receiving feedback. Panel (a) shows the tradeoff between new code quality and old code rewritten. New code quality is measured by the code-level acceptance outcome, reported in column (3) under "Code Quality" in Table 5. Panel (a2) reports the effect of rewritten code measures the log of code revisions made to reviewed code. Panel (b) reports the effects of feedback on developer career mobility over the next year using LinkedIn data, measured by firm switches and occupational switches. Panel (c) presents spillover effects of feedback on the developer's messages to coworkers (c1) and on the developer's code in other teams (c2). Panel (c1) shows how receiving feedback in the focal team affects the share of positive messages that developers send to coworkers. Panel (c2) reports effects on the quality of code developers submit to other teams. Feedback exposure is instrumented using reviewer-level feedback tendencies within the random assignment sample. Lines indicate 90% confidence intervals. Standard errors are clustered at the reviewer level.

Figure 10: Predicting Reviewer Value-Added from Feedback: Developer Code Quantity



Notes: This figure reports out-of-sample R^2 from models predicting reviewer value-added (VA) to developer code quantity. Panel (a) uses the automatic-assignment sample; Panel (b) uses the full sample. VA is estimated using the forecast-based approach of Chetty, Friedman, and Rockoff (2014). Prediction models are trained on 80% of the sample and evaluated on the remaining 20%. "Demographics" includes reviewer gender and race. "Quantity" is the number of feedback messages a reviewer sends. "Quantity + Full Text" adds 768-dimensional text embeddings. "Quantity + Selected Feedback" adds toxicity, positivity, and constructiveness. Appendix E.3 provides model details and analogous results for code quality.

Table 1: Descriptive Statistics

Description	Full Sample	Random Reviewer Sample
Panel A: Teams and Cases		
Teams (#)	525,841	3,457
Cases (#)	46,159,639	4,228,687
Cases with reviewers (%)	20.99	33.42
Reviewers (#)	364,889	35,072
Avg. cases per team (#)	88	1,223
Avg. team size (#)	4.0	19.42
Panel B: All Messages		
Messages (#)	203,692,177	24,453,479
Avg. message per case (#)	4.41	5.78
Avg. messages per team (#)	387	7,074
Panel C: Reviewer Feedback		
Messages sent by reviewers (%)	36.33	32.60
Toxic (%)	0.21	0.21
Positive (%)	41.87	39.40
Constructive (%)	34.65	31.36
Panel D: Team Feedback Structure		
Reciprocal reviewing rate (%)	52.17	23.67

Notes: This table reports summary statistics for teams with at least one reviewed case (Full Sample) and for teams that adopted random reviewer assignment, from the month of adoption onward (Random Reviewer Sample), between 2017 and 2023. Panel A reports team and case characteristics. Averages are means across teams; team size is the number of formal team members over the sample period. Panel B covers all messages, sent by both team members and external users. Panel C restricts to feedback messages sent by reviewers in cases with at least one reviewer, the focus of this paper; the toxic, positive, and constructive rows report shares of reviewer feedback. Toxic and positive feedback are labeled with BERT-family classifiers and constructive feedback with an LLM. For the Random Reviewer Sample, all reviewer feedback is used; for the Full Sample, toxic and positive feedback use all reviewer messages, while constructive feedback uses a random subsample due to budget constraints. Panel D characterizes team hierarchy: the reciprocal reviewing rate is the probability that a developer also reviews the reviewer who reviewed them. Reviewer seniority and the share of reviewers who also write code are reported in Appendix A.7.

Table 2: Developer–Team Weekly Productivity Summary Statistics

	N	Mean	Std. Dev	P75	P95	Max
Focal Team						
Code Lines	12,216,340	1,119.21	54,898.64	3	537	41,436,660
Rewritten Lines	12,216,340	507.81	39,439.85	0	114	29,285,783
Non-Code Activities	12,216,340	6.36	39.89	5	26	62,784
Cases	12,216,340	3.18	11.83	3	13	16,722
Case Quality	3,607,865	0.63	0.42	1	1	1
Code Quality	4,095,714	0.74	0.38	1	1	1
Other Teams						
Code Lines	12,216,340	2,812.08	199,277.21	0	617	78,347,840
Rewritten Lines	12,216,340	861.94	56,482.51	0	97	29,285,786
Non-Code Activities	12,216,340	549.94	2,938.83	43	1,819	62,801
Cases	12,216,340	85.41	457.56	16	97	16,741
Case Quality	2,054,565	0.67	0.34	1	1	1
Code Quality	2,467,433	0.70	0.40	1	1	1

Notes: This table shows the summary statistics of productivity measures for all developers in the random reviewer sample from 2017 to 2023. Each observation is a developer-team-week. "Focal Team" refers to the team in the current row; "Other Teams" aggregates activity from all remaining teams during the same week. Winsorization at the 95th percentile is applied to all continuous variables to reduce the influence of extreme outliers, which are common in developer productivity data (e.g., unusually large code commits or bursts of activity). Statistics are computed on the raw data. In the regression analysis, all continuous outcomes are winsorized at the 95th percentile to limit the influence of extreme values, which are common in developer productivity data (for example, unusually large commits). Bounded outcomes — case quality and code quality are unaffected by winsorization. See Appendix A.8 and Appendix A.9 for variable definitions.

Table 3: Distribution and Examples of Reviewer Feedback Types

Panel A: Distribution of Feedback Types

Constructive

	Positive	**Negative**
Toxic	0.000	0.01
Non-Toxic	9.43	21.91

Non-Constructive

	Positive	**Negative**
Toxic	0.02	0.17
Non-Toxic	29.95	38.50

Panel B: Examples of Feedback Types

Toxic	**Positive**	**Constructive**	**Feedback Message**	**Share (%)**
1	1	0	Sounds good, can't wait to restaple my ass on.	0.02
1	0	1	You should just keep the file name, not the full file path. This is Pantherinternal and doesn't provide value...	0.01
1	0	0	You will have to answer for it and you are talking nonsense.	0.17
0	1	1	Docs look good overall. I think it would also be helpful to add a few more screenshots showing the overall page...	9.43
0	1	0	Looks good to me.	29.95
0	0	1	Still feels redundant. An angle should just be a union of float and FreeParameter. Once the parameter is bound...	21.91
0	0	0	We need a way to detect the cluster domain.	38.50

Notes: This table shows the distribution of reviewer feedback by toxicity, positivity, and constructiveness in the random reviewer sample. Panel A reports the share of feedback in each category. Entries are percentages and sum to 100. Panel B reports representative examples from each feedback type. The share column reports the percentage of feedback in the corresponding category.

Table 4: First-Stage Estimates by Feedback Type

	(1)	(2)	(3)
Panel A: Toxic Feedback			
Reviewer Toxicity	0.708***	0.703***	0.703***
	(0.013)	(0.013)	(0.013)
F-Stat.	2931.913	2866.244	2868.936
Dependent mean	0.008	0.008	0.008
Panel B: Positive Feedback			
Reviewer Positivity	0.843***	0.820***	0.820***
	(0.005)	(0.005)	(0.005)
F-Stat.	34335.891	28057.043	27450.902
Dependent mean	0.706	0.706	0.706
Panel C: Constructive Feedback			
Reviewer Constructiveness	0.875***	0.864***	0.863***
	(0.004)	(0.004)	(0.004)
F-Stat.	61420.629	60183.796	60733.265
Dependent mean	0.518	0.518	0.518
Team FE	✓		
Team × Year FE		✓	✓
Baseline controls			✓
N	1,179,525	1,179,525	1,179,525

Notes: Each panel reports a first-stage regression where the dependent variable is an indicator for receiving a specific type of feedback. Baseline controls include developer gender, race, GitHub activity in the year before submission, and the first year of GitHub activity. Case characteristics account for issue labels such as Bug and Enhancement, and project stage is defined based on the release version of the team product. Appendix A.4 provides details on the gender and race variables, and Appendix A.10 describes the construction of project stage. Standard errors are clustered at the reviewer level. *$p < .10$, ** $p < .05$, *** $p < .01$.

Table 5: Effect of Reviewer Feedback on Developer Productivity Within a Team

	Code Quantity		Code Quality		Non-Code Quantity	
	Ever	Log	Code	Case	Ever	Log
Panel A: Tone						
Toxic Feedback						
OLS	-0.013***	0.065***	0.001	-0.007***	0.004***	0.080***
	(0.003)	(0.015)	(0.002)	(0.002)	(0.001)	(0.008)
2SLS	-0.074*	-0.558*	-0.091***	-0.050	-0.019	-0.138
	(0.039)	(0.328)	(0.033)	(0.044)	(0.019)	(0.145)
Positive Feedback						
OLS	0.015***	0.178***	-0.003**	-0.005***	0.010***	0.137***
	(0.002)	(0.008)	(0.001)	(0.001)	(0.001)	(0.004)
2SLS	-0.033**	-0.192	0.044***	0.022	-0.011	-0.130**
	(0.017)	(0.136)	(0.013)	(0.018)	(0.008)	(0.061)
Panel B: Information Content						
Constructive Feedback						
OLS	0.017***	0.184***	-0.015***	-0.019***	0.009***	0.122***
	(0.001)	(0.007)	(0.001)	(0.001)	(0.001)	(0.003)
2SLS	-0.046***	-0.072	-0.072***	-0.100***	-0.009**	-0.023
	(0.010)	(0.080)	(0.008)	(0.010)	(0.005)	(0.035)
Dependent mean	0.846	5.018	0.693	0.545	0.957	2.450
N	1,176,459	995,139	1,028,793	998,141	1,176,459	1,125,874
Team × Year FE	✓	✓	✓	✓	✓	✓
Decile t–1 activity	✓	✓	✓	✓	✓	✓
Decile t–1 message	✓	✓	✓	✓	✓	✓

Notes: This table reports OLS, reduced-form (RF), and two-stage least squares (2SLS) estimates of receiving each feedback type on developer outcomes during weeks $t+1$ to $t+4$. Panel A reports the two tone dimensions, toxic and positive feedback; Panel B reports the information dimension, constructive feedback. Figure D2 reports estimates from the augmented IV system in equations (3)–(6), which includes all three feedback dimensions jointly and yields similar estimates. Outcomes are measured on new code written by the developer within the team during weeks 1–4 after receiving feedback. Within each outcome group, *Ever* is the extensive margin (whether the developer is active at all) and *Log* is the intensive margin (log code lines); for Code Quality, *Code* is the share of new lines accepted and *Case* is the share of cases merged. All statistics are reported for the sample used in the 2SLS estimation. All models include team-by-year fixed effects and controls for the developer's lagged activity and message deciles. The unit of observation is a developer-team-week feedback event in the random reviewer sample. Standard errors, in parentheses, are clustered at the reviewer level. * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$.

References

Adams-Prassl, A., K. Huttunen, E. Nix, and N. Zhang (2024). Violence Against Women at Work. *The Quarterly Journal of Economics 139*(2), 937–991.

Agan, A., J. L. Doleac, and A. Harvey (2023). Misdemeanor Prosecution. *The Quarterly Journal of Economics 138*(3), 1453–1505.

Andersen-Gott, M., G. Ghinea, and B. Bygstad (2012). Why Do Commercial Companies Contribute to Open Source Software? *International Journal of Information Management 32*(2), 106–117.

Angrist, J. and G. Imbens (1994). Identification and Estimation of Local Average Treatment Effects.

Angrist, J. D. and B. Frandsen (2022). Machine Labor. *Journal of Labor Economics 40*(S1), S97–S140.

Ashford, S. J. and L. L. Cummings (1983). Feedback as an Individual Resource: Personal Strategies of Creating Information. *Organizational Behavior and Human Performance 32*(3), 370–398.

Bandiera, O., A. Prat, S. Hansen, and R. Sadun (2020). CEO Behavior and Firm Performance. *Journal of Political Economy 128*(4), 1325–1369.

Banihashem, S. K., O. Noroozi, S. Van Ginkel, L. P. Macfadyen, and H. J. Biemans (2022). A Systematic Review of the Role of Learning Analytics in Enhancing Feedback Practices in Higher Education. *Educational Research Review 37*, 100489.

Bekker, P. A. (1994). Alternative Approximations to the Distributions of Instrumental Variable Estimators. *Econometrica*, 657–681.

Beknazar-Yuzbashev, G., R. Jiménez Durán, J. McCrosky, and M. Stalinski (2022). Toxic Content and User Engagement on Social Media: Evidence from a Field Experiment. *Available at SSRN 4307346*.

Bénabou, R. and J. Tirole (2002). Self-confidence and Personal Motivation. *The Quarterly Journal of Economics 117*(3), 871–915.

Bénabou, R. and J. Tirole (2003). Intrinsic and extrinsic motivation. *The review of economic studies 70*(3), 489–520.

Beuschlein, J. (2024). Designing Debt Restructuring: The Adverse Effects on Labor Market Outcomes.

Bhuller, M., G. B. Dahl, K. V. Løken, and M. Mogstad (2020). Incarceration, Recidivism, and Employment. *Journal of Political Economy 128*(4), 1269–1324.

Brynjolfsson, E., D. Li, and L. Raymond (2025). Generative AI at Work. *The Quarterly Journal of Economics*, qjae044.

Chetty, R., J. N. Friedman, and J. E. Rockoff (2014). Measuring the Impacts of Teachers I: Evaluating Bias in Teacher Value-added Estimates. *American Economic Review 104*(9), 2593–2632.

Collis, M. R. and C. Van Effenterre (2025). Workplace Hostility.

Compte, O. and A. Postlewaite (2004). Confidence-enhanced Performance. *American Economic Review 94*(5), 1536–1557.

Dobbie, W., J. Goldin, and C. S. Yang (2018). The Effects of Pre-trial Detention on Conviction, Future Crime, and Employment: Evidence from Randomly Assigned Judges. *American Economic*

Review 108(2), 201–240.

Dobbie, W., P. Goldsmith-Pinkham, and C. S. Yang (2017). Consumer Bankruptcy and Financial Health. *Review of Economics and Statistics 99*(5), 853–869.

Dobbie, W. and J. Song (2015). Debt Relief and Debtor Outcomes: Measuring the Effects of Consumer Bankruptcy Protection. *American Economic Review 105*(3), 1272–1311.

Ederer, F., P. Goldsmith-Pinkham, and K. Jensen (2024). Anonymity and Identity Online. *arXiv Preprint arXiv:2409.15948*.

El-Komboz, L. A. and M. Goldbeck (2024). Career Concerns as Public Good the Role of Signaling for Open Source Software Development.

Emanuel, N., E. Harrington, and A. Pallais (2026). The power of proximity to coworkers. *The Quarterly Journal of Economics*.

Folke, O. and J. Rickne (2022). Sexual Harassment and Gender Inequality in the Labor Market. *The Quarterly Journal of Economics 137*(4), 2163–2212.

Frandsen, B., L. Lefgren, and E. Leslie (2023). Judging Judge Fixed Effects. *American Economic Review 113*(1), 253–277.

Frederiksen, A., L. B. Kahn, and F. Lange (2020). Supervisors and Performance Management Systems. *Journal of Political Economy 128*(6), 2123–2187.

Freimane, M. (2024). Gender Bias, Feedback, and Productivity.

Gallup (2022). How Effective Feedback Fuels Performance. Retrieved April 19, 2025, from https://www.gallup.com/workplace/357764/fast-feedback-fuels-performance.aspx.

Gelfand, M. J., J. L. Raver, L. Nishii, L. M. Leslie, J. Lun, B. C. Lim, L. Duan, A. Almaliach, S. Ang, J. Arnadottir, et al. (2011). Differences Between Tight and Loose Cultures: A 33-nation Study. *Science 332*(6033), 1100–1104.

GitHub (2025). GitHub Enterprise. Accessed: August 27, 2025.

Haegele, I. (2026). Talent hoarding in organizations. *American Economic Review 116*(8), 3110–3151.

Hanu, L. and Unitary team (2020). Detoxify. Github. https://github.com/unitaryai/detoxify.

Hartmann, J., M. Heitmann, C. Siebert, and C. Schamp (2023). More Than a Feeling: Accuracy and Application of Sentiment Analysis. *International Journal of Research in Marketing 40*(1), 75–87.

Hartvigsen, T., S. Gabriel, H. Palangi, M. Sap, D. Ray, and E. Kamar (2022). ToxiGen: A Large-Scale Machine-Generated Dataset for Implicit and Adversarial Hate Speech Detection. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*.

Hattie, J. and H. Timperley (2007). The Power of Feedback. *Review of Educational Research 77*(1), 81–112.

Hoffman, M. and S. Tadelis (2021). People Management Skills, Employee Attrition, and Manager Rewards: An Empirical Analysis. *Journal of Political Economy 129*(1), 243–285.

Hoffmann, M., F. Nagle, and Y. Zhou (2024). The Value of Open Source Software. *Harvard Business School Strategy Unit Working Paper* (24-038).

Holmström, B. (1982). Moral Hazard in Teams. *The Bell Journal of Economics*, 324–340.

Hull, P. (2017). Examiner Designs and First-stage F Statistics: A Caution. *Brown University Working Paper 5*.

Ilgen, D. R., C. D. Fisher, and M. S. Taylor (1979). Consequences of Individual Feedback on Behavior in Organizations. *Journal of Applied Psychology 64*(4), 349.

Jones, B. F. (2021). The Rise of Research Teams: Benefits and Costs in Economics. *Journal of Economic Perspectives 35*(2), 191–216.

Kim, S. Y., Y. Wang, D. Orozco-Lapray, Y. Shen, and M. Murtuza (2013). Does "Tiger Parenting" Exist? Parenting Profiles of Chinese Americans and Adolescent Developmental Outcomes. *Asian American Journal of Psychology 4*(1), 7.

Kluger, A. N. and A. DeNisi (1996). The Effects of Feedback Interventions on Performance: A Historical Review, a Meta-analysis, and a Preliminary Feedback Intervention Theory. *Psychological Bulletin 119*(2), 254.

Kolesár, M. (2013). Estimation in an Instrumental Variables Model with Treatment Effect Heterogeneity.

Kolesár, M., R. Chetty, J. Friedman, E. Glaeser, and G. W. Imbens (2015). Identification and Inference with Many Invalid Instruments. *Journal of Business & Economic Statistics 33*(4), 474–484.

Kolhatkar, V., N. Thain, J. Sorensen, L. Dixon, and M. Taboada (2020). Classifying Constructive Comments. *arXiv Preprint arXiv:2004.05476*.

Kőszegi, B., G. Loewenstein, and T. Murooka (2022). Fragile Self-esteem. *The Review of Economic Studies 89*(4), 2026–2060.

Laohaprapanon, S., G. Sood, and B. Naji (2017). Ethnicolr Algorithm.

Lee, D. S., J. McCrary, M. J. Moreira, and J. Porter (2022). Valid T-ratio Inference for IV. *American Economic Review 112*(10), 3260–3290.

Liu, Y., M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov (2019). Roberta: A Robustly Optimized Bert Pretraining Approach. *arXiv Preprint arXiv:1907.11692*.

Marschak, J. and R. Radner (1958). Economic Theory of Teams. Chapter 5.

McIntosh, S., Y. Kamei, B. Adams, and A. E. Hassan (2016). An Empirical Study of the Impact of Modern Code Review Practices on Software Quality. *Empirical Software Engineering 21*, 2146–2189.

Minni, V. (2026). Making the invisible hand visible: Managers and the allocation of workers to jobs. *The Quarterly Journal of Economics 141*(3), 1871–1920.

Mirrlees, J. (1974). Notes on Welfare Economics, Information and Uncertainty. *Essays on Economic Behavior Under Uncertainty*, 243–261.

Mirrlees, J. A. (1976). The Optimal Structure of Incentives and Authority Within an Organization. *The Bell Journal of Economics*, 105–131.

Molitor, D. (2018). The evolution of physician practice styles: evidence from cardiologist migration. *American Economic Journal: Economic Policy 10*(1), 326–356.

Murphy, K. R. and J. N. Cleveland (1991). *Performance appraisal: An organizational perspective.* Allyn & Bacon.

Nagle, F. (2019). Open Source Software and Firm Productivity. *Management Science 65*(3), 1191–1215.

Nielsen, M. B. and S. Einarsen (2012). Outcomes of Exposure to Workplace Bullying: A Meta-analytic Review. *Work & Stress 26*(4), 309–332.

O*NET (2025). Management Occupations. https://www.onetonline.org/find/family?f=11. Retrieved August 19, 2025.

Pinquart, M. and R. Kauser (2018). Do the Associations of Parenting Styles with Behavior Problems and Academic Achievement Vary by Culture? Results from a Meta-analysis. *Cultural Diversity & Ethnic Minority Psychology 24*(1), 75.

Rothstein, J. (2010). Teacher Quality in Educational Production: Tracking, Decay, and Student Achievement. *The Quarterly Journal of Economics 125*(1), 175–214.

Rothstein, J. (2017). Measuring the Impacts of Teachers: Comment. *American Economic Review 107*(6), 1656–1684.

Sarker, J., A. K. Turzo, and A. Bosu (2020). A Benchmark Study of the Contemporary Toxicity Detectors on Software Engineering Interactions. In *2020 27th Asia-Pacific Software Engineering Conference (APSEC)*, pp. 218–227. IEEE.

Saygin, P. O., G. Pollard, T. Knight, and M. Rush (2025). Gender Biased Resistance to Harsh Feedback.

Shastry, G. K., O. Shurchkov, and L. L. Xia (2020). Luck or Skill: How Women and Men React to Noisy Feedback. *Journal of Behavioral and Experimental Economics 88*, 101592.

Staiger, D. O. and J. H. Stock (1994). Instrumental Variables Regression with Weak Instruments.

Sutton, R. and B. Wigert (2019). More Harm Than Good: The Truth About Performance Reviews. *GALLUP: Workplace*.

Tepper, B. J., L. Simon, and H. M. Park (2017). Abusive supervision. *Annual Review of Organizational Psychology and Organizational Behavior 4*(1), 123–152.

Vasilescu, B., Y. Yu, H. Wang, P. Devanbu, and V. Filkov (2015). Quality and Productivity Outcomes Relating to Continuous Integration in GitHub. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, pp. 805–816.

Wagner, S. and M. Ruhe (2018). A Systematic Review of Productivity Factors in Software Development. *arXiv Preprint arXiv:1801.06475*.

Weidmann, B., J. Vecci, F. Said, S. Bhalotra, A. Adhvaryu, A. Nyshadham, J. Tamayo, and D. Deming (2026). How do you identify a good manager? *The Quarterly Journal of Economics*, qjag004.

Appendix

Managing by Feedback

Jinci Liu

October 25, 2026

A Setting, Data, and Variable Definitions

A.1 GitHub as a Workplace

GitHub is the largest online coding platform, where developers store code, share projects, and collaborate. As of July 2026, GitHub hosts over 331 million public repositories and more than 217 million users. Hoffmann et al. (2024) estimate that the open-source software ecosystem these repositories support is worth roughly $8.8 trillion in demand-side value to the firms that use it. GitHub records the full version history of each file and supports asynchronous collaboration across time zones and teams.

To join GitHub, each user must create a unique username and can publicly display personal information on a profile, which usually lists name, profile image, location, firm, bio, and email. Many developers highlight programming skills to attract potential employers, often providing accurate details and links to LinkedIn profiles or personal websites to signal credibility and professional identity (El-Komboz and Goldbeck, 2024). These profiles are the source of the demographic measures described below.

Studying workplaces on GitHub is challenging because millions of volunteers contribute and many activities are sporadic. To focus on professional settings, I include only teams owned by organizations, where developers work in teams with defined roles, and I exclude individual projects outside organizational teams so that the sample reflects structured workplace collaboration rather than ad hoc or personal coding.

A.2 Defining a Team

Economists have defined the concept of a "team" in various ways. Marschak and Radner (1958) defines a team as "an organization the members of which have only common interests," highlighting the alignment of incentives. Holmström (1982) defines a team "rather loosely as a group of individuals who are organized so that their productive inputs are related," emphasizing joint production. In more recent work, such as Jones (2021), teams are observed through patterns of co-authorship in science. Following this last definition, I treat each organization-owned GitHub repository as a team: a group of developers jointly contributing code toward a shared project, much like a group of researchers co-authoring a paper. I restrict the analysis to organization-

affiliated teams for three reasons. First, they use structured workflows. Second, they align with firm objectives such as product development or commercialization (Andersen-Gott et al., 2012). Third, they better represent workplace settings than individual projects, which often reflect personal work.

A.3 Identifying Team Members

GitHub's *author association* field classifies each user, when they act, as *owner*, *member*, *collaborator*, *contributor*, *none*, or *mannequin*. I define a team member as a user labeled *owner*, *member*, or *collaborator*. *Owner* and *member* are formally affiliated with the organization; *collaborator* is an invited contributor with similar technical access. Users labeled *contributor* or *none* are external or occasional participants without sustained permissions, and *mannequin* is a placeholder account that I exclude. Because *author association* is observed only when a user acts, months with no activity are missing, making it difficult to track continuous membership directly. To address this, I impute continuous membership between the first and last months in which a user is observed as a team member.

A.4 Predicting Gender

To predict the gender of developers based on their profile page avatars, I select a model whose training images closely match GitHub avatars in resolution and cropping. The chosen model trains vision transformers on male and female portraits and produces, for each image, predicted probabilities for both genders. To determine the optimal cutoff for gender classification, I use a sample labeled by human annotators and evaluate accuracy across different thresholds. The highest accuracy is achieved with a cutoff of 0.85. For each image, I first identify the gender with the higher predicted probability and then assign that gender only if this maximum probability exceeds 0.85; otherwise, the observation is coded as missing. Classification accuracy is computed on the labeled subset where a gender is assigned, excluding cases left unclassified.

The resulting variable, *Predicted Gender*, has three categories: male, female, and missing. In the regressions, I enter these categories as separate indicators, with one category omitted as the reference group. Thus, unclassified developers form a distinct category and are not treated as lying between male and female.

A.5 Predicting Race

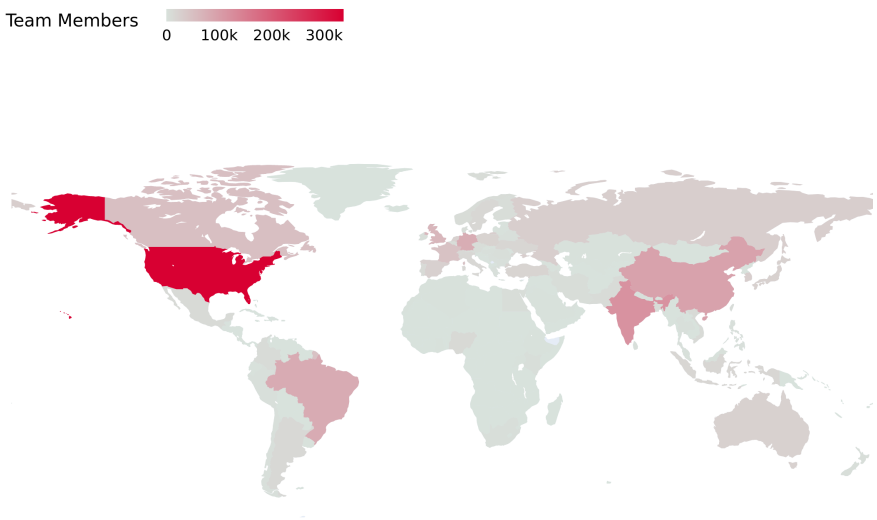
I use Ethnicolr, developed by Laohaprapanon et al. (2017), to predict developers' race and ethnicity from their names. The algorithm uses first and last names to classify individuals into three racial categories: Asian, Black, and White. It is trained on data from the US Census, Florida voter registration, and Wikipedia. For each developer, I take the race with the highest predicted probability as the assigned category. When a name is unavailable, the observation is coded as "missing."

This yields four mutually exclusive outcomes: Asian, Black, White, and Missing. For the heterogeneous analysis, I construct separate indicators for each racial category. For example, *Asian* equals one when the predicted category is Asian and zero otherwise. I also include a separate indicator for developers whose race is missing. I construct the *White* indicator analogously.

A.6 Predicting Location

I built the country variable in three steps. Between October 2023 and March 2024, I scraped each developer's free-text "location" field from their GitHub profile. I then standardized entries—e.g., "Québec" → "Canada," "北京" → "China," "São Paulo" → "Brazil", "München" → "Germany". Finally, I assigned each user to exactly one country or flagged the entry as missing if it failed to match any country. The resulting variable records each developer's self-reported nation of residence.

Figure A1: Geographic Distribution of Team Members



Notes: This world map shows the number of team members by country, based on their reported location. Team member is defined in Section 2.1.2. Darker red shades indicate higher team member counts. The United States has the largest concentration, followed by India, China, and Brazil. Participation is also notable in parts of Europe and Southeast Asia.

A.7 Comparing Reviewers and Developers

Within the automatic assignment sample, I compare reviewers and developers along five dimensions: gender, race, platform reputation, experience, and activity. Figure A2 shows that reviewers are more likely to be male (89% vs. 86%) and less likely to be Asian (16% vs. 21%). They

have greater platform visibility (133 followers vs. 83) and are more active (1,844 yearly activities vs. 668). LinkedIn job titles confirm this pattern: reviewers cluster in senior roles such as *Senior Software Engineer*, *Staff*, and *Principal*, whereas developers concentrate in junior roles such as *Software Engineer* and *Intern*.

Reviewers and developers also differ in their roles within the team. Reviewers are not always senior in title: in 31.97% of cases in the automatic assignment sample (28.04% in the full sample), the assigned reviewer is junior to the developer they review. This is consistent with my functional definition of a reviewer—based on evaluation and approval authority over the case rather than on title—under which a reviewer can hold gatekeeping authority without out-ranking the developer. Reviewing and coding are also not mutually exclusive: 45.91% of reviewers in the automatic assignment sample (41.43% in the full sample) also submit code within the same team-year, indicating that many reviewers act as working managers who both write and evaluate code.

A.8 Code Quantity Measures

I construct two weekly code-quantity measures from the case (pull-request) lifecycle. Let t denote a week, i a developer, and k a case.

New Code Lines

$$new_code_lines_{i,t} = \sum_{\substack{k: \\ \text{open_week}(k)=t}} \text{additions}_{i,k,\text{opened}}$$

where $\text{additions}_{i,k,\text{opened}}$ is the lines developer i first submits for case k in week t. New code lines capture the total lines first submitted by developer i in week t, before any review.

Rewritten Code Lines

$$rewritten_code_lines_{i,t} = \sum_{\substack{k: \\ \text{close_week}(k)=t}} |\text{additions}_{i,k,\text{closed}} - \text{additions}_{i,k,\text{opened}}|$$

measures the revision a case undergoes between submission and closing. Revisions are attributed to the week the case closes, when the revised code enters the codebase. A case opened in one week and closed in a later week therefore contributes new lines to the first week and rewritten lines to the second.

Consider a developer i who makes a single code submission in week 2020-01, adding 120 lines of code to a case. By the time the case is closed in week 2020-02, the cumulative number of added lines has increased to 200. I therefore define

$$new_code_lines_{i,\text{2020-01}} = 120,$$

and

$$rewritten_code_lines_{i,2020\text{-}02} = 200 - 120 = 80.$$

A.9 Code Quality Measures

Code Quality I define code quality only for developer–team–weeks in which the developer submits at least one new line of code. Observations with no new code are excluded, including weeks that contain revisions but no new submission. Rewritten lines associated with a case are assigned to the week in which that case was opened. Unlike the rewritten- lines quantity measure, which is dated at case closing, the quality measure dates both components at case opening so that the ratio refers to the same cases. The code-quality regressions are therefore conditional on positive new-code activity.

$$code_quality_{i,t} \;=\; \frac{new_code_lines_{i,t}}{new_code_lines_{i,t} + rewritten_code_lines_{i,t}}.$$

This metric captures the proportion of code submitted by developer i in week t that required no subsequent revision. Both new and rewritten lines are assigned to the week in which the case was opened. Thus, $code_quality_{i,t}$ is computed only from cases opened in week t. A value of 1 indicates that none of the lines submitted at case opening were subsequently rewritten, while lower values indicate more extensive revisions. Higher $code_quality_i, t$ therefore reflects greater first-pass accuracy and higher code quality.

Case Quality

$$case_quality_{i,t} \;=\; \frac{\sum_{\substack{k:\,\text{author}(k)=i,\\ \text{open_week}(k)=t}} \mathbf{1}\{\text{correct}_k\}}{\sum_{\substack{k:\,\text{author}(k)=i,\\ \text{open_week}(k)=t}} 1},$$

where

$$\mathbf{1}\{\text{correct}_k\} = \begin{cases} 1, & \text{if the initial code submission for case } k \text{ is merged,} \\ 0, & \text{otherwise.} \end{cases}$$

$case_quality_{i,t}$ measures the share of cases opened by developer i in week t that meet review standards without requiring substantial changes.

A.10 Classifying Project Stage

A GitHub *release* is the project's official, stable snapshot. Because each release package includes bug fixes, new features, or architectural changes, the sequence of releases offers a natural way to track a project's progress. Each release includes a *Semantic Versioning* (SemVer) tag in the format `MAJOR.MINOR.PATCH`, such as `v1.2.3`. The three components indicate the scale of change: the patch number increases for bug fixes, the minor number for backward-compatible feature additions, and the major number for breaking or incompatible changes.

Most modern projects adopt the SemVer tag, and the release tag provides a reliable and comparable signal of project stages. To classify project maturity, I extract the `MAJOR` and `MINOR` components from each release's Semantic Versioning tag. I then group releases into three stages. Stage 1 (Early) includes all tags with `MAJOR` = 0, as well as `1.0.0`, reflecting either pre-release development or an initial stable version. Stage 2 (Stable) covers all remaining releases with `MAJOR` = 1, that is, every `1.x.y` tag other than `1.0.0`, representing backward-compatible feature growth after the first stable release. Stage 3 (Mature) includes all releases with `MAJOR` ≥ 2, indicating that the project has undergone at least one major upgrade. [30] Projects that do not follow SemVer, for example, those using date stamps, commit hashes, or missing tags, are grouped into a separate "non-SemVer" category. This approach yields a simple, interpretable stage variable that can be compared across projects.

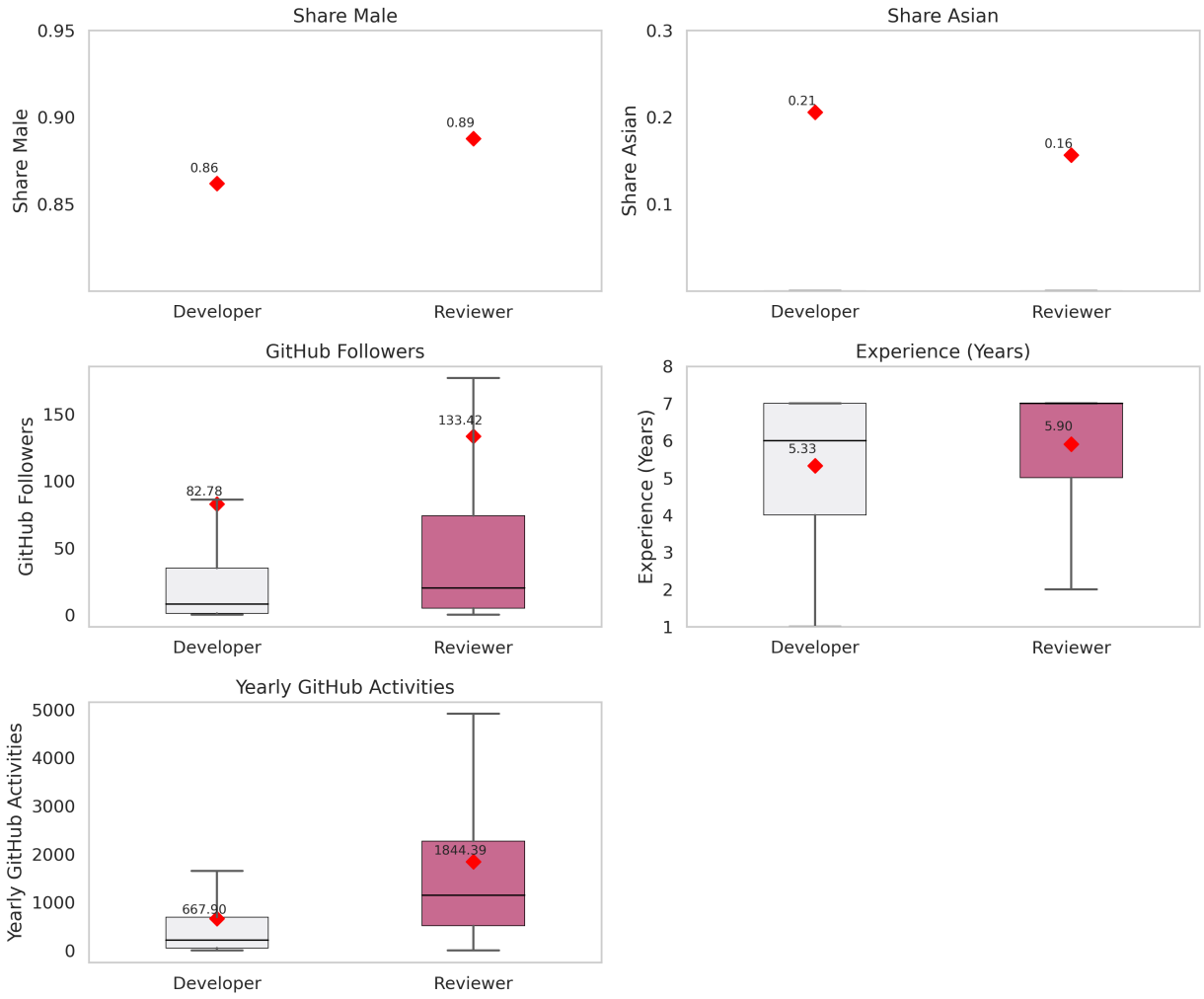
Assigning Cases to Project Stage. Let d_k denote the opening date of case k, and let $\{(s_i, t_i)\}_{i=1}^{N}$ represent the sequence of project releases, where s_i is the stage assigned to release i and t_i is the release date. The project stage at the time of case k is assigned as:

$$\text{stage}_k = s_{i^*} \quad \text{where} \quad i^* = \max\{i : t_i \leq d_k\}.$$

This procedure links each case to the most recently completed release stage available at the time the case was opened. Among cases in the automatic assignment sample, 15.45% fall into Stage 1 (early development), 17.40% into Stage 2 (stable), 42.19% into Stage 3 (mature), and 24.96% are associated with a non-SemVer category.

[30] Incrementing the `PATCH` number (e.g. 1.0.1 $\rightarrow$ 1.0.2) records a bug fix; raising the `MINOR` number (1.0.2 $\rightarrow$ 1.1.0) adds new, backward-compatible features; and advancing the `MAJOR` number (1.1.0 $\rightarrow$ 2.0.0) signals an intentional, incompatible redesign. See https://semver.org/ for more information.

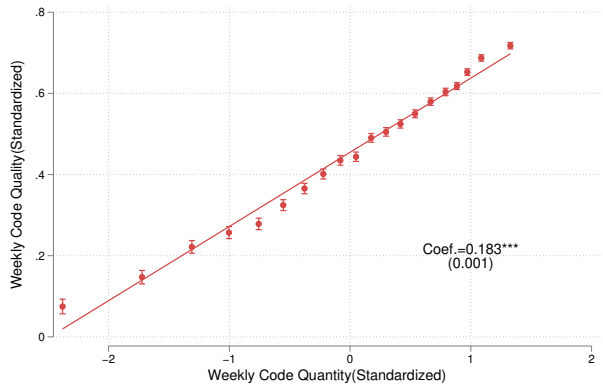
Figure A2: Comparison between Developers and Reviewers



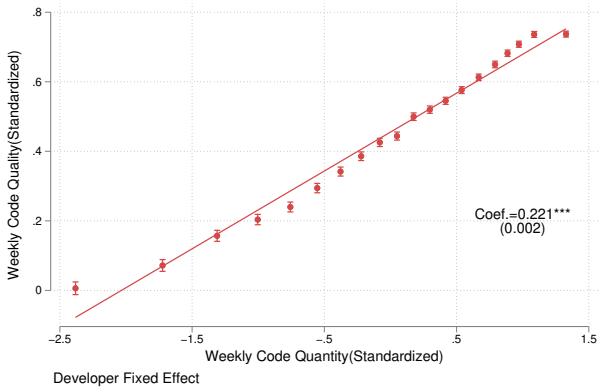
Notes: Each panel compares characteristics between developers and reviewers on GitHub. For this comparison, developers are restricted to members who never serve as reviewers. The sample includes teams that adopt random reviewer assignment. "Share Male" and "Share Asian" report the average share across individuals (red diamonds). "GitHub Followers," "Experience," and "Yearly GitHub Activities" display distributions and mean values (red diamonds) using standard box plots. The black line inside each box represents the median; whiskers extend to 1.5 times the interquartile range. GitHub followers are scraped as of 2023. Experience is calculated as 2024 minus the first year a developer appeared on GitHub. Yearly GitHub activities measure the total number of activities in the most recent calendar year.

Figure A3: Developer Weekly Code Quantity vs. Code Quality

(a) Without Developer Fixed Effect



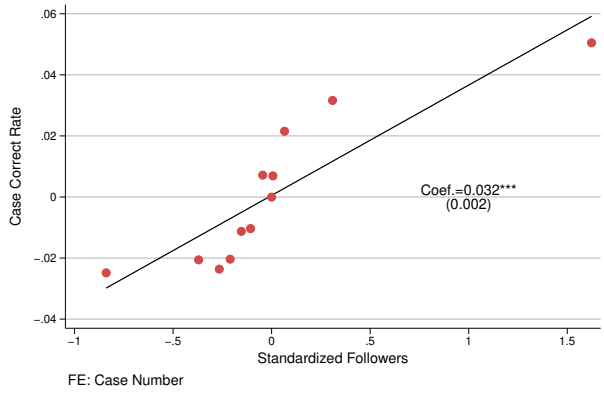
(b) With Developer Fixed Effect



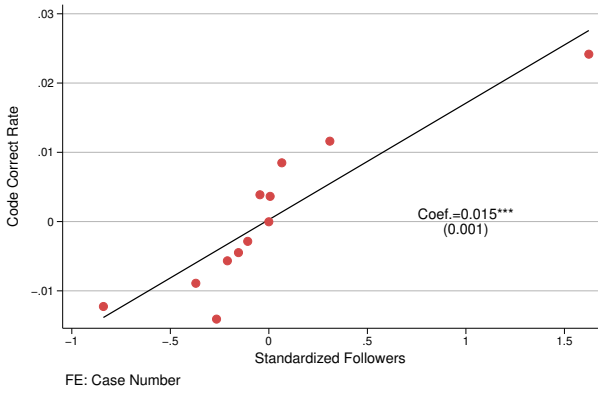
Notes: This figure plots binned relationships between weekly code quantity and weekly code quality. Both variables are standardized. Panel (a) reports the raw relationship without developer fixed effects. Panel (b) reports the within-developer relationship after absorbing developer fixed effects. The fitted line shows the corresponding linear regression coefficient, with standard errors in parentheses. The number of observations is 256,998 in Panel (a) and 253,050 in Panel (b).

Figure A4: Output Quality Validation

(a) Case Quality



(b) Code Quality



Notes: This figure validates output-quality measures. The x-axis displays each developer's standardized follower count (as a proxy for reputation), and the y-axis shows their average Case Quality and Code Quality as defined in Appendix A.9. The binscatter controls for the total number of cases per developer to account for varying activity levels. Developers with more followers exhibit higher quality.

B Reviewer Assignment

I study teams that use pre-existing rules to assign reviewers, limiting developers' discretion over whom to request. These rules take several forms. Lottery workflows draw reviewers from a predefined pool, while other automated procedures allocate reviews according to workload or rotation. Code-ownership mappings route submissions to the individuals or teams responsible for the affected code. Table B1 lists the configuration files used to identify these arrangements and describes their functions.

Table B1: Configuration Files for Reviewer Assignment

Configuration File	Purpose
`CODEOWNERS`	Maps code paths to designated reviewers or teams and automatically requests their review, limiting the developer's discretion over whom to ask.
`.github/auto_assign.yml` or `.github/auto-assign.yml`	Config for the "auto-assign" Action: lists eligible reviewers and rules (e.g. random selection) for automatic assignment on pull request open.
`.github/reviewer_lottery.yml` or `.github/reviewer-lottery.yml`	Defines reviewer pools and lottery rules for custom bots that rotate or randomly pick reviewers per pull request.
`.github/assign_reviewers.yml` or `.github/assign-reviewers.yml`	Generic config used by GitHub Actions to specify reviewer pools and selection logic (e.g. round-robin, weighted).
`.github/workflows/assign_reviewers.yml` or `.github/workflows/assign-reviewers.yml`	A workflow that reads the above config and auto-assigns reviewers on pull request events.
`.github/workflows/auto_assign.yml` or `.github/workflows/auto-assign.yml`	Workflow file for an Action that implements automated reviewer selection (e.g. based on past workload or random choice).

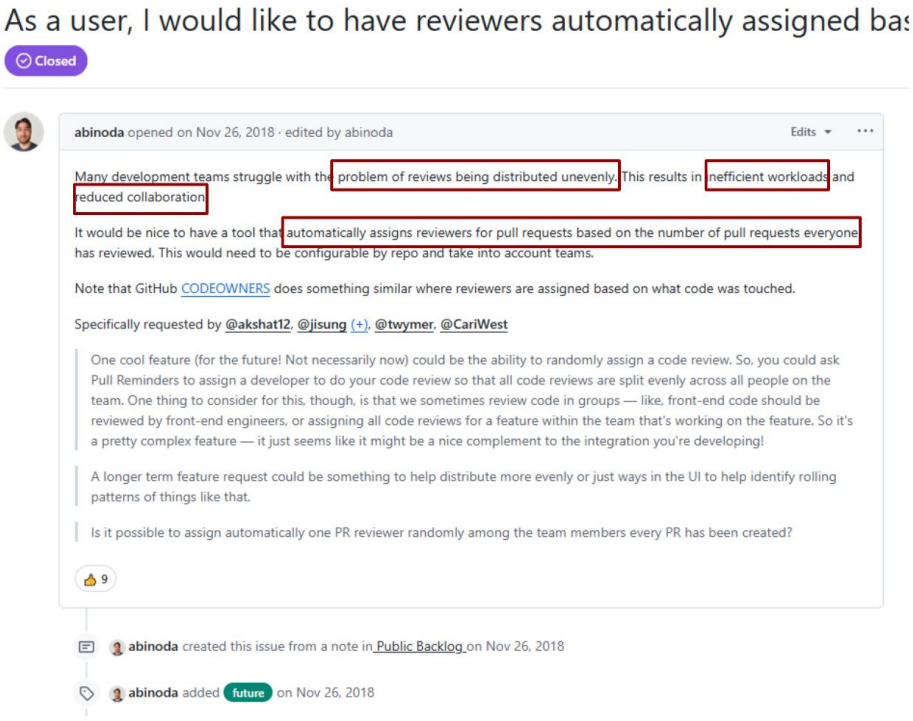
Notes: This table lists configuration files and workflows used to identify automatic reviewer assignment. These files govern reviewer eligibility, route submissions to designated reviewers or teams, or implement automated selection. Additional custom or third-party configurations may exist.

GH Archive does not directly report when these arrangements become active. I therefore use the earliest observed pull request that adds or modifies a relevant configuration file as a proxy for adoption and include cases from that month onward. This procedure dates the first observed configuration change rather than the exact activation of the assignment rule.

Identification requires that, conditional on the controls in the empirical specification, developers assigned to reviewers with different feedback propensities would otherwise have similar productivity. The observed assignment patterns provide supporting evidence. The assigned reviewer's feedback style is uncorrelated with observable developer and case characteristics (Figure 6).

Figure B1 illustrates the motivation for automatic assignment through a discussion of uneven review workloads. Figure B2 contrasts a review request initiated by a bot with one initiated manually by a developer. Under manual selection, developers can choose a reviewer suggested by GitHub or request another team member directly.

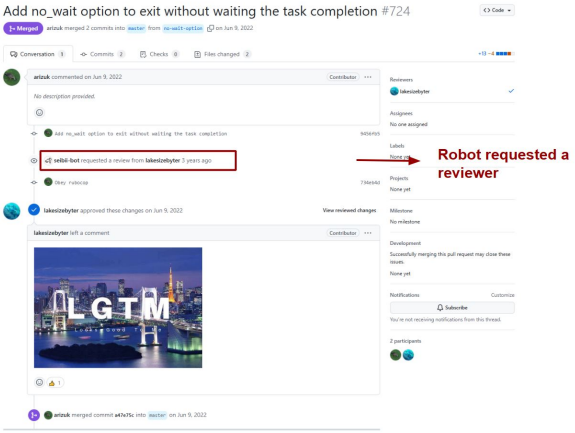
Figure B1: Motivation for Automatic Assignment



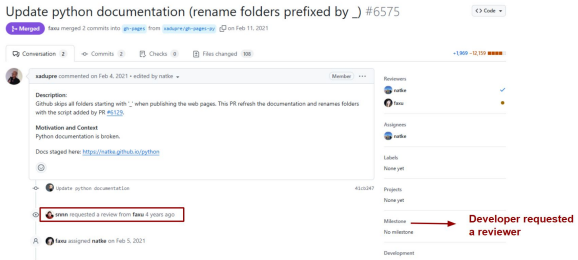
Notes: This figure reproduces a discussion proposing automatic reviewer assignment to distribute review workloads more evenly. The proposal considers each member's recent review workload and allows reviewer pools to vary across teams and repositories. Original post, accessed December 31, 2024.

Figure B2: Automatic and Manual Review Requests

(a) Review requested by a bot



(b) Review requested by a developer



Notes: Panel (a) shows a review request initiated automatically by a GitHub bot. Panel (b) shows a review request initiated manually by a developer. The examples illustrate how review requests are initiated; the selection of individual reviewers depends on the underlying assignment rule.

Table B2 reports the number and share of teams at major technology firms identified as using automatic reviewer assignment.

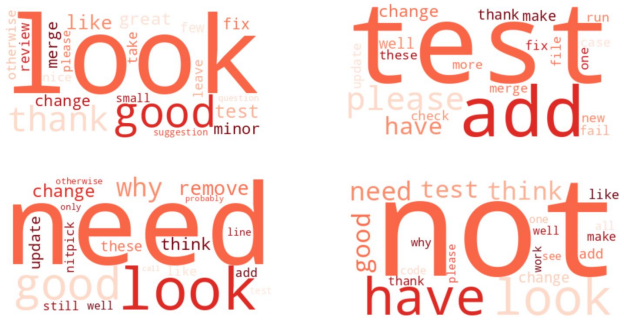
Table B2: Share of Teams in Major Tech Firms adopting Random Reviewer Assignment

Firm	**Share(%)**	**Teams**	**Firm**	**Share (%)**	**Teams**
GitHub	3.226	341	Spotify	0.508	197
Pinterest	3.030	66	IBM	0.337	1780
Intel	1.173	597	Adobe	0.140	714
Apple	1.156	173	Uber	0.000	205
Netflix	1.156	173	PayPal	0.000	189
Salesforce	1.145	262	Yahoo	0.000	185
NVIDIA	1.124	267	Amazon	0.000	146
Microsoft	1.119	4645	Twitter	0.000	137
Stripe	0.917	109	Airbnb	0.000	120
Facebook	0.855	234	Dropbox	0.000	120
Oracle	0.800	250	eBay	0.000	100
OpenAI	0.794	126	Reddit	0.000	60
Google	0.723	1797	Zoom	0.000	43
Lyft	0.699	143	Snapchat	0.000	27
Tencent	0.585	171	Tesla	0.000	5

Notes: This table summarizes the share and total number of teams in major tech firms adopting the random reviewer feature. Across the full sample, 3,457 of 525,841 teams (approximately 0.66%) are identified as adopters. GitHub, Pinterest, and Intel show the highest adoption rates, with GitHub at 3.23% across 341 teams. Microsoft (4,645 teams) and Google (1,797 teams) also have above-average adoption. Firms like Uber, Twitter, and Tesla show no adoption during the sample period.

C Feedback Classification and Descriptive Facts

Figure C1: Word Clouds of Principal Components of Reviewer Feedback



Notes: This figure presents word clouds for the top four principal components extracted from reviewer feedback in the automatic assignment sample. PCA reduces millions of feedback messages into a few components that capture most of the meaningful variation in the data. Each word cloud visualizes one component, where larger words load more heavily on that component. The components consistently capture meaningful dimensions of feedback—such as positive (e.g., *good*, *thank*), negative (e.g., *not*), and constructive (e.g., *test*, *add*) language. A single word may load on multiple components if it conveys more than one meaning.

C.1 Pre-processing

To ensure reliable measurement, I clean and standardize the feedback text before classification. First, I remove quoted blocks from other users' messages to prevent misattribution, as well as code snippets. Second, I replace common software-engineering acronyms, such as *"lgtm" (looks good to me)* and *"kiss" (keep it simple)*, with their full forms. Third, I account for software-specific terminology that general-purpose models often misclassify: words like "kill," "trash," and "dump" refer to legitimate coding operations but are frequently flagged as toxic (Sarker et al., 2020). Addressing these ambiguities yields more accurate classification.

Examples of Feedback Types with Continuous Scores

The following examples report ToxiGen-predicted toxicity scores. Blunt critiques of the work are classified as non-toxic, while feedback that targets the person is flagged as toxic:

1. *This is bad code* → non-toxic, probability of the predicted label: 0.918
2. *Your paper is not of general interest to publish at AER* → non-toxic, probability: 0.999
3. *What's wrong with you to write such bad code?* → toxic, probability: 0.712
4. *Women are just not good at coding* → toxic, probability: 0.988

The next examples report GPT-based constructiveness scores on the 0–10 scale, where a threshold of 4 defines constructive feedback:

1. *Agree* → non-constructive, score: 0
2. *This is not clear* → non-constructive, score: 1
3. *I don't think all introspector errors are reported in the domain status. We need to make sure that all severe jrf ones are* → constructive, score: 4

C.2 Toxicity

Figure C2: ToxiGen Human-Annotation Guidelines



Notes: This is from Figure 6 of Hartvigsen et al. (2022), which shows how human annotators label toxicity for the ToxiGen Classification Model.

C.3 Constructiveness

I use large language models (LLMs) to detect constructive feedback on GitHub. I experimented with small changes in prompt wording, aiming to give the LLM a clear and objective instruction. I also tested including and excluding code snippets in the input. These changes did not alter the

binary constructive versus non-constructive classification on a manually labeled validation set of 1000 feedback messages, although minor variations appeared in token-level probabilities.

My specific LLM prompt is as follows:

System Content:

You are an experienced principal software engineer reviewing a pull request comment. Rate its constructiveness on a scale from 0 (not constructive) to 10 (highly constructive). Focus on whether the comment provides specific or actionable information. A specific comment identifies a bug, suggests an improvement, or points to a concrete issue. If the comment is vague or lacks detail, treat that as a weakness. Focus on the content, not on the tone. Provide them in JSON format with the following key: info_score.

User Content:

Code: `<INPUT>`

Comment: `<INPUT>`

To identify the best-performing LLM for this task, I compare four OpenAI models (GPT-4.1-nano, GPT-4o-mini, GPT-4o, and o3) to find the one whose judgments most closely match those of software industry professionals, while minimizing cost. The initial human labels are produced by a master's student in computer science and verified by a software engineer.

The annotator rates each feedback item on a 0–10 scale: scores of 0–2 indicate little actionable information, 3 indicates borderline relevance, and 4–10 indicate specific or actionable information. I define feedback as constructive when the score is at least 4. This cutoff matches the paper's definition of constructiveness as concrete, actionable information rather than general helpfulness.

Once I obtain the ground truth and model predictions from different LLMs, I evaluate performance using binary rather than absolute scores to reduce measurement error from minor discrepancies in human ratings. To generate binary labels, I test three decision cutoffs (≥ 3, ≥ 4, and ≥ 5). Each cutoff changes the share of feedback classified as constructive and affects model performance. Table C1 reports results for each cutoff.

Table C1: Accuracy and F1 Scores by Model and Cutoff

Cutoff	**GPT-4.1-nano**		**GPT-4o-mini**		**GPT-4o**		**o3**	
	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1
≥ 3	0.87	0.817	0.76	0.760	0.82	0.804	0.80	0.783
≥ 4	0.84	0.579	0.90	0.814	0.88	0.767	0.90	0.828
≥ 5	0.84	0.200	0.96	0.875	0.87	0.723	0.93	0.829

I select the GPT-4o-mini model because it offers the best balance between precision and recall, aligns closely with the human annotator's definition of clear and actionable feedback, and delivers performance comparable to higher-end models at a fraction of the cost. For example, it achieves an F1 score of 0.814 at a cost of about $0.93 to classify 10,000 feedback messages, compared with 0.828 and $23.25 for o3—a saving of more than 95%. I use a decision cutoff of 4 because it maintains a reasonable class balance (about 25% positives) and reflects the annotator's standard for clear and

actionable feedback. I also report results for cutoff = 3 as a robustness check. The empirical results are not sensitive to this choice: Figure D5 re-estimates the main specification with constructiveness defined at the ≥ 3 cutoff, and the estimates are similar. Table C2 presents examples of constructive feedback from the real data.

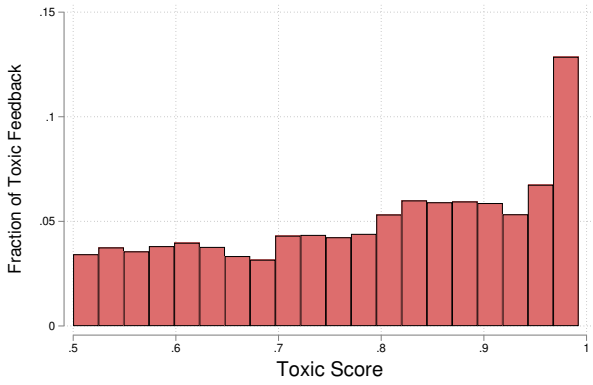
Table C2: Examples of Reviewer Feedback and Constructiveness Scores

Feedback Message	**Constructive**	**Score**
Agree	0	0
Just checked one more time - now it's okay	0	1
Same advice on the assets point of view	0	2
Explain why there are private fields in this visitor. too multiple of them	0	3
I don't think all introspector errors are reported in the domain status. We need to make sure that all severe jrf ones are	1	4
You only need to define the acronym once, so I'd suggest doing it in the text not the title	1	5
This should be an async function that can use await and, as a result, standard trycatch blocks	1	6
We need to be generating a new nonce used to encrypt here rather than reusing the one that was used previously to encrypt the keyring. Otherwise there's a crypto bug in this that would allow an attacker to combine the old ciphertext with the new ciphertext in a way that could reveal the xor'd plaintext of the two messages.	1	10

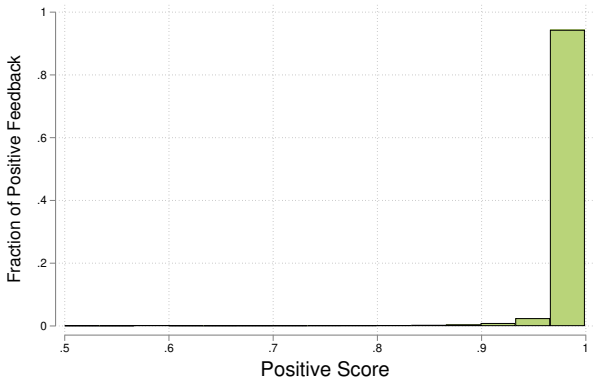
Notes: This table reports examples of reviewer feedback messages scored by GPT-4o-mini. Scores range from 0 to 10, with higher scores indicating more specific and actionable information about the submitted code. *Constructive* equals one if the score is at least 4.

Figure C3: Distribution of Predicted Feedback Probability

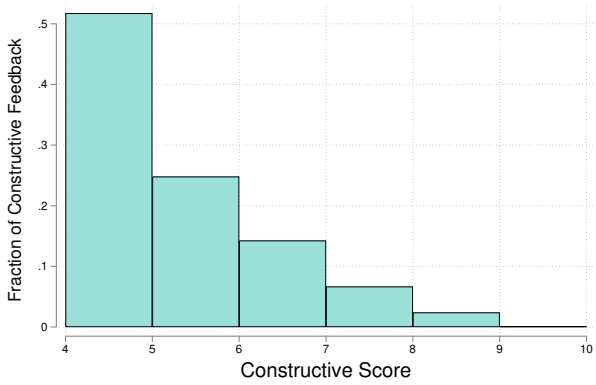
(a) Conditional on Toxic Feedback



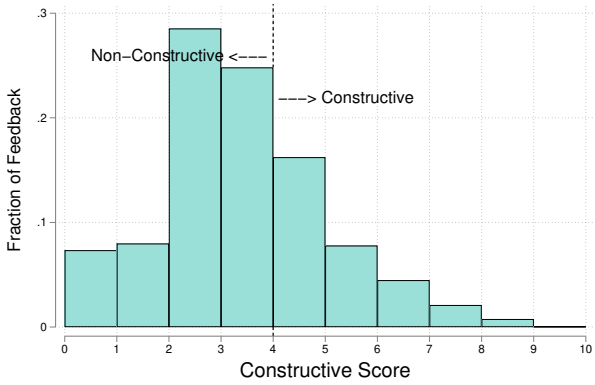
(b) Conditional on Positive Feedback



(c) Conditional on Constructive Feedback



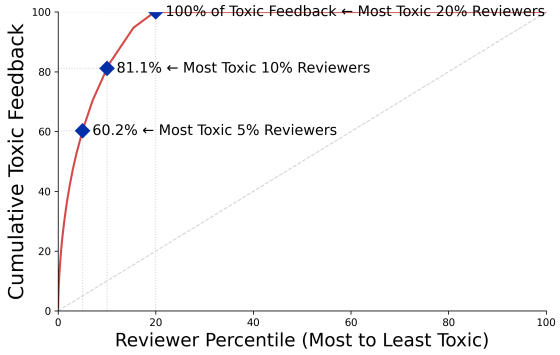
(d) Constructive Feedback Cutoff



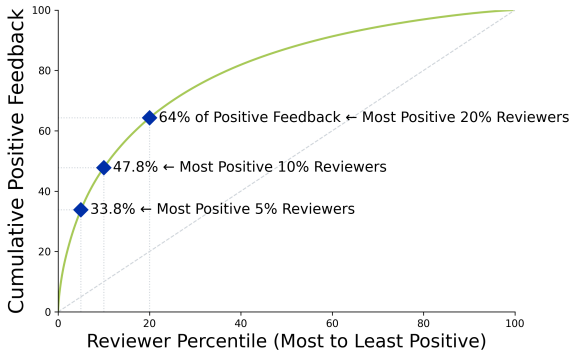
Notes: These figures illustrate the distribution of predicted probability scores for each feedback dimension, based on all feedback messages written by code reviewers in the automatic assignment sample. Figures C3a and C3b report score distributions for toxic and positive feedback, respectively, based on binary labels assigned by a pre-trained classifier. Figure C3c shows the distribution of predicted constructive scores generated by GPT-4o-mini. Figure C3d illustrates the classification of feedback as constructive or non-constructive using a cutoff on the GPT-based score: messages above the threshold are labeled constructive, while those below are labeled non-constructive. As a robustness check, I replicate the analysis with an alternative cutoff of 3 (rather than 4); the results remain qualitatively unchanged (see Figure D5).

Figure C4: Reviewer Heterogeneity in Feedback Styles

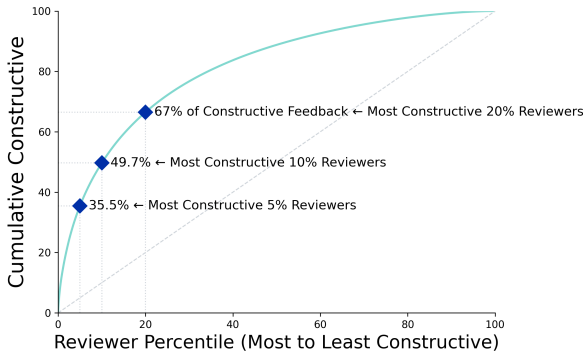
(a) Toxic Feedback



(b) Positive Feedback



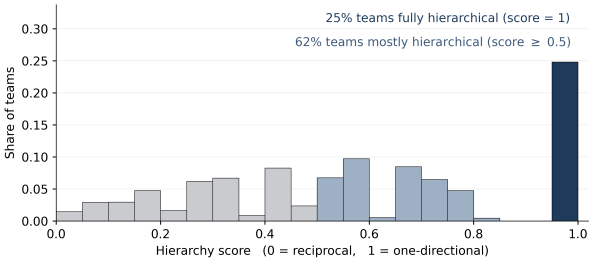
(c) Constructive Feedback



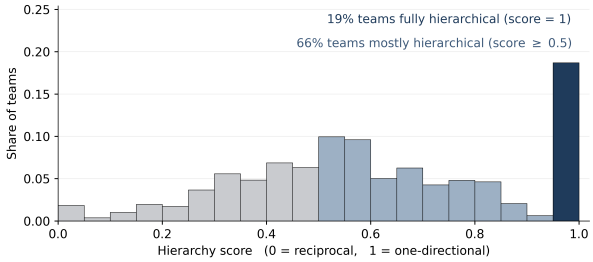
Notes: Each panel plots the cumulative share of feedback messages of a given type accounted for by reviewers, ranked from most to least likely to provide that type of feedback. Panel (a) shows toxic feedback, Panel (b) positive feedback, and Panel (c) constructive feedback. The diagonal line represents an equal distribution across reviewers. A steeper curve indicates greater concentration, meaning that a small share of reviewers accounts for a large share of messages of that type. The sample is restricted to the automatic assignment sample.

Figure C5: Team Feedback Structure

(a) Teams with Size 5



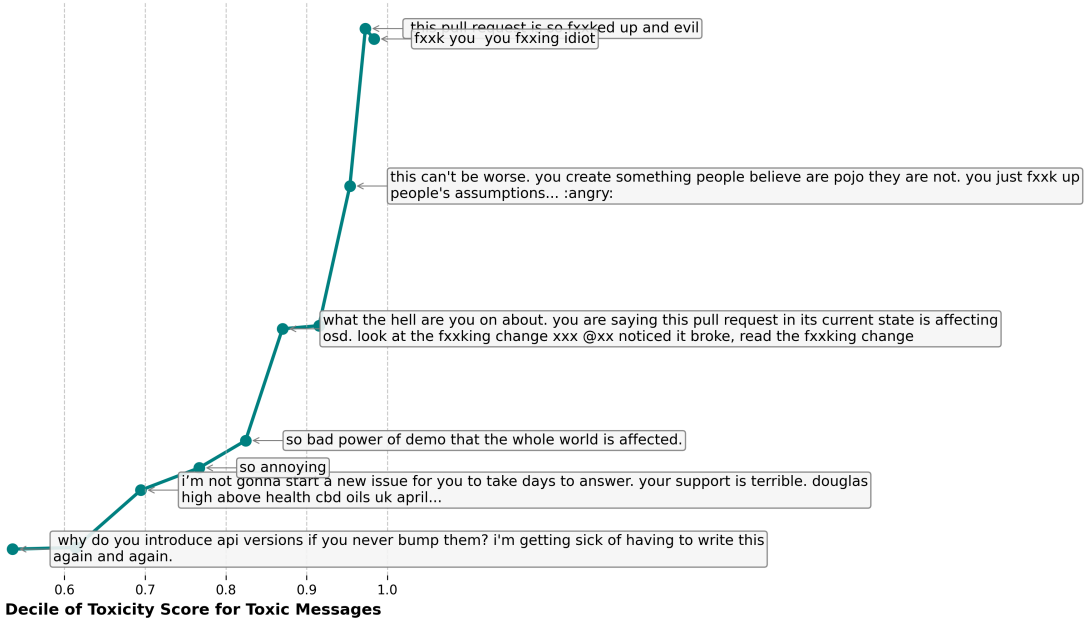
(b) automatic assignment sample



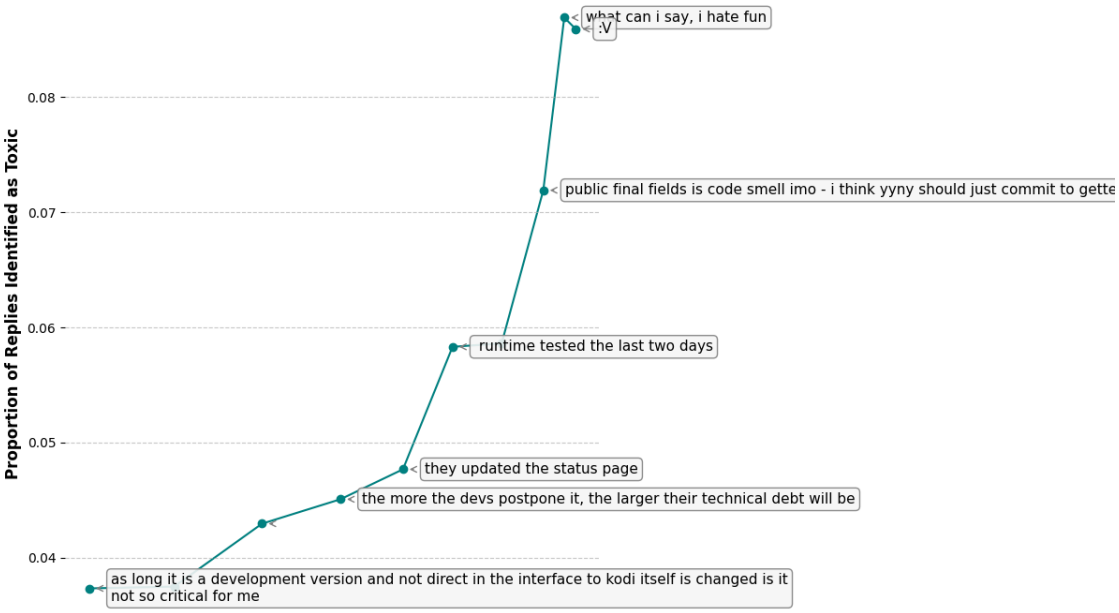
Notes: This figure plots the distribution of team hierarchy scores over 2017–2023. The unit of observation is the team–year. For each team, the hierarchy score is $1 - 2R/E$, where E is the number of directed reviewer–developer pairs and R the number of reciprocated reviewer–developer pairs; it ranges from 0, when members review each other symmetrically, to 1, when review flows in one direction. Higher scores indicate that review is concentrated among a small subset of members; lower scores reflect a more evenly distributed structure. Teams with a single case or a single member are excluded, since one-directional review is mechanical when only one case is reviewed. **Panel (a)** restricts to teams with exactly five members. **Panel (b)** uses the random-reviewer sample, which includes teams that use random reviewer assignment, with at least two reviewers per year and at least ten cases reviewed per reviewer per year. Figure 4 plots the full sample.

Figure C6: Toxic Feedback and Replies

(a) Toxic Feedback by Classification Probability Decile



(b) Corresponding Replies



Notes: This figure displays randomly selected examples of toxic feedback and the corresponding replies from the full sample. These interactions often occur when reviewers provide feedback to junior developers. Recipients of toxic feedback frequently respond by explaining or defending their actions, and their replies also tend to contain more toxic language.

D Identification and Robustness

D.1 Alternative IV Estimators

The main instrument is a residualized leave-developer-out reviewer feedback tendency, based on a reviewer's feedback messages in cases involving other developers. Following standard practice (Dobbie and Song, 2015; Dobbie et al., 2017; Bhuller et al., 2020; Agan et al., 2023), I report F-statistics without correcting for the fact that the instrument is estimated.

As Agan et al. (2023) note, including reviewer fixed effects directly may lead to bias due to the large number of potentially weak instruments. In addition, my specification includes many fixed effects (e.g., team-by-year), which are necessary for identifying cases with as-if random reviewer assignment. Many fixed effects can, however, bias jackknife IV estimators(Kolesár, 2013).

To assess robustness, I explore several alternative approaches commonly used when IV estimates may be biased due to many (potentially weak) instruments. Table D1 reports these results. Column (1) repeats the main 2SLS estimates using the residualized leave-out mean leniency score. Column (2) reports estimates from a limited information maximum likelihood (LIML) model using all reviewer dummies as instruments (Bekker, 1994; Angrist and Frandsen, 2022). Column (3) presents results from the modified bias-corrected two-stage least squares (MBTSLS) estimator of Kolesár et al. (2015), and column (4) shows estimates from the unbiased jackknife IV estimator (UJIVE) of Kolesár (2013). In each case, the estimated coefficients are nearly identical to the main 2SLS results.

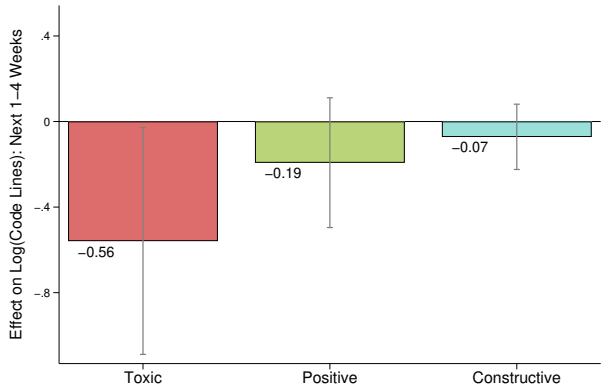
Table D1: Alternative IV Estimators

	(1) Main	(2) LIML	(3) MBTSLS	(4) UJIVE
Panel A: Toxic Feedback				
Code Quantity Ever	-0.074*	-0.074*	-0.074*	-0.077**
	(0.039)	(0.039)	(0.039)	(0.039)
Code Quantity Log	-0.558*	-0.558*	-0.558*	-0.569*
	(0.328)	(0.328)	(0.328)	(0.328)
Non-Code Quantity Ever	-0.019	-0.019	-0.019	-0.020
	(0.019)	(0.019)	(0.019)	(0.019)
Non-Code Quantity Log	-0.138	-0.138	-0.138	-0.143
	(0.145)	(0.145)	(0.145)	(0.145)
Case Quality	-0.050	-0.050	-0.050	-0.049
	(0.044)	(0.044)	(0.044)	(0.044)
Code Quality	-0.091***	-0.091***	-0.091***	-0.091***
	(0.033)	(0.033)	(0.033)	(0.033)
Panel B: Positive Feedback				
Code Quantity Ever	-0.033**	-0.033**	-0.033**	-0.033**
	(0.017)	(0.017)	(0.017)	(0.017)
Code Quantity Log	-0.192	-0.192	-0.192	-0.190
	(0.136)	(0.136)	(0.136)	(0.135)
Code Quality	0.044***	0.044***	0.044***	0.044***
	(0.013)	(0.013)	(0.013)	(0.013)
Case Quality	0.022	0.022	0.022	0.022
	(0.018)	(0.018)	(0.018)	(0.018)
Non-Code Quantity Ever	-0.011	-0.011	-0.011	-0.011
	(0.008)	(0.008)	(0.008)	(0.008)
Non-Code Quantity Log	-0.130**	-0.130**	-0.130**	-0.130**
	(0.061)	(0.061)	(0.061)	(0.061)
Panel C: Constructive Feedback				
Code Quantity Ever	-0.046***	-0.046***	-0.046***	-0.046***
	(0.010)	(0.010)	(0.010)	(0.010)
Code Quantity Log	-0.072	-0.072	-0.072	-0.071
	(0.080)	(0.080)	(0.080)	(0.080)
Code Quality	-0.072***	-0.072***	-0.072***	-0.072***
	(0.008)	(0.008)	(0.008)	(0.008)
Case Quality	-0.100***	-0.100***	-0.100***	-0.099***
	(0.010)	(0.010)	(0.010)	(0.010)
Non-Code Quantity Ever	-0.009*	-0.009*	-0.009*	-0.009*
	(0.005)	(0.005)	(0.005)	(0.005)
Non-Code Quantity Log	-0.023	-0.023	-0.023	-0.023
	(0.035)	(0.035)	(0.035)	(0.035)
Team × Year FE	✓	✓	✓	✓
Decile t–1 activity	✓	✓	✓	✓
Decile t–1 message	✓	✓	✓	✓

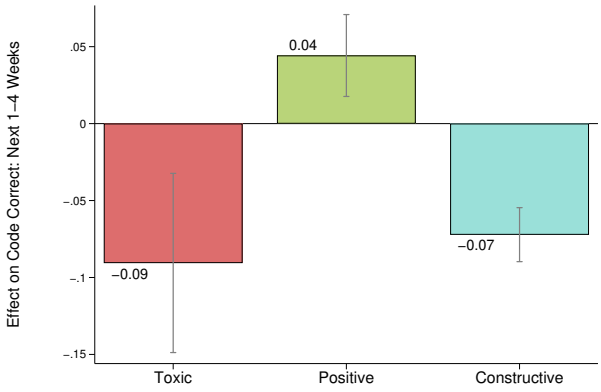
Notes: This table reports 2SLS estimates using alternative estimators, as indicated in the column headers. All specifications include team-by-year fixed effects and developer controls. OLS and reduced-form estimates for the same specification appear in Table 5. Standard errors are clustered at the reviewer level. Column (1) presents the main 2SLS estimates using the residualized leave-out mean leniency score. Column (2) applies limited information maximum likelihood (LIML) with all reviewer dummies as instruments. Column (3) reports modified bias-corrected 2SLS estimates following Kolesár et al. (2015). Column (4) reports UJIVE estimates following Kolesár (2013). *$p < .10$, ** $p < .05$, *** $p < .01$.

Figure D1: Robustness: Clustering at the Team Level

(a) Code Quantity



(b) Code Quality



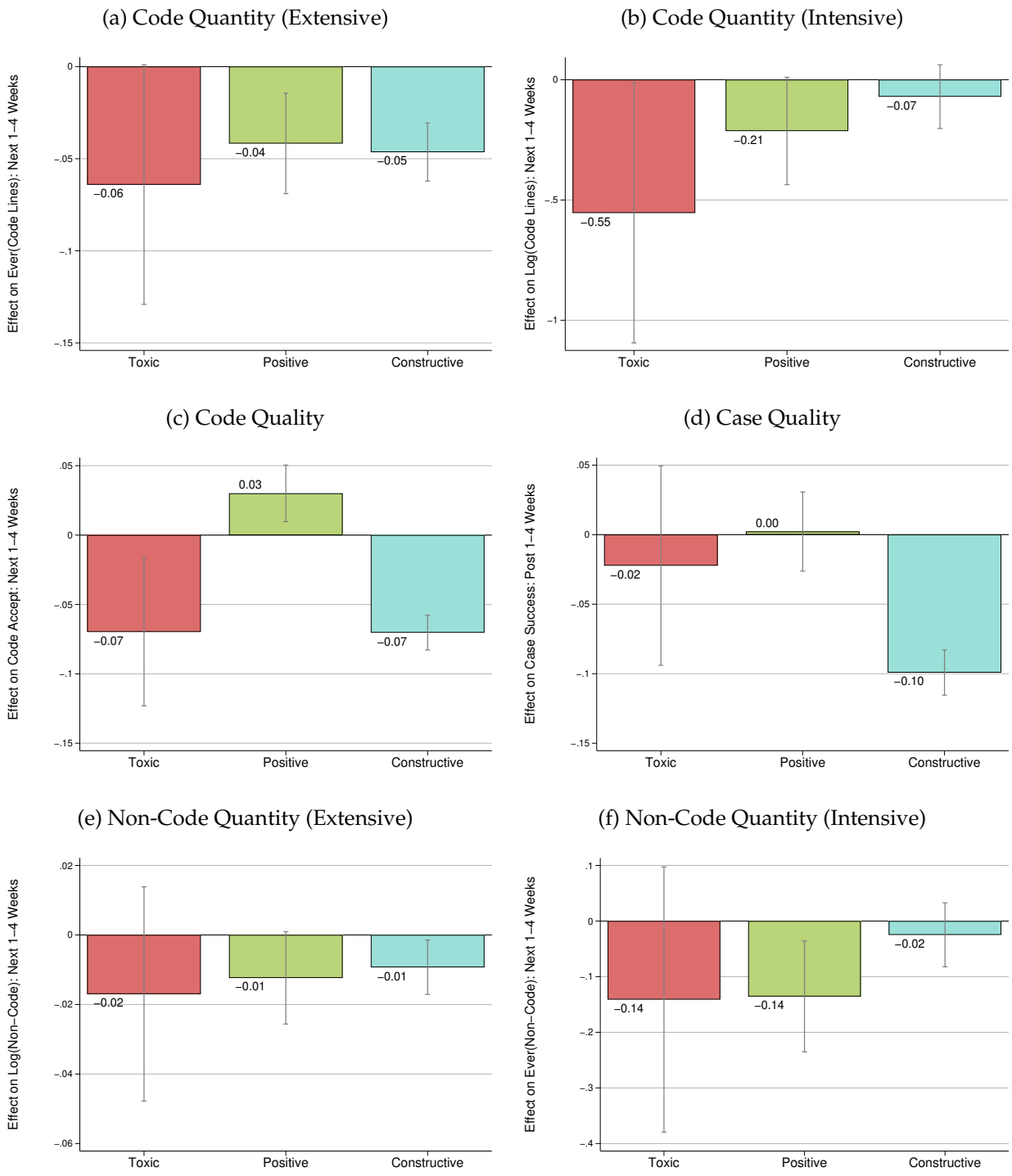
Notes: Panels (a) and (b) show estimates of the effect of feedback on code quantity and code quality for developers in the next 1-4 weeks. Standard errors are clustered at the team level. Lines report 90% confidence intervals.

Table D2: Subgroup First Stages (Monotonicity Diagnostic)

	Split I (Gender)		Split II (Asian)		Split III (Activity)		Split IV (Experience)		Split V (Followers)	
	Male	Others	Asian	Others	Less	More	Less	More	Less	More
Panel A. Toxic										
Reviewer Toxicity	0.701***	0.708***	0.727***	0.714***	0.706***	0.697***	0.702***	0.712***	0.701***	0.725***
	(0.015)	(0.046)	(0.046)	(0.018)	(0.014)	(0.028)	(0.016)	(0.023)	(0.015)	(0.034)
F-Stat.	2,087.50	239.38	251.04	1,623.17	2,721.44	632.20	1,866.46	970.15	2,257.34	451.62
Panel B. Positive										
Reviewer Positivity	0.816***	0.789***	0.832***	0.816***	0.813***	0.846***	0.807***	0.845***	0.824***	0.811***
	(0.006)	(0.017)	(0.015)	(0.007)	(0.005)	(0.009)	(0.006)	(0.006)	(0.005)	(0.013)
F-Stat.	18,476.72	2,225.65	2,964.45	13,847.83	26,252.07	8,830.59	16,173.51	17,913.26	28,733.81	4,078.64
Panel C. Constructive										
Reviewer Constructiveness	0.860***	0.866***	0.885***	0.858***	0.867***	0.859***	0.852***	0.885***	0.867***	0.849***
	(0.004)	(0.011)	(0.010)	(0.005)	(0.004)	(0.007)	(0.005)	(0.005)	(0.004)	(0.010)
F-Stat.	38,760.69	5,984.54	7,343.11	28,196.83	47,478.40	16,968.30	35,778.10	28,685.66	55,841.15	7,488.37
N	789,792	89,921	109,912	557,633	829,262	350,017	719,968	459,309	983,909	195,407

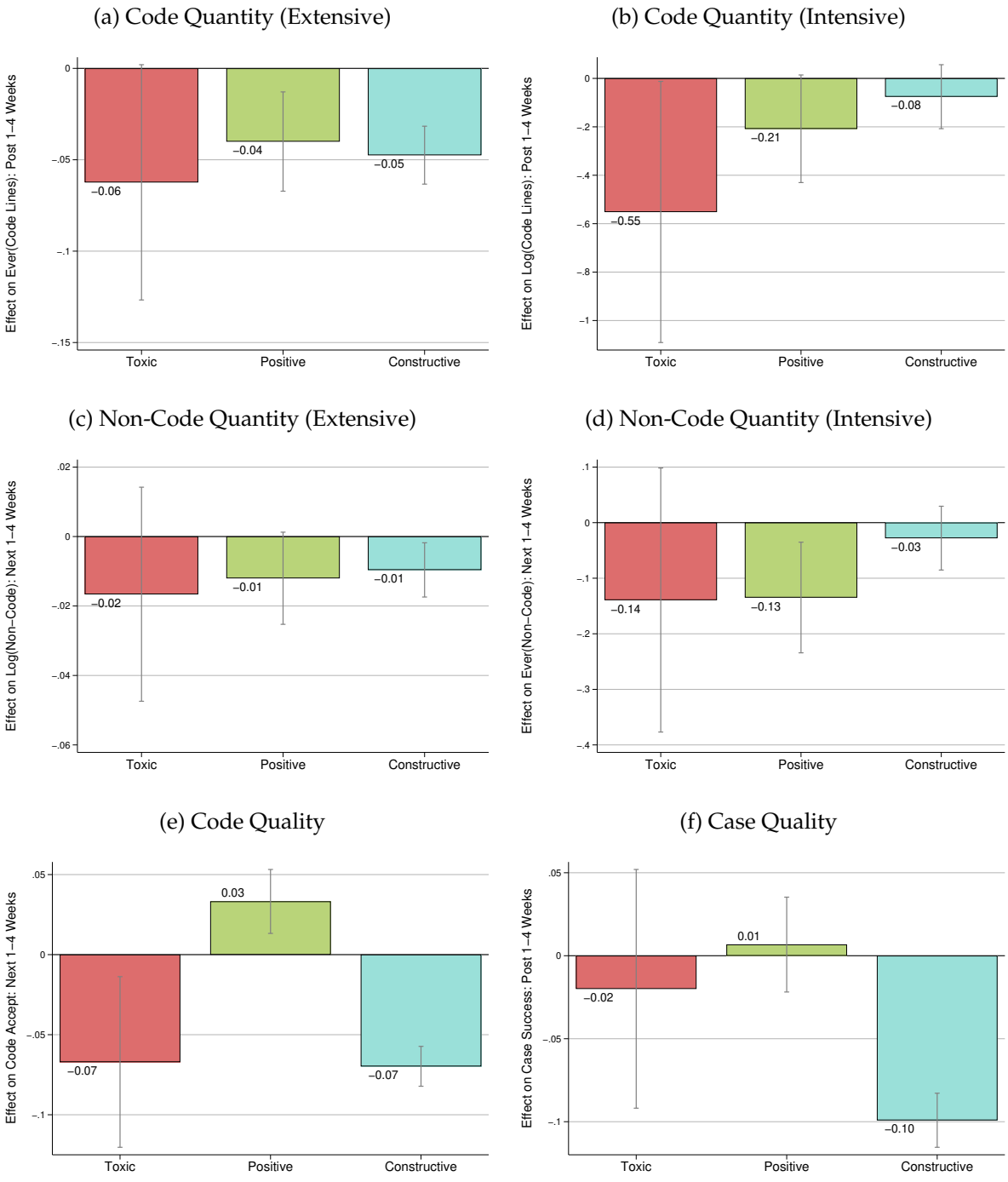
Notes: This table reports the first-stage coefficients and F-statistics from regressions of an indicator for receiving each feedback type on the corresponding reviewer instrument across different sample splits. Columns 1–2 separate the sample by gender, Columns 3–4 by race (Asian vs. others), Columns 5–6 by pre-week activity, Columns 7–8 by experience (based on first active year on GitHub), and Columns 9–10 by number of followers. Standard errors are clustered at the reviewer level. Significance levels: * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$.

Figure D2: Jointly Instrumenting All Three Feedback Dimensions



Notes: The sample is the automatic assignment sample from 2017 to 2023. Controls include all variables listed in Table 5. The results for each type of feedback are from the augmented IV models given by equations (3)–(6). Standard errors are clustered at the reviewer level. Lines report 90% confidence intervals.

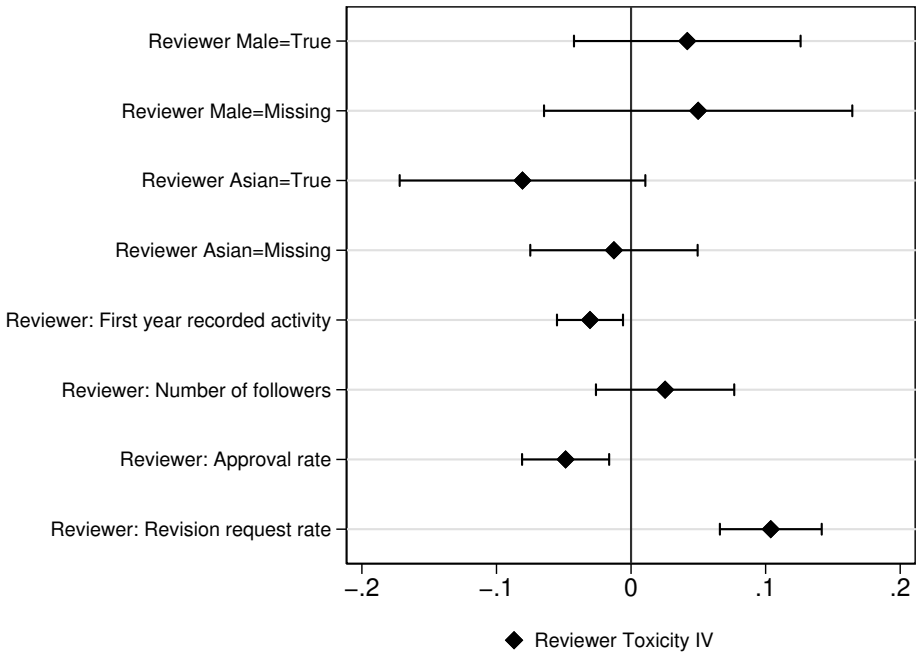
Figure D3: Controlling for Reviewer Feedback Styles in Other Feedback Dimensions



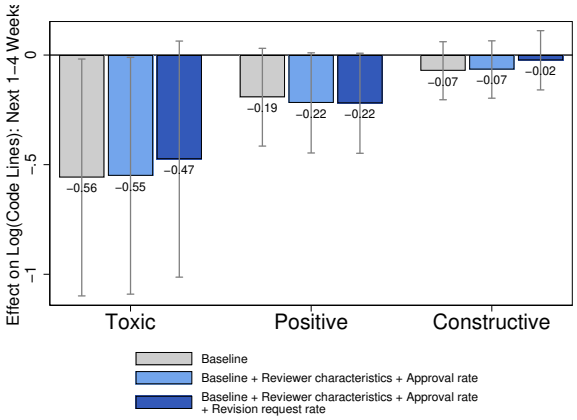
Notes: Baseline sample of cases for automatic assignment sample from 2017 to 2023. Controls include all variables listed in Table 5. Each panel reports augmented IV estimates that add reviewer-style instruments for the non-focal feedback dimensions as controls. The toxicity results control for the reviewer positivity and constructiveness instruments; the positivity results control for the reviewer toxicity and constructiveness instruments; and the constructiveness results control for the reviewer toxicity and positivity instruments. Standard errors are clustered at the reviewer level. Lines report 90% confidence intervals.

Figure D4: Controlling for Reviewer Characteristics and Strictness

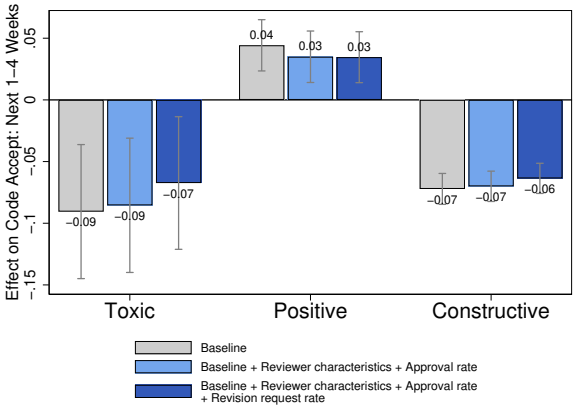
(a) Reviewer Characteristics, Strictness, and Toxicity



(b) Code Quantity

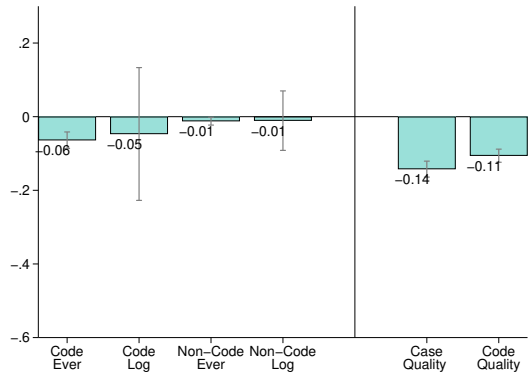


(c) Code Quality



Notes: Panel (a) reports OLS coefficients from a regression of the standardized reviewer toxicity instrument on standardized reviewer characteristics and strictness measures, with team-by-year fixed effects. Characteristics are gender, race, the number of followers (in 2023), and the first year of GitHub activity (seniority). Strictness measures are the reviewer's leave-developer-out approval rate (the share of other cases the reviewer approves) and revision request rate (the share of the reviewer's other cases on which the reviewer requested changes). Standard errors are clustered at the reviewer level. Panels (b) and (c) report 2SLS estimates for code quantity and code quality from three specifications: the baseline of Table 5; the baseline with reviewer characteristics (first year of GitHub activity, gender, and race) and the reviewer's leave-developer-out approval rate added to both stages; and the latter with the reviewer's leave-developer-out revision request rate added. Lines report 90% confidence intervals.

Figure D5: Effect of Constructive Feedback on Productivity: Alternative Constructiveness Cutoff



Notes: This figure reports the effect of constructive feedback on developer productivity using an alternative definition of constructive feedback. The main analysis defines feedback as constructive when the LLM-predicted constructiveness score is at least 4. This figure lowers the cutoff to at least 3 and reestimates the leave-developer-out reviewer constructiveness instrument. Lines report 90% confidence intervals.

E Reviewer Value-Added and Feedback Prediction

E.1 Reviewer Value-Added: Measurement and Validation

I measure reviewer quality by reviewer value-added (VA) to developer productivity. The central challenge is sorting. If more productive developers are systematically matched with particular reviewers, estimated VA may reflect developer ability rather than reviewer quality. This is the same problem faced in the teacher value-added literature, where students of different baseline ability may be assigned to different teachers (Rothstein, 2010, 2017).

The ideal design would randomly assign many developers to each reviewer and measure whether developers systematically produce more after working with that reviewer, relative to their own baseline. I approximate this ideal in two steps. I first estimate reviewer VA in the automatic assignment sample, where reviewer assignment is as good as random. I then extend the estimator to the full sample and validate it using reviewers who appear in both random and non-random assignment settings.

I adopt the forecast-based estimator of Chetty, Friedman, and Rockoff (2014). The estimator residualizes developer output on lagged output, developer characteristics, team characteristics, and case characteristics. It then forecasts a reviewer's contribution in a period using the reviewer's residualized performance in other periods. This jackknife procedure reduces mechanical correlation with the outcome being validated and shrinks noisy estimates toward the mean. Section E.2 gives the formal estimator.

Developer weekly code output is the total number of code lines submitted to a team in a week. In the VA regressions, I control for case characteristics, team- and firm-level output, and developer demographics. Since reviewers are assigned in week t, but some code produced in week t may predate the review, I compare output in $t+1$ to output in $t-1$, using week t only for assignment. I winsorize output at the 95th percentile, standardize log code lines, and include an indicator for missing output.

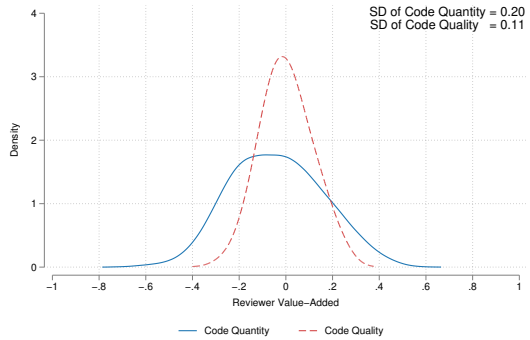
Figure E1 summarizes the VA estimates and their validation. Panel (a) plots the distribution of reviewer VA in the automatic assignment sample. A one standard deviation increase in reviewer VA is associated with 0.20 standard deviations higher developer code quantity and 0.10 standard deviations higher developer code quality. Panel (b) shows that VA based on code quantity and VA based on code quality are positively correlated. Panel (c) compares reviewers who appear in both the random and non-random samples; their VA rankings are strongly correlated, supporting the use of the estimator in the full sample.

E.2 The Forecast-Based Value-Added Estimator

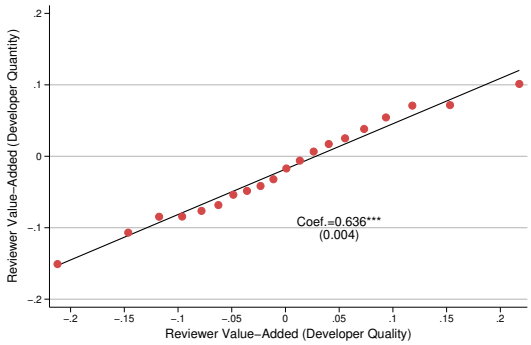
I estimate reviewer value-added (VA) for the full sample using the forecast-based estimator proposed by Chetty et al. (2014). Equation (7) models developer code output Y_{isjt} after receiving feedback from reviewer j, controlling for lagged output $Y_{i,t-1}$, developer characteristics $\mathbf{X}_{it}$, and team characteristics $\mathbf{S}_{st}$. Reviewer effects are captured through a vector of indicators $\mathbf{T_j}$,

Figure E1: Reviewer Value-Added: Distribution, Correlation, and Validation

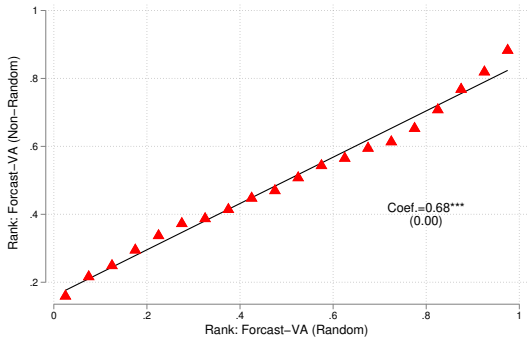
(a) Distribution of VA Estimates



(b) Correlation of VA Measures



(c) Validation of VA Rankings



Notes: Panels (a) and (b) use the automatic assignment sample. **Panel (a)** plots the empirical distribution of reviewer value-added (VA) estimated from developer code quantity and code quality. **Panel (b)** plots the relationship between reviewer VA based on code quantity and reviewer VA based on code quality. **Panel (c)** compares VA rankings for reviewers who appear in both sample. Each dot is a bin of reviewers; the slope and standard error come from a regression of one ranking on the other. The validation sample contains 6,952 reviewers.

with coefficients $\boldsymbol{\theta}$ representing reviewer value-added. From this regression, I obtain residuals Y^*_{isjt} by subtracting the fitted values excluding reviewer fixed effects, as shown in Equation (8). I then compute $\bar{Y}^*_{jt}$, the average residual for reviewer j in week t, defined in Equation (9). Finally,

Equation (10) forecasts reviewer VA γ_{jt} based on the leave-one-week-out residual averages $\bar{\mathbf{Y}}^*_{\mathbf{jt}-}$. Throughout, t indexes the assignment week. The outcome Y_{isjt} is developer output in week $t+1$, the first full week after the review, and the lagged control $Y_{i,t-1}$ is output in the week before assignment. Week t itself is used only for assignment and is excluded from both, since code produced in that week may predate the review.

$$Y_{isjt} = \beta_0 + Y_{i,t-1}\beta_1 + \mathbf{X_{it}}'\boldsymbol{\beta_2} + \mathbf{S_{st}}'\boldsymbol{\beta_3} + \mathbf{T_j}'\boldsymbol{\theta} + \varepsilon_{isjt} \tag{7}$$

$$Y^*_{isjt} = Y_{isjt} - \left[\beta_0 + Y_{i,t-1}\beta_1 + \mathbf{X_{it}}'\boldsymbol{\beta_2} + \mathbf{S_{st}}'\boldsymbol{\beta_3}\right] \tag{8}$$

$$\bar{Y}^*_{jt} = \frac{1}{n_{jt}} \sum_{i \in \{i:j(i,t)=j\}} Y^*_{isjt} \tag{9}$$

$$\gamma_{jt} = \pi_0 + \overline{\mathbf{Y}}^{*\prime}_{j,-t}\boldsymbol{\pi}_1 + u_{jt} \tag{10}$$

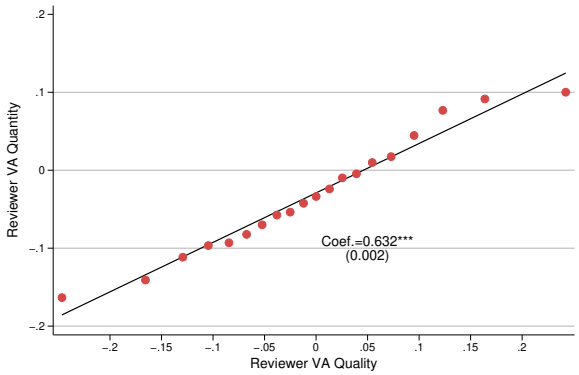
The forecast-based estimator offers several advantages for estimating reviewer value-added. First, instead of including reviewer fixed effects directly in the regression, the model omits them from the fitted values. This procedure yields a modified residual, Y^*_{isjt}, defined in Equation (8), that preserves the full reviewer contribution while removing variation explained by past developer output and other covariates. By residualizing the outcome without partialling out reviewer fixed effects, I avoid the shrinkage toward zero that would arise from estimating $\boldsymbol{\theta}$ directly for reviewers observed in only a few weeks. Second, to avoid overfitting and make use of prior information, the model predicts reviewer value-added in week t using the average residuals from all other weeks (a leave-week-out approach). This produces $\hat{\gamma}_{jt}$, the best linear predictor of reviewer j's impact in week t. Third, Equation (10) allows the reviewer VA to vary over time.

E.3 Predicting Reviewer Value-Added from Feedback

To predict reviewer value-added, I apply the Extreme Gradient Boosting (XGBoost) algorithm. The goal is descriptive: I ask how much independently measured feedback characteristics predict independently estimated reviewer VA. For the continuous outcome, the model uses squared error loss (*reg:squarederror*) and is evaluated using out-of-sample R^2. I randomly split the data into a training set (80%) and a hold-out test set (20%). Model training and tuning are performed on the training set using five-fold cross-validation.

Hyperparameters are selected using a two-stage procedure: a random search explores a wide parameter space, followed by a grid search that refines the best-performing regions. I repeat the tuning process separately for each predictor set. Figure E4 examines how feedback explains variation in reviewer value-added, measured by changes in developer code quality before and after assignment. Panel (a) uses the random-assignment sample, while Panel (b) uses the full sample, including both random and non-random assignments. Reviewer value-added is estimated using

Figure E2: Reviewer Value-Added Based on Developer Code Quantity vs. Code Quality

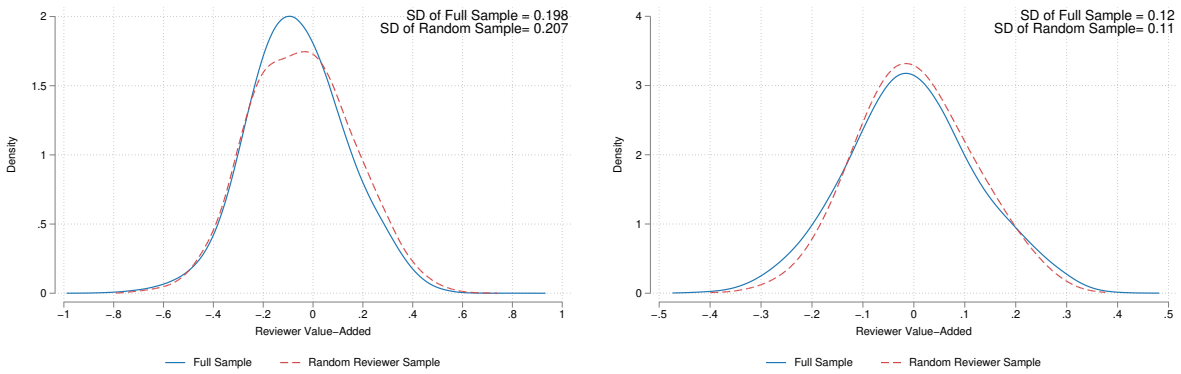


Notes: This figure compares reviewer VA estimates based on two different developer outcomes: code quantity (y-axis) and code quality (x-axis) in the full sample. Reviewer VA is estimated using the forecast-based estimator by Chetty, Friedman, and Rockoff (2014). The fitted line shows a positive and statistically significant relationship between the two measures, indicating that reviewers who increase developer output also tend to improve code quality.

Figure E3: Reviewer VA Distribution: Full Sample versus Automatic Assignment Sample

(a) VA based on Code Quantity

(b) VA based on Code Quality



Notes: This figure compares the distributions of reviewer VA measured in the random and full samples, using code quantity and code quality, respectively.

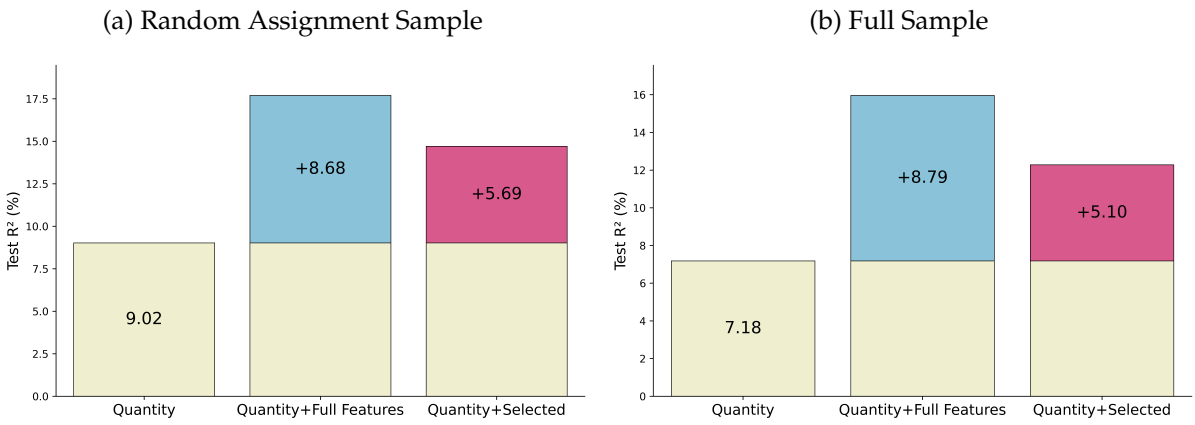
the forecast-based estimator of Chetty et al. (2014).

I compare four sets of predictors. The first is reviewer demographics, measured by gender and race. The second is feedback quantity, measured as the total number of messages reviewers send per week. The third is full feedback text, represented by 768-dimensional semantic embeddings. The fourth is feedback quantity combined with selected feedback features, defined as the weekly shares of toxic, positive, and constructive feedback per reviewer.

The results show that feedback explains substantial variation in reviewer value-added. In

the random sample, feedback quantity alone accounts for about 9 pp of the variance. Adding embeddings increases explanatory power by roughly 18 pp. Using only the selected features captures most of this improvement. The full sample yields similar patterns.

Figure E4: Predicting Reviewer Value-Added from Feedback: Developer Code Quality



Notes: This figure shows how different predictors explain variation in reviewer value-added (VA), measured by the reviewer's effect on developer code quality. Each bar reports out-of-sample R^2. *Demographics* includes reviewer gender and race. *Quantity* is the number of feedback messages written. *Quantity + Full Text* adds 768-dimensional text embeddings. *Quantity + Selected Feedback* adds the weekly shares of toxic, positive, and constructive feedback per reviewer. Values on the bars indicate the improvement in explanatory power relative to the quantity-only model.

F Employment Career Outcomes

Matching GitHub and LinkedIn The goal is to build a panel on developers' messages, activities, and employers by merging GitHub with Revelio Labs. I obtain names, employers, biographies, profile images, locations, and emails from the GitHub API. Because profiles are used for career search, developers tend to report accurate details (El-Komboz and Goldbeck, 2024). In the automatic assignment sample, about 70% of developers list a LinkedIn profile, personal website, or other social account on GitHub, and many reuse the same photo, which aids identification.

The matching process proceeds in three steps. First, I create direct matches for developers who link their LinkedIn URL on GitHub or vice versa. Second, for unmatched cases, I construct a match pool using names, locations, employment histories, and biographies. To account for naming variations, I match on common nicknames (for example, "Alex" for "Alexander" or "Alexandra"), abbreviated names (for example, "Tina L." for "Tina Liu"), and names with or without middle names (for example, "Tina Jin Liu" simplified to "Tina Liu"). The pool includes exact matches, first name with last initial matches, and matches involving middle names; developers with entirely different names or typos are excluded. Third, I incorporate additional characteristics such as position, firm, and geographic location to identify the best LinkedIn candidate for each GitHub developer. A research assistant reviews all candidates in the pool and selects the best match. I also tested an LLM-based approach (GPT-4o-mini and DeepSeek R1) to select matches, but its performance was inferior to manual review.

Selection into LinkedIn Table F1 compares developers with and without a matched LinkedIn profile. Of the 29,210 developers in the main sample, 7,387 (25%) are matched. Matched developers are more active and visible on GitHub: they have nearly twice as many followers, first appear in the activity data about half a year earlier, and record approximately 61% more activity in the calendar year before the feedback event. They are also more likely to share their personal information on their GitHub profiles, consistent with the matching procedure's reliance on this information. The share classified as male is 4.1 pp higher among matched developers. The LinkedIn sample thus disproportionately represents developers who are more active, more visible, and disclose more profile information. The career results in Figure 9b should be interpreted with this selection in mind and may not generalize to the full developer sample.

Table F1: Selection into the LinkedIn-Matched Sample

	No LinkedIn (1)	LinkedIn (2)	Difference (2)−(1)	p-value
Number of followers	121.0	224.9	103.9***	0.000
First year recorded activity	2018.6	2018.1	-0.5***	0.000
Total activities in the past year	510.0	821.7	311.8***	0.000
Male	0.860	0.901	0.041***	0.000
White	0.757	0.759	0.002	0.804
Asian	0.187	0.176	-0.012*	0.093
Has name on profile	0.801	0.984	0.183***	0.000
Has location on profile	0.478	0.777	0.299***	0.000
Observations	21,823	7,387		

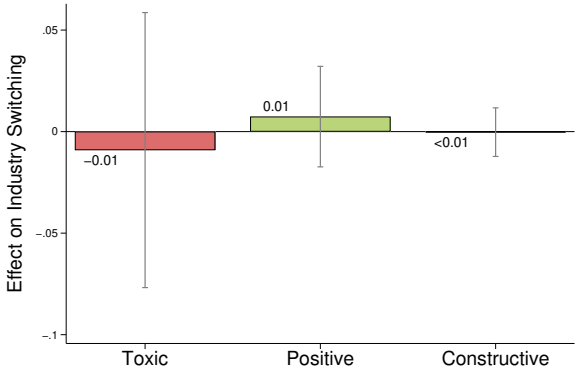
Notes: The sample is all developers in the main 2SLS sample. Column (1) reports means for developers without a matched LinkedIn profile and column (2) for developers with one; column (3) is the difference and column (4) the p-value from a two-sided Welch t-test. *Number of followers* is the developer's GitHub follower count. *First year recorded activity* is the first year the developer appears in the GitHub activity data. *Total activities in the past year* is the number of GitHub events by the developer in the calendar year before the feedback event. *Male* is predicted from the profile image; *White* and *Asian* are predicted from the developer's name and are missing when no name is available; *White* corresponds to the "Greater European" category of the name classifier. *Has name* and *Has location* indicate whether the field is filled in on the developer's GitHub profile. * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$.

Retention Firm switching is measured from LinkedIn employment spells. For each developer-week, I identify the LinkedIn employer active in that week and define an employer switch as the start of a subsequent LinkedIn position at a different employer within 365 days. I define an occupation switch analogously, as a subsequent position in a different SOC major group (the first two digits of the O*NET-SOC code) within 365 days; occupation measures the worker's role, unlike NAICS, which measures the employer's industry. I define a broad-industry switch as a subsequent position in a different one-digit NAICS sector group (the leading digit of the six-digit NAICS code) within 365 days. The one-digit grouping is coarse: leading digit 5, for example, spans Information (51), Finance (52), Real Estate (53), Professional, Scientific and Technical Services (54), Management (55), and Administrative Services (56), so moves within that block are not counted as industry switches. These measures capture transitions in the developer's reported LinkedIn career history; they do not require the LinkedIn employer to be the GitHub repository owner. A switch is coded only over a fully observed follow-up window, so a developer-week whose 365-day horizon extends past the end of the LinkedIn panel is treated as missing rather than as a non-switch.

When a developer holds several overlapping LinkedIn positions in the same week, I collapse them to a single person-week so that a move to a *simultaneous* position is not counted as a switch. The employer-switch result is robust to keying the switch on Revelio's canonical firm identifier (`rcid`) rather than the employer name. Figure F1 reports the effects of feedback on broad-industry switches, and Figure 9b panel (b) reports occupation switches. Neither is significantly affected by any feedback type.

Figure F1: Effect of Feedback on Industry Switching

Dep Mean: 0.017



Notes: This figure reports IV estimates of the effects of toxic, positive, and constructive feedback on the probability that a developer switches broad industries within one year. Industry switches are measured from LinkedIn career histories. Feedback exposure is instrumented using leave-developer-out reviewer feedback styles in the automatic assignment sample. Vertical lines report 90% confidence intervals.

Backgrounds and Career Outcomes of Toxic and Non-Toxic Reviewers

Table F2: Pre-Reviewer Histories of Toxic and Non-Toxic Reviewers

	Never toxic (1)	Repeat toxic (2)	Difference (3)	Adjusted difference (4)	N (5)
Panel A. Activity before first review					
Months of history	17.3	18.3	0.935**	1.868***	8,444
Cases authored	49.9	70.2	20.3***	6.5*	8,444
Feedback messages received	861.5	1,214.6	353.1***	3.7	8,444
Panel B. Exposure to toxicity before first review					
Received any toxic feedback	0.236	0.355	0.119***	0.079***	8,444
Share of received feedback toxic	0.0020	0.0035	0.0015***	0.0017***	8,357
Toxicity of randomly assigned reviewers	0.0072	0.0118	0.0046***	0.0056***	6,264
Received toxic feedback	0.038	0.059	0.021***	0.022***	6,264
Toxic rate in teams worked in	0.0017	0.0034	0.0016***	0.0016***	8,296

Notes: The sample is reviewers in the random-assignment sample with at least 20 review feedback messages whose first review occurs in 2018 or later and who have GitHub activity before it. *Never toxic* reviewers wrote no toxic feedback; *repeat toxic* reviewers wrote two or more. All measures are computed before the reviewer's first review. *Toxicity of randomly assigned reviewers* is the average leave-one-out toxic rate of the reviewers assigned to the individual's own cases in the random-assignment sample. *Toxic rate in team worked in* is the toxic share of review feedback in teams where the individual was active. Column (4) reports the coefficient on the repeat-toxic indicator from an OLS regression that controls for the log number of review feedback messages and first-review-year fixed effects. Stars in column (3) are from a two-sided Welch t-test. * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$.

Table F3: Professional Background of Toxic and Non-Toxic Reviewers

	Never toxic (1)	Repeat toxic (2)	Difference (3)	Adjusted difference (4)	N (5)
Years of work experience	11.9	11.3	-0.504*	-0.735**	3,719
At a large technology firm	0.256	0.262	0.006	-0.035*	3,706
Located in the United States	0.435	0.436	0.001	-0.014	3,706
Master's degree or above	0.413	0.424	0.010	0.011	3,788

Notes: The sample is reviewers in the estimation sample with at least 20 review feedback messages who are matched to a LinkedIn career history. *Never toxic* reviewers wrote no toxic feedback; *repeat toxic* reviewers wrote two or more. All characteristics are measured at the reviewer's first review in the sample. Column (4) reports the coefficient on the repeat-toxic indicator from an OLS regression that controls for the log number of review feedback messages. Stars in column (3) are from a two-sided Welch t-test. * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$.

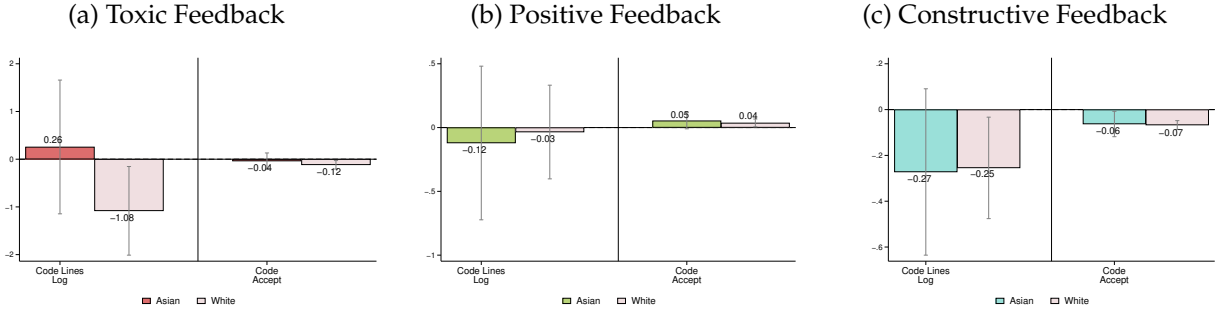
Table F4: Career Outcomes of Toxic and Non-Toxic Reviewers within Three Years

	Non-toxic (1)	Toxic (2)	Difference (3)	Adjusted difference (s.e.) (4)	N (5)
Promoted (any employer)	0.215	0.241	0.025	0.034 (0.026)	1,953
Promoted within the same firm	0.113	0.110	-0.003	0.003 (0.020)	1,953
Switched employer	0.323	0.403	0.080***	0.076** (0.031)	1,953

Notes: The sample is reviewers matched to a LinkedIn career history who wrote at least 10 review feedback messages in the 12 months after their first review in the sample (t_0), held a position at t_0, and are observed for three years thereafter. Outcomes are measured over the following three years. Promotion indicates a move to a higher seniority level, and switching employer indicates a new position at a different firm. Column (4) reports OLS estimates controlling for the log number of review feedback messages and years of experience at t_0, and year-of-t_0 fixed effects; robust standard errors are in parentheses. Stars in column (3) are from two-sided Welch t-tests. $p < 0.10$, $p < 0.05$, $p < 0.01$.

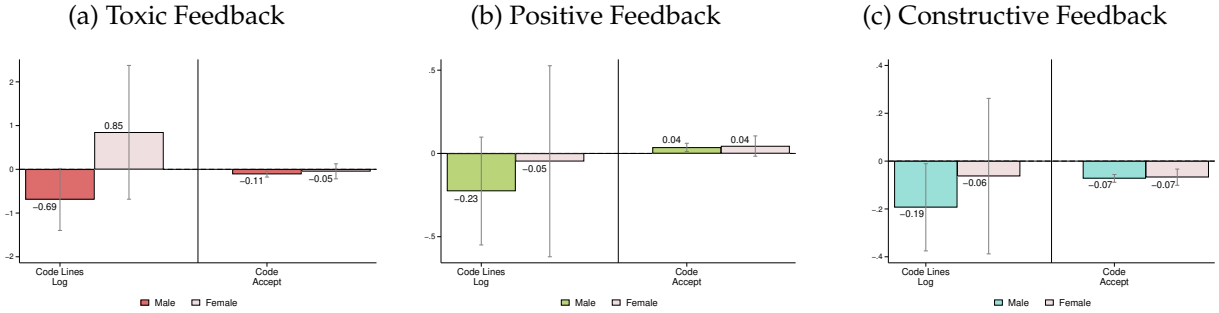
G Exploratory Heterogeneity and Additional Outcomes

Figure G1: Heterogeneous Effects of Feedback by Developer Race (Asian versus White)



Notes: This figure shows the heterogeneous effects of feedback on developer outcomes during the one to four weeks following feedback, separately by developer race. Dark bars denote Asian developers, and light bars denote White developers. The x-axis reports, from left to right, effects on the intensive margin of code production, measured by log lines of code, and on code quality, measured by the share of new lines accepted. Each estimate comes from a separate 2SLS regression for the indicated subsample. All specifications follow the baseline specification in Table 5. Vertical lines show 90% confidence intervals based on standard errors clustered at the reviewer level.

Figure G2: Heterogeneous Effects of Feedback by Developer Gender



Notes: This figure shows the heterogeneous effects of feedback on developer outcomes in the 1–4 weeks following feedback, by developer gender. Panels (a)–(c) report estimates for toxic, positive, and constructive feedback, respectively. Dark bars represent male developers; light bars represent female developers. The x-axis reports, from left to right, effects on the intensive margin of code production, measured by log lines of code, and on code quality, measured by the share of new lines accepted. Each estimate comes from a separate 2SLS regression for the indicated subsample. All specifications follow the baseline specification in Table 5. Vertical lines show 90% confidence intervals based on standard errors clustered at the reviewer level.

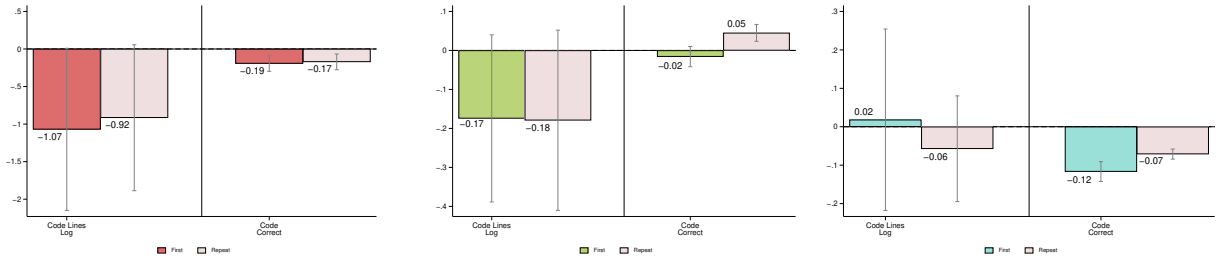
Table G1: Assessing Bias in On-Platform Feedback Effects

	Interaction	s.e.(Interaction)	p-value
Toxic Feedback			
Code Quantity Ever	0.199	0.176	0.261
Code Quantity Log	−1.625	1.584	0.305
Code Quality	0.206	0.165	0.210
Case Quality	0.082	0.202	0.684
Non-Code Quantity Ever	0.033	0.101	0.744
Non-Code Quantity Log	0.557	0.637	0.382
Positive Feedback			
Code Quantity Ever	−0.004	0.067	0.949
Code Quantity Log	0.685	0.451	0.129
Code Quality	−0.058	0.052	0.263
Case Quality	−0.000	0.068	1.000
Non-Code Quantity Ever	0.014	0.037	0.714
Non-Code Quantity Log	0.113	0.202	0.576
Constructive Feedback			
Code Quantity Ever	0.033	0.034	0.326
Code Quantity Log	−0.369	0.236	0.118
Code Quality	0.019	0.026	0.463
Non-Code Quantity Ever	−0.014	0.018	0.439
Non-Code Quantity Log	0.251**	0.113	0.026
Case Quality	−0.012	0.036	0.737
Team × Year FE	✓	✓	✓
Decile t–1 activity	✓	✓	✓
Decile t–1 message	✓	✓	✓

Notes: The "Interaction" column reports the coefficient on $I^{\theta}_{im,t} \times C_{im}$. The Co-located dummy C_{im} equals 1 if all team members are in the same country. See Appendix A.6 for details on how team location is constructed. Outcomes are measured 1–4 weeks after the developer received feedback. Standard errors are clustered at the reviewer level. * $p < .10$, ** $p < .05$, *** $p < .01$.

Figure G3: Heterogeneous Effects of Feedback by First vs. Repeat Exposure

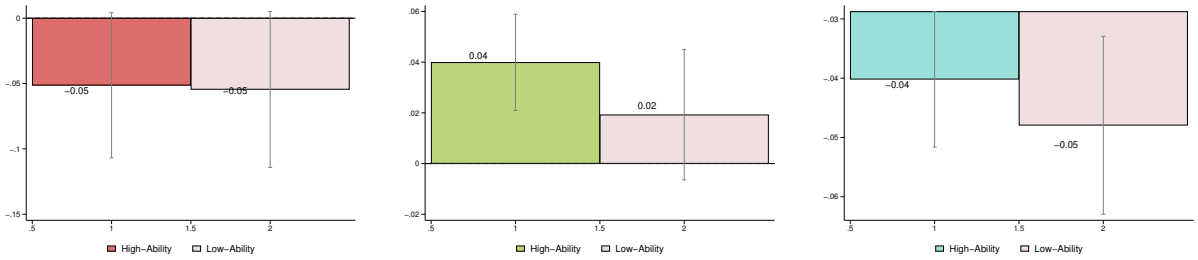
(a) Toxic Feedback (b) Positive Feedback (c) Constructive Feedback



Notes: This figure shows the heterogeneous effects of reviewer feedback by whether the feedback was given on a developer's first time ("First") or later ("Repeat"). Dark bars represent first-time feedback; light bars represent later feedback. The x-axis reports, from left to right, effects on the intensive margin of code production, measured by log lines of code, and on code quality, measured by the share of new lines accepted. Each estimate comes from a separate 2SLS regression for the indicated subsample. All specifications follow the baseline specification in Table 5. Vertical lines show 90% confidence intervals based on standard errors clustered at the reviewer level.

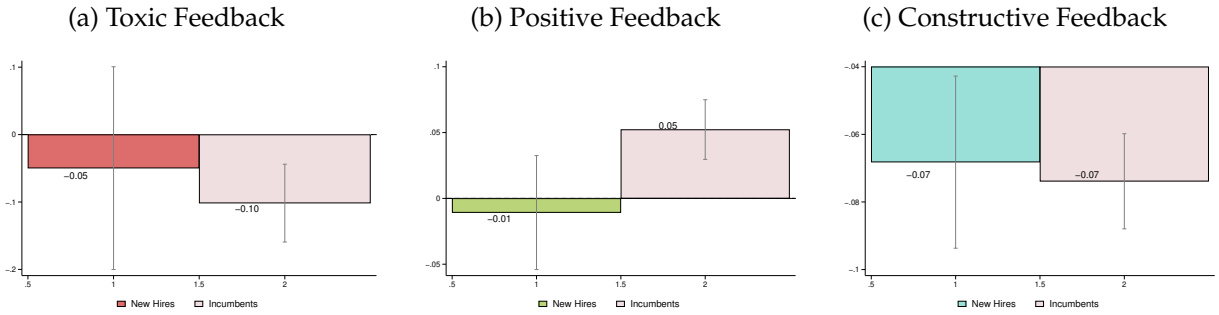
Figure G4: Heterogeneous Effects of Feedback by Developer Ability

(a) Toxic Feedback (b) Positive Feedback (c) Constructive Feedback



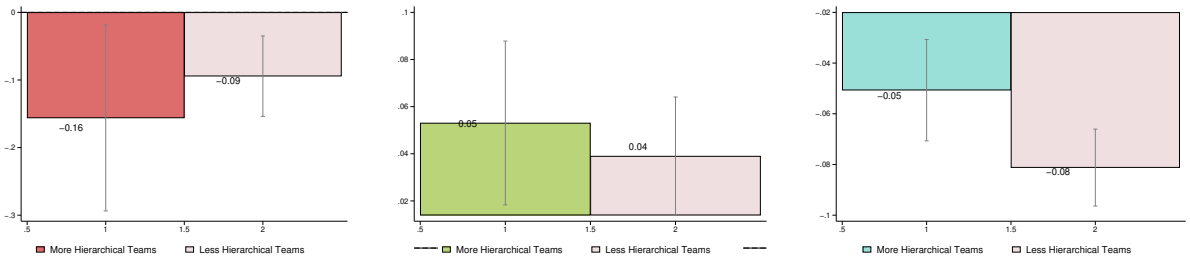
Notes: This figure shows how the effects of reviewer feedback vary by developer ability, defined using lagged code quality. Developers are split into high- and low-ability groups based on their past performance. Dark bars represent high-ability developers; light bars represent low-ability developers. The x-axis reports, from left to right, effects on the intensive margin of code production, measured by log lines of code, and on code quality, measured by the share of new lines accepted. Each estimate comes from a separate 2SLS regression for the indicated subsample. All specifications follow the baseline specification in Table 5. Vertical lines show 90% confidence intervals based on standard errors clustered at the reviewer level.

Figure G5: Heterogeneous Effects of Feedback by New Developers vs. Incumbents



Notes: This figure shows the heterogeneous effects of reviewer feedback by worker status as a new hire (within the first two months on a team) or as an incumbent (longer than two months). Dark bars represent new developers; light bars represent incumbents. The x-axis reports, from left to right, effects on the intensive margin of code production, measured by log lines of code, and on code quality, measured by the share of new lines accepted. Each estimate comes from a separate 2SLS regression for the indicated subsample. All specifications follow the baseline specification in Table 5. Vertical lines show 90% confidence intervals based on standard errors clustered at the reviewer level.

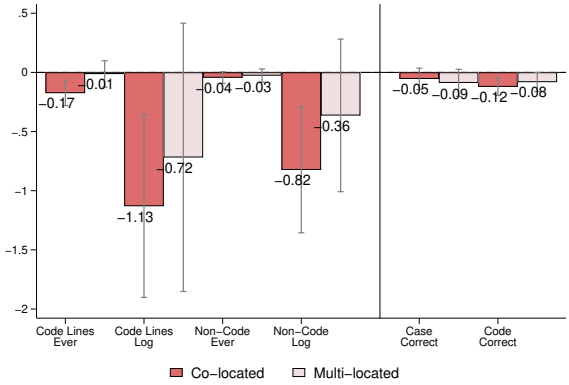
Figure G6: Heterogeneous Effects in More vs. Less Hierarchical Teams



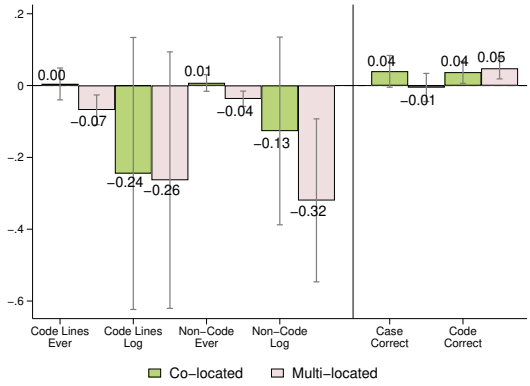
Notes: The figure shows heterogeneous effects by team hierarchy score (see Section 3.4). Dark bars represent hierarchical teams with a score greater than 0.5; light bars represent flat teams. The x-axis reports, from left to right, effects on the intensive margin of code production, measured by log lines of code, and on code quality, measured by the share of new lines accepted. Each estimate comes from a separate 2SLS regression for the indicated subsample. All specifications follow the baseline specification in Table 5. Vertical lines show 90% confidence intervals based on standard errors clustered at the reviewer level.

Figure G7: Heterogeneous Effects of Feedback by Multi-location Team Structure

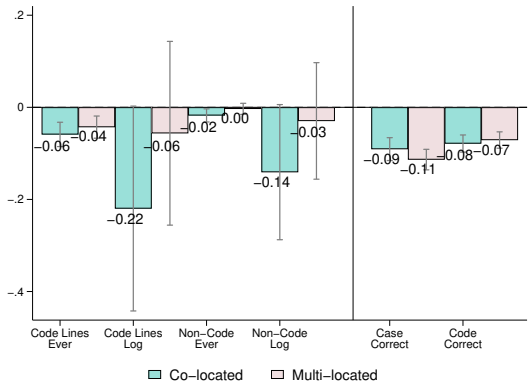
(a) Toxic Feedback



(b) Positive Feedback



(c) Constructive Feedback

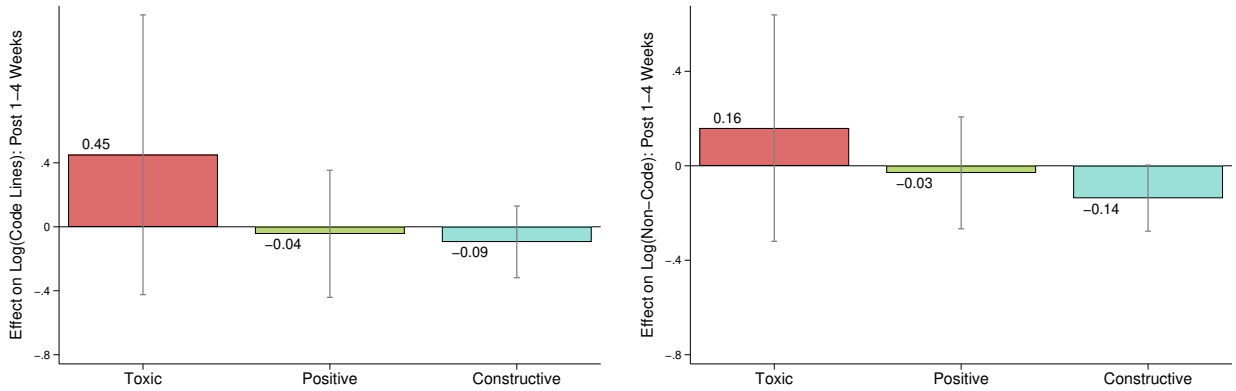


Notes: This figure splits teams by whether the share of members in the most represented country is above or below the sample median. This is a coarser, more balanced split than the co-location definition used in Table G1, where a team is co-located only if *all* members share a country. Dark bars: co-located teams (above-median share). Light bars: multi-located teams (below-median share). I estimate the effect in each subsample. Baseline sample of cases for the automatic assignment sample from 2017 to 2023. Controls include all variables listed in Table 5. Standard errors are clustered at the reviewer level. Vertical lines show 90% confidence intervals.

Figure G8: Effect of Feedback on Developer Outcomes in Other Teams

(a) Code Quantity

(b) Non-Code Quantity

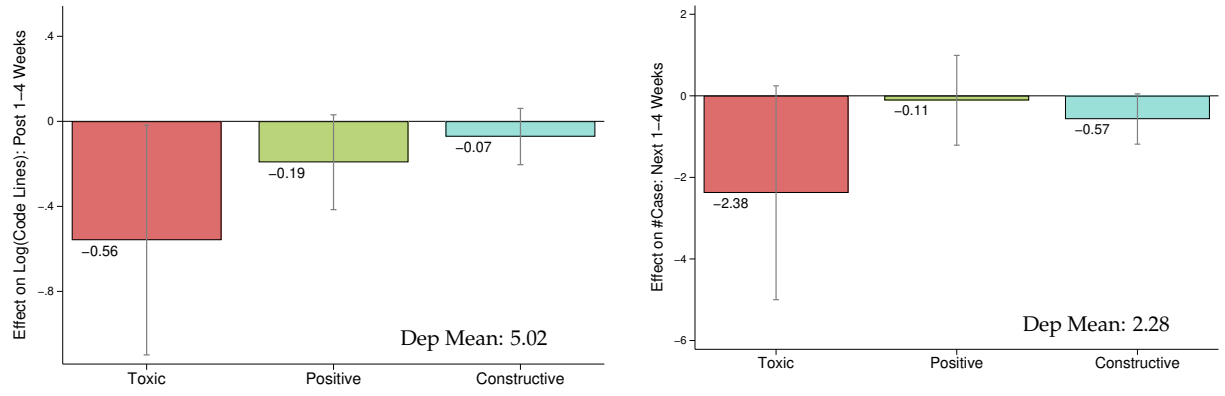


Notes: This figure shows the 2SLS estimates of receiving feedback in a focal team on a developer's productivity in *other* teams during the following 1–4 weeks. Standard errors are clustered at the reviewer level. Lines report 90% confidence intervals.

Figure G9: Effect of Feedback on Code Quantity and Number of Cases

(a) Code Quantity

(b) Number of Cases



Notes: This figure reports 2SLS estimates of the effect of a given feedback type on developer outcomes within the same team during weeks 1–4 after the feedback. The sample includes teams with random reviewer assignment, at least two reviewers per year, and reviewers with at least 10 cases per year. Standard errors are clustered by reviewer, and lines show 90% confidence intervals.